

제 68 회 서울과학전람회 작품설명서

부 록

시뮬레이션 전체 소스코드

병원체의 공간 분포와 화학주성 신호가 면역세포 이동 전략의 효율에 미치는 영향

부록 안내

본 부록은 탐구에 사용한 시뮬레이션 프로그램의 전체 소스코드를 수록한 것으로, 제 3 자가 동일한 결과를 재현할 수 있도록 생략 없이 원문 그대로 제시하였다.

모든 코드는 Python 3.12 환경에서 작성되었으며, 수치 계산에 NumPy, 통계 검정에 SciPy, 시각화에 Matplotlib 을 사용하였다. 각 파일의 첫머리에는 해당 모형이 참고한 선행 연구의 서지사항이 주석으로 기록되어 있으며, 파라미터마다 그 값의 출처가 되는 논문을 태그로 표시하였다. 논문에서 직접 인용한 실측값과, 정성적 기전만 문헌에 근거하고 정량값은 모형에서 설정한 가정값은 코드 내에서 구분하여 표기하였다.

두 실험 조건(정상적 환경 / 염증성 환경)은 이동 방향을 결정하는 규칙에서만 차이가 있으며, 격자 구조·시간 축척·면역세포 초기 개체수·병원체 파라미터·개체수 축소비율($\kappa = 1/1000$)은 완전히 동일하게 통제하였다. 이 차이가 구현된 부분은 각 파일의 MovementEngine 클래스이다.

수록 파일 목록

구분	내용	파일명	줄 수
부록 A	인플루엔자 A 감염 모형 — 정상적 환경 (무작위 순찰)	immune_influenza_random.py	2,389
부록 B	인플루엔자 A 감염 모형 — 염증성 환경 (주화성 유도 이동)	immune_influenza_inflammatory.py	2,365
부록 C	폐렴사슬알균 감염 모형 — 정상적 환경 및 염증성 환경 통합	immune_pneumococcus.py	1,342
부록 D	폐렴사슬알균 반복 시행 실행 스크립트	run_replicates_pneumococcus.py	50
부록 E	인플루엔자 A 반복 시행 실행 스크립트	run_replicates_influenza.py	44
합계			6,190

부록 A. 인플루엔자 A 감염 모형 — 정상적 환경 (무작위 순찰)

면역세포가 화학신호를 감지하지 않고 균일난수 방향으로 이동하는 조건의 Agent-Based Model 전체 소스코드. 선행 연구에서 도출한 파라미터, 격자·시간 축척 변환, 감염 동역학, 선천·후천 면역반응, 결과 분석 및 문헌 대조 검증 루틴을 포함한다.

파일명	immune_influenza_random.py	총 줄 수	2,389 줄
실행 환경	Python 3.12 / NumPy · SciPy · Matplotlib	난수 시드	700000, 700037, 700074, 700111, 700148

```

1 |#!/usr/bin/env python3
2 |# -*- coding: utf-8 -*-
3 |"""
4 |=====
5 | immune_random_walk_abm.py
6 | 면역세포 무작위 이동(Random Walk) 기반 감염-면역 반응 Agent-Based Model
7 |-----
8 | 연구 주제 : "면역세포는 무작위로 이동하는 것이 효과적인가,
9 |             체계적으로 이동하는 것이 효과적인가?"
10 | 본 버전 : 무작위 이동(RANDOM WALK) 조건 단독 실험 (1회 실행)
11 |           -> 체계적 이동(systematic) 모델은 MovementEngine 만 교체하여
12 |             동일 생물학 조건에서 후속 비교 예정
13 |
14 | 대표 병원체 : Influenza A virus (H1N1)
15 | 모델 도메인 : 상기도(URT) 상피 조직 슬랩 + 배액 림프절(lymph node) 구획
16 |
17 | 축소 원칙 : 실제값 -> 논문 출처 -> 축소비율 -> 시뮬레이션값
18 |             모든 개체 (상피세포/면역세포/바이러스)에 동일 축소비 kappa = 1/1000 적용
19 |             논문에서 값을 찾지 못한 항목은 "모델 가정값"으로 명시
20 |
21 | 주의 : 본 스크립트에 기재된 DOI/PMID 는 작성 시점 기준이며,
22 |       보고서에 인용하기 전 PubMed 에서 최종 확인 권장.
23 |=====
24 |"""
25 |
26 | from __future__ import annotations
27 |
28 | import argparse
29 | import json
30 | import math
31 | import os
32 | import sys
33 | import time
34 | from dataclasses import dataclass, field
35 | from typing import Dict, List, Optional, Tuple
36 |
37 | import numpy as np
38 |
39 | # =====
40 | # SECTION 1. LITERATURE REFERENCE DATABASE (논문 근거 데이터베이스)

```

```

41 | # =====
42 |
43 |
44 | @dataclass(frozen=True)
45 | class Ref:
46 |     """논문 1 건의 서지 정보"""
47 |     tag: str
48 |     authors: str
49 |     title: str
50 |     journal: str
51 |     year: int
52 |     doi: str = ""
53 |     pmid: str = ""
54 |
55 |     def short(self) -> str:
56 |         return f"{self.authors.split(',')[0]} et al., {self.journal} {self.year}"
57 |
58 |     def full(self) -> str:
59 |         ident = []
60 |         if self.doi:
61 |             ident.append(f"DOI:{self.doi}")
62 |         if self.pmid:
63 |             ident.append(f"PMID:{self.pmid}")
64 |         return (f"{self.authors}. \"{self.title}\" "
65 |                f"{self.journal} ({self.year}). {' '.join(ident)}")
66 |
67 |
68 | REFS: Dict[str, Ref] = {
69 |     "baccam2006":
70 |         Ref("baccam2006",
71 |             "Baccam P, Beauchemin C, Macken CA, Hayden FG, Perelson AS",
72 |             "Kinetics of influenza A virus infection in humans",
73 |             "Journal of Virology 80(15):7590-7599", 2006,
74 |             "10.1128/JVI.01623-05", "16840338"),
75 |     "carrat2008":
76 |         Ref("carrat2008",
77 |             "Carrat F, Vergu E, Ferguson NM, Lemaître M, Cauchemez S, Leach S, Valleron AJ",
78 |             "Time lines of infection and disease in human influenza: "
79 |             "a review of volunteer challenge studies",
80 |             "American Journal of Epidemiology 167(7):775-785", 2008,
81 |             "10.1093/aje/kwm375", "18230677"),
82 |     "hayden1998":
83 |         Ref("hayden1998",
84 |             "Hayden FG, Fritz R, Lobo MC, Alvord W, Strober W, Straus SE",
85 |             "Local and systemic cytokine responses during experimental human "
86 |             "influenza A virus infection",
87 |             "Journal of Clinical Investigation 101(3):643-649", 1998,
88 |             "10.1172/JCI1355", "9449698"),
89 |     "harris2006":
90 |         Ref("harris2006",
91 |             "Harris A, Cardone G, Winkler DC, Heymann JB, Brecher M, White JM, Steven AC",
92 |             "Influenza virus pleiomorphy characterized by cryoelectron tomography",
93 |             "PNAS 103(50):19123-19127", 2006,

```

94 | "10.1073/pnas.0607614103", "17146053"),
 95 | "miller2002":
 96 | Ref("miller2002", "Miller MJ, Wei SH, Parker I, Cahalan MD",
 97 | "Two-photon imaging of lymphocyte motility and antigen response "
 98 | "in intact lymph node", "Science 296(5574):1869-1873", 2002,
 99 | "10.1126/science.1070051", "12016203"),
 100 | "miller2003":
 101 | Ref("miller2003", "Miller MJ, Wei SH, Cahalan MD, Parker I",
 102 | "Autonomous T cell trafficking examined in vivo with intravital "
 103 | "two-photon microscopy", "PNAS 100(5):2604-2609", 2003,
 104 | "10.1073/pnas.2628040100", "12601158"),
 105 | "lammermann2013":
 106 | Ref("lammermann2013",
 107 | "Lammermann T, Afonso PV, Angermann BR, Wang JM, Kastenmuller W, "
 108 | "Parent CA, Germain RN",
 109 | "Neutrophil swarms require LTB4 and integrins at sites of cell death in vivo",
 110 | "Nature 498(7454):371-375", 2013, "10.1038/nature12175", "23708969"),
 111 | "ariotti2012":
 112 | Ref("ariotti2012",
 113 | "Ariotti S, Beltman JB, Chodaczek G, Hoekstra ME, van Beek AE, "
 114 | "Gomez-Eerland R, et al.",
 115 | "Tissue-resident memory CD8+ T cells continuously patrol skin "
 116 | "epithelia to quickly recognize local antigen", "PNAS 109(48):19739-19744",
 117 | 2012, "10.1073/pnas.1208927109", "23150545"),
 118 | "mempel2004":
 119 | Ref("mempel2004", "Mempel TR, Henrickson SE, von Andrian UH",
 120 | "T-cell priming by dendritic cells in lymph nodes occurs in three "
 121 | "distinct phases", "Nature 427(6970):154-159", 2004,
 122 | "10.1038/nature02238", "14712275"),
 123 | "lawrence2005":
 124 | Ref("lawrence2005", "Lawrence CW, Ream RM, Braciale TJ",
 125 | "Frequency, specificity, and sites of expansion of CD8+ T cells "
 126 | "during primary pulmonary influenza virus infection",
 127 | "Journal of Immunology 174(9):5332-5340", 2005,
 128 | "10.4049/jimmunol.174.9.5332", "15843530"),
 129 | "wrammert2008":
 130 | Ref("wrammert2008",
 131 | "Wrammert J, Smith K, Miller J, Langley WA, Kokko K, Larsen C, et al.",
 132 | "Rapid cloning of high-affinity human monoclonal antibodies against "
 133 | "influenza virus", "Nature 453(7195):667-671", 2008,
 134 | "10.1038/nature06890", "18449194"),
 135 | "kaech2002":
 136 | Ref("kaech2002", "Kaech SM, Wherry EJ, Ahmed R",
 137 | "Effector and memory T-cell differentiation: implications for vaccine "
 138 | "development", "Nature Reviews Immunology 2(4):251-262", 2002,
 139 | "10.1038/nri778", "11970378"),
 140 | "regoes2007":
 141 | Ref("regoes2007", "Regoes RR, Barber DL, Ahmed R, Antia R",
 142 | "Estimation of the rate of killing by cytotoxic T lymphocytes in vivo",
 143 | "PNAS 104(5):1599-1603", 2007, "10.1073/pnas.0508830104", "17242364"),
 144 | "ganusov2008":
 145 | Ref("ganusov2008", "Ganusov VV, De Boer RJ",
 146 | "Estimating in vivo death rates of targets due to CD8 T-cell-mediated "

147 "killing", "Journal of Virology 82(23):11749-11757", 2008,
 148 "10.1128/JVI.01128-08", "18815293"),
 149 "alanio2010":
 150 Ref("alanio2010", "Alanio C, Lemaitre F, Law HKW, Hasan M, Albert ML",
 151 "Enumeration of human antigen-specific naive CD8+ T cells reveals "
 152 "conserved precursor frequencies", "Blood 115(18):3718-3725", 2010,
 153 "10.1182/blood-2009-10-251124", "20220118"),
 154 "blattman2002":
 155 Ref("blattman2002",
 156 "Blattman JN, Antia R, Sourdive DJD, Wang X, Kaech SM, Murali-Krishna K, "
 157 "Altman JD, Ahmed R",
 158 "Estimating the precursor frequency of naive antigen-specific CD8 T cells",
 159 "Journal of Experimental Medicine 195(5):657-664", 2002,
 160 "10.1084/jem.20001021", "11877489"),
 161 "bisset2004":
 162 Ref("bisset2004", "Bisset LR, Lung TL, Kaelin M, Ludwig E, Dubs RW",
 163 "Reference values for peripheral blood lymphocyte phenotypes "
 164 "applicable to the healthy adult population in Switzerland",
 165 "European Journal of Haematology 72(3):203-212", 2004,
 166 "10.1046/j.0902-4441.2003.00199.x", "14962239"),
 167 "pillay2010":
 168 Ref("pillay2010",
 169 "Pillay J, den Braber I, Vrisekoop N, Kwast LM, de Boer RJ, Borghans JAM, "
 170 "Tesselaar K, Koenderman L",
 171 "In vivo labeling with 2H20 reveals a human neutrophil lifespan of 5.4 days",
 172 "Blood 116(4):625-627", 2010, "10.1182/blood-2010-01-259028", "20410504"),
 173 "patel2017":
 174 Ref("patel2017",
 175 "Patel AA, Zhang Y, Fullerton JN, Boelen L, Rongvaux A, Maini AA, et al.",
 176 "The fate and lifespan of human monocyte subsets in steady state and "
 177 "systemic inflammation", "Journal of Experimental Medicine 214(7):1913-1923",
 178 2017, "10.1084/jem.20170355", "28606987"),
 179 "zhang2007":
 180 Ref("zhang2007",
 181 "Zhang Y, Wallace DL, de Lara CM, Ghattas H, Asquith B, Worth A, et al.",
 182 "In vivo kinetics of human natural killer cells: the effects of ageing "
 183 "and acute and chronic viral infection", "Immunology 121(2):258-265", 2007,
 184 "10.1111/j.1365-2567.2007.02573.x", "17346281"),
 185 "mohler2005":
 186 Ref("mohler2005", "Mohler L, Flockerzi D, Sann H, Reichl U",
 187 "Mathematical model of influenza A virus production in large-scale "
 188 "microcarrier culture", "Biotechnology and Bioengineering 90(1):46-58",
 189 2005, "10.1002/bit.20363", "15736163"),
 190 "memoli2015":
 191 Ref("memoli2015",
 192 "Memoli MJ, Czajkowski L, Reed S, Athota R, Bristol T, Proudfoot K, et al.",
 193 "Validation of the wild-type influenza A human challenge model H1N1pdMIST",
 194 "Clinical Infectious Diseases 60(5):693-702", 2015,
 195 "10.1093/cid/ciu924", "25416753"),
 196 "clements1986":
 197 Ref("clements1986", "Clements ML, Betts RF, Tierney EL, Murphy BR",
 198 "Serum and nasal wash antibodies associated with resistance to "
 199 "experimental challenge with influenza A wild-type virus",

```

200 |         "Journal of Clinical Microbiology 24(1):157-160", 1986,
201 |         "10.1128/jcm.24.1.157-160.1986", "3722363"),
202 |     "ivashkiv2014":
203 |     Ref("ivashkiv2014", "Ivashkiv LB, Donlin LT",
204 |         "Regulation of type I interferon responses",
205 |         "Nature Reviews Immunology 14(1):36-49", 2014, "10.1038/nri3581",
206 |         "24362405"),
207 |     "olmsted2001":
208 |     Ref("olmsted2001", "Olmsted SS, Padgett JL, Yudin AI, Whaley KJ, Moench TR, Cone RA",
209 |         "Diffusion of macromolecules and virus-like particles in human cervical mucus",
210 |         "Biophysical Journal 81(4):1930-1937", 2001,
211 |         "10.1016/S0006-3495(01)75844-4", "11566767"),
212 |     "bhat2007":
213 |     Ref("bhat2007", "Bhat R, Watzl C",
214 |         "Serial killing of tumor cells by human natural killer cells - "
215 |         "enhancement by therapeutic antibodies", "PLoS ONE 2(3):e326", 2007,
216 |         "10.1371/journal.pone.0000326", "17389917"),
217 |     "janeway":
218 |     Ref("janeway", "Murphy K, Weaver C", "Janeway's Immunobiology, 9th ed.",
219 |         "Garland Science (textbook)", 2016),
220 |     "dacie":
221 |     Ref("dacie", "Bain BJ, Bates I, Laffan MA",
222 |         "Dacie and Lewis Practical Haematology, 12th ed. (reference intervals)",
223 |         "Elsevier (textbook)", 2017),
224 |     "textbook_hist":
225 |     Ref("textbook_hist", "Mescher AL",
226 |         "Junqueira's Basic Histology (respiratory epithelium cell dimensions)",
227 |         "McGraw-Hill (textbook)", 2018),
228 | }
229 |
230 | MODEL_ASSUMPTION = "모델 가정값"
231 |
232 |
233 | # =====
234 | # SECTION 2. SCALING (축소비율 정의)
235 | # =====
236 |
237 |
238 | class Scale:
239 |     """
240 |     공간/시간/개체수 축소 규약.
241 |
242 |     [공간]
243 |     1,000 x 1,000 = 1,000,000 격자칸.
244 |     1칸 = 15 um (호흡기 상피세포 직경 10~20 um 의 중앙값, textbook_hist)
245 |     -> 도메인 = 15 mm x 15 mm 상피 조직 슬랩
246 |
247 |     [개체]
248 |     kappa = 1/1000 을 상피세포, 면역세포, 바이러스에 공통 적용.
249 |     즉 격자 1칸(상피) = 실제 상피세포 1,000 개
250 |         면역세포 agent 1개 = 실제 백혈구 1,000 개
251 |         바이러스 agent 1개 = 실제 감염성 바이러스 1,000 TCID50
252 |

```

```

253 | [시간]
254 |     1 tick = 6분 = 0.1시간 -> 1일 = 240 tick
255 |     모든 생물학적 속도상수는 (1/day 또는 1/hour) -> tick 단위로 동일 환산.
256 |     """
257 |
258 |     GRID = 1000
259 |     SITES = GRID * GRID                # 1,000,000 칸
260 |     SITE_UM = 15.0                     # 칸 1개의 한변 길이 (um)
261 |     DOMAIN_MM = GRID * SITE_UM / 1000.0 # 15 mm
262 |
263 |     KAPPA = 1.0 / 1000.0                # 개체 축소비율
264 |     INV_KAPPA = 1000.0
265 |
266 |     DT_MIN = 6.0                       # 1 tick = 6 분
267 |     DT_HOUR = DT_MIN / 60.0            # 0.1 시간
268 |     TICKS_PER_HOUR = 10
269 |     TICKS_PER_DAY = 240
270 |
271 |     # 조직 구성: 20 행 주기로 8 행은 상피(target), 12 행은 간질/혈관/림프관
272 |     # -> 상피 비율 40% => 400,000 칸 = 실제 4.0e8 상피세포 (Baccam 2006 T0)
273 |     EPI_PERIOD = 20
274 |     EPI_ROWS = 8
275 |     EPI_FRACTION = EPI_ROWS / EPI_PERIOD # 0.40
276 |     N_TARGET_SITES = int(SITES * EPI_FRACTION) # 400,000
277 |
278 |     # 인터페론/보체 필드 해상도 (5x5 칸 = 75 um 단위)
279 |     FIELD_BIN = 5
280 |     FIELD_N = GRID // FIELD_BIN        # 200 x 200
281 |
282 |     @staticmethod
283 |     def per_tick_from_per_day(rate_per_day: float) -> float:
284 |         """1/day 속도상수 -> tick 당 사건확률 (지수분포)"""
285 |         return 1.0 - math.exp(-rate_per_day / Scale.TICKS_PER_DAY)
286 |
287 |     @staticmethod
288 |     def per_tick_from_half-life_h(half-life_hours: float) -> float:
289 |         """반감기(시간) -> tick 당 소멸확률"""
290 |         k = math.log(2.0) / half-life_hours # 1/hour
291 |         return 1.0 - math.exp(-k * Scale.DT_HOUR)
292 |
293 |     @staticmethod
294 |     def motility_to_sigma_sites(motility_um2_per_min: float) -> float:
295 |         """
296 |         운동성계수 M (um^2/min) -> 1 tick 당 축(axis)별 변위 표준편차 (격자칸 단위)
297 |
298 |         2차원 무작위보행에서 <math>\langle r^2 \rangle = 4Mt</math> 이므로 축별 분산 = <math>2Mt</math>.
299 |         <math>\sigma_{um} = \sqrt{2 * M * dt}</math>, <math>\sigma_{sites} = \sigma_{um} / SITE\_UM</math>
300 |         """
301 |         sigma_um = math.sqrt(2.0 * motility_um2_per_min * Scale.DT_MIN)
302 |         return sigma_um / Scale.SITE_UM
303 |
304 |     @staticmethod
305 |     def speed_to_motility(speed_um_per_min: float,

```



```

306 |             persistence_min: float = 1.0) -> float:
307 |         """
308 |         순간속도 v(um/min)와 방향지속시간 tau(min) -> 운동성계수 M = v^2*tau/4 (2D)
309 |         * 논문은 보통 '속도'를 보고하므로, 무작위보행으로 옮길 때 반드시
310 |         이 변환을 거친다 (사양서 23 항: 이동방향=무작위, 이동속도=논문값 축소).
311 |         """
312 |         return speed_um_per_min ** 2 * persistence_min / 4.0
313 |
314 |
315 | # =====
316 | # SECTION 3.  PARAMETER TABLE  (논문 근거 표 / 사양서 34 항)
317 | # =====
318 |
319 |
320 | @dataclass(frozen=True)
321 | class ParamRow:
322 |     item: str          # 항목
323 |     real: str          # 실제값
324 |     unit: str          # 단위
325 |     source: str        # 논문 출처(REFS key) 또는 MODEL_ASSUMPTION
326 |     scaling: str       # 축소 방식
327 |     sim: str           # 시뮬레이션값
328 |     note: str = ""     # 선택 이유 / 비교
329 |
330 |
331 | def _M(v: float) -> str:
332 |     return f"{v:,.0f}"
333 |
334 |
335 | # ---- 실제값 상수 (논문값) -----
336 | REAL = {
337 |     # 혈액 / 면역세포 (Dacie&Lewis, Bisset 2004)
338 |     "wbc_per_ml": 5.0e6,          # 백혈구 4.0~11.0 x10^9/L 의 중앙 부근
339 |     "neut_frac": 0.60,           # 호중구 40~70%
340 |     "mono_frac": 0.06,          # 단핵구 2~10%
341 |     "lymph_frac": 0.30,         # 림프구 20~45%
342 |     "cd4_of_lymph": 0.45,
343 |     "cd8_of_lymph": 0.25,
344 |     "bcell_of_lymph": 0.12,
345 |     "nk_of_lymph": 0.13,
346 |
347 |     # 바이러스 (Baccam 2006, Harris 2006, Mohler 2005)
348 |     "virion_nm": 100.0,         # 80~120 nm
349 |     "eclipse_h": 6.0,          # Baccam 2006: ~6 h
350 |     "productive_h": 5.0,        # Baccam 2006: ~5 h 생산
351 |     "infected_lifetime_h": 11.0, # Baccam 2006: 평균 11 h
352 |     "virus_half-life_h": 3.0,   # Baccam 2006: 유리 바이러스 반감기 ~3 h
353 |     "R0_within_host": 22.0,     # Baccam 2006: 1개 감염세포 -> ~22 신규감염
354 |     "burst_particles": 5000.0,  # 10^3~10^4 particles/cell (Mohler 2005)
355 |     "target_cells_ur": 4.0e8,   # Baccam 2006 T0
356 |     "inoculum_tcid50": 1.0e5,   # 인체감염모델 접종량 10^4~10^6 (Memoli 2015)
357 |     "virion_D_um2_s": 1.0,     # 혈액 내 확산 (Olmsted 2001 계열, 하한 채택)
358 |

```

```

359 | # 세포 운동성 (논문 '속도' 값)
360 | "neut_speed": 15.0,          # 10~20 um/min (Lammermann 2013)
361 | "mono_speed": 2.5,          # 1~4 um/min
362 | "nk_speed": 8.0,            # 6~10 um/min
363 | "naiveT_speed": 11.0,       # Miller 2002/2003
364 | "effT_speed": 5.0,          # Ariotti 2012 (조직 내 effector/TRM)
365 | "b_speed": 6.0,             # Miller 2002 계열
366 |
367 | # 수명
368 | "neut_lifespan_d": 1.5,      # 혈중 t1/2 7~19h, 조직 1~2일 (Pillay 2010 논쟁적)
369 | "mono_lifespan_d": 3.0,      # Patel 2017 classical monocyte ~1.6d, 조직 연장
370 | "nk_lifespan_d": 14.0,       # Zhang 2007
371 | "naive_lymph_lifespan_d": 180.0,
372 | "effector_T_halfliife_d": 1.5, # 수축기 반감기
373 |
374 | # 후천면역 타임라인
375 | "dc_migration_h": 18.0,      # DC 이주 12~24h (Lawrence 2005 계열)
376 | "priming_h": 24.0,           # 3-phase priming ~24h (Mempel 2004)
377 | "cd8_doubling_h": 7.0,       # in vivo 6~8 h
378 | "cd4_doubling_h": 9.0,
379 | "b_doubling_h": 10.0,
380 | "cd8_max_divisions": 11.0,
381 | "expansion_program_d": 6.0,  # 항원 노출 후 자율 증식 프로그램 (van Stipdonk 2001 / Kaech 2002)
382 | "net_expansion_factor": 0.53, # 분열 6~8h + 확장기 사멸 -> 순 배가시간 ~13h
383 | "cd4_max_divisions": 9.0,
384 | "b_max_divisions": 10.0,
385 | "contraction_rate_d": 0.75,  # 정점 후 90~95% 소실 (Kaech 2002)
386 | "plasmablast_halfliife_d": 3.5, # 여포외 형질모세포 단명 (3~5 일)
387 | "gc_pc_halfliife_d": 7.0,      # GC 유래 형질세포 (골수 이주 전)
388 | "eff_t_halfliife_expand_d": 4.0, # 확장기 조직 효과기 반감기 (모델 가정값)
389 | "cd8_peak_day": 9.0,           # 8~10 일 (Lawrence 2005)
390 | "plasmablast_peak_day": 7.0,   # Wrammert 2008
391 | "memory_fraction": 0.075,     # 정점의 5~10% (Kaech 2002)
392 | "igm_halfliife_d": 5.0,        # Janeway
393 | "igg_halfliife_d": 21.0,       # Janeway
394 | "igm_detect_day": 6.0,         # 5~7 일 (Clements 1986)
395 | "igg_detect_day": 12.0,        # 10~14 일 (Clements 1986)
396 |
397 | # 전구세포 빈도 (Alanio 2010, Blattman 2002)
398 | "cd8_precursor_systemic": 5.0e4,
399 | "cd4_precursor_systemic": 1.0e5,
400 | "b_precursor_systemic": 5.0e4,
401 |
402 | # 자연면역
403 | "ifn_peak_day": 2.0,          # Hayden 1998
404 | "antiviral_state_h": 5.0,     # 4~6h (Ivashkiv 2014)
405 | "c3_mg_per_ml": 1.2,          # 보체 C3 혈장농도 0.9~1.8 mg/mL
406 | "symptom_peak_day": 3.0,       # Carrat 2008 (전신증상 2 일, 총증상 3 일)
407 | "shedding_days": 4.8,         # Carrat 2008 (95%CI 4.31~5.29)
408 | }
409 |
410 |
411 | PARAM_TABLE: List[ParamRow] = [

```

```

412 ParamRow("혈액/조직 기준 부피", "1", "mL 상당", "본 모델 정의", "기준",
413         f"{_M(Scale.SITES)}칸 (15mm x 15mm 상피 슬랩)",
414         "사양서 2 항: 100 만 칸= 축소 기준 공간"),
415 ParamRow("격자 1 칸 크기", "10~20", "um (상피세포 직경)", "textbook_hist",
416         "공간축소", f"{Scale.SITE_UM:.0f} um/칸", "중앙값 채택"),
417 ParamRow("표적 상피세포 총수", f"{REAL['target_cells_urt']:.1e}", "cells",
418         "baccam2006", "kappa=1/1000",
419         f"{_M(Scale.N_TARGET_SITES)}칸 (40% 격자)",
420         "Baccam 2006 T0 =  $4 \times 10^8$  을 1/1000 축소"),
421 ParamRow("백혈구 총수", f"{REAL['wbc_per_ml']:.1e}", "cells/mL", "dacie",
422         "kappa=1/1000", "5,000 agent", "4.0~11.0  $\times 10^9$ /L 중앙 부근"),
423 ParamRow("중성구", f"{REAL['wbc_per_ml']*REAL['neut_frac']:.1e}", "cells/mL",
424         "dacie", "kappa=1/1000", "3,000 agent", "백혈구의 40~70%"),
425 ParamRow("단핵구/대식세포", f"{REAL['wbc_per_ml']*REAL['mono_frac']:.1e}",
426         "cells/mL", "dacie", "kappa=1/1000", "300 agent", "백혈구의 2~10%"),
427 ParamRow("NK 세포",
428         f"{REAL['wbc_per_ml']*REAL['lymph_frac']*REAL['nk_of_lymph']:.1e}",
429         "cells/mL", "bisset2004", "kappa=1/1000", "195 agent",
430         "림프구의 5~20%"),
431 ParamRow("조력 T 세포 (CD4)",
432         f"{REAL['wbc_per_ml']*REAL['lymph_frac']*REAL['cd4_of_lymph']:.1e}",
433         "cells/mL", "bisset2004", "kappa=1/1000", "675 agent", ""),
434 ParamRow("살해 T 세포 (CD8)",
435         f"{REAL['wbc_per_ml']*REAL['lymph_frac']*REAL['cd8_of_lymph']:.1e}",
436         "cells/mL", "bisset2004", "kappa=1/1000", "375 agent", ""),
437 ParamRow("B 세포",
438         f"{REAL['wbc_per_ml']*REAL['lymph_frac']*REAL['bcell_of_lymph']:.1e}",
439         "cells/mL", "bisset2004", "kappa=1/1000", "180 agent", ""),
440 ParamRow("초기 병원체 (접종량)", f"{REAL['inoculum_tcid50']:.1e}", "TCID50",
441         "memoli2015", "kappa=1/1000", "100 agent",
442         "인체감염모델 접종량  $10^4$ ~ $10^6$  중  $10^5$  채택"),
443 ParamRow("바이러스 크기", "80~120", "nm", "harris2006", "공간축소",
444         "점 (무부피) 입자, 1 칸=15um 대비 무시 가능",
445         "세포/공간이 바이러스보다 훨씬 크다는 관계 유지"),
446 ParamRow("바이러스 확산계수", f"{REAL['virion_D_um2_s']}", "um2/s",
447         "olmsted2001", "시간/공간축소",
448         f"sigma={Scale.motility_to_sigma_sites(REAL['virion_D_um2_s']*60):.2f}칸/tick",
449         "점액 결합성으로 자유확산보다 느림, 하한값 채택"),
450 ParamRow("바이러스 잠복기 (eclipse)", f"{REAL['eclipse_h']}", "시간",
451         "baccam2006", "시간축소",
452         f"{int(REAL['eclipse_h']*Scale.TICKS_PER_HOUR)} tick", ""),
453 ParamRow("감염세포 생산기간", f"{REAL['productive_h']}", "시간", "baccam2006",
454         "시간축소", f"{int(REAL['productive_h']*Scale.TICKS_PER_HOUR)} tick",
455         "평균 감염세포 수명 11h = 6h + 5h"),
456 ParamRow("유리 바이러스 반감기", f"{REAL['virus_half-life_h']}", "시간",
457         "baccam2006", "시간축소",
458         f"{Scale.per_tick_from_half-life_h(REAL['virus_half-life_h']):.4f}/tick",
459         "비특이 제거 (점액섬모 등)"),
460 ParamRow("체내 기초재생산수 R0", f"{REAL['R0_within_host']}", "-",
461         "baccam2006", "동일", "burst/감염확률 보정으로 재현",
462         "감염세포 1 개당 신규 생산감염 ~22"),
463 ParamRow("감염세포당 바이러스 생산", " $10^3$ ~ $10^4$ ", "particles/cell",
464         "mohler2005", "kappa + 감염성비율",

```

```

465 |         "25 agent/site (site=1000 세포)",
466 |         "입자:감염단위 비~100:1 반영, R0=22 정합되도록 보정"),
467 | ParamRow("중성구 이동속도", f"{REAL['neut_speed']}", "um/min", "lammermann2013",
468 |         "시간/공간축소",
469 |         f"M={Scale.speed_to_motility(REAL['neut_speed']):.0f} um^2/min "
470 |         f"-> sigma={Scale.motility_to_sigma_sites(Scale.speed_to_motility(REAL['neut_speed'])):.2f}칸/tick",
471 |         "속도->운동성계수 변환 후 무작위보행 적용"),
472 | ParamRow("단핵구/대식세포 이동속도", f"{REAL['mono_speed']}", "um/min",
473 |         "lammermann2013", "시간/공간축소",
474 |         f"sigma={Scale.motility_to_sigma_sites(Scale.speed_to_motility(REAL['mono_speed'])):.2f}칸/tick",
475 |         "1~4 um/min 범위 중앙"),
476 | ParamRow("NK 세포 이동속도", f"{REAL['nk_speed']}", "um/min", "bhat2007",
477 |         "시간/공간축소",
478 |         f"sigma={Scale.motility_to_sigma_sites(Scale.speed_to_motility(REAL['nk_speed'])):.2f}칸/tick",
479 |         ""),
480 | ParamRow("나이트 T 세포 이동속도", f"{REAL['naiveT_speed']}", "um/min",
481 |         "miller2002", "시간/공간축소",
482 |         f"M={Scale.speed_to_motility(REAL['naiveT_speed']):.0f} um^2/min",
483 |         "림프절 내 2 광자 관측 (M=50~70 보고와 정합)"),
484 | ParamRow("효과기 CD8 조직내 이동속도", f"{REAL['effT_speed']}", "um/min",
485 |         "ariotti2012", "시간/공간축소",
486 |         f"sigma={Scale.motility_to_sigma_sites(Scale.speed_to_motility(REAL['effT_speed'])):.2f}칸/tick",
487 |         "조직 내에서는 림프절보다 느림"),
488 | ParamRow("B 세포 이동속도", f"{REAL['b_speed']}", "um/min", "miller2002",
489 |         "시간/공간축소",
490 |         f"sigma={Scale.motility_to_sigma_sites(Scale.speed_to_motility(REAL['b_speed'])):.2f}칸/tick",
491 |         ""),
492 | ParamRow("중성구 수명", f"{REAL['neut_lifespan_d']}", "일", "pillay2010",
493 |         "시간축소", f"{int(REAL['neut_lifespan_d']*Scale.TICKS_PER_DAY)} tick",
494 |         "혈중 t1/2 7~19h vs 5.4 일 보고 상충, 조직 1~2 일 채택"),
495 | ParamRow("단핵구 수명", f"{REAL['mono_lifespan_d']}", "일", "patel2017",
496 |         "시간축소", f"{int(REAL['mono_lifespan_d']*Scale.TICKS_PER_DAY)} tick",
497 |         "classical monocyte ~1.6 일+ 조직 대식세포 전환 연장"),
498 | ParamRow("NK 세포 수명", f"{REAL['nk_lifespan_d']}", "일", "zhang2007",
499 |         "시간축소", f"{int(REAL['nk_lifespan_d']*Scale.TICKS_PER_DAY)} tick", ""),
500 | ParamRow("DC 항원수송 지연", f"{REAL['dc_migration_h']}", "시간", "lawrence2005",
501 |         "시간축소", f"{int(REAL['dc_migration_h']*Scale.TICKS_PER_HOUR)} tick",
502 |         "12~24h 범위 중앙"),
503 | ParamRow("T 세포 프라이밍 시간", f"{REAL['priming_h']}", "시간", "mempel2004",
504 |         "시간축소", f"{int(REAL['priming_h']*Scale.TICKS_PER_HOUR)} tick",
505 |         "3 단계 priming 후 첫 분열까지 ~24h"),
506 | ParamRow("CD8 분열주기", f"{REAL['cd8_doubling_h']}", "시간", "lawrence2005",
507 |         "시간축소", f"{int(REAL['cd8_doubling_h']*Scale.TICKS_PER_HOUR)} tick",
508 |         "in vivo 6~8h"),
509 | ParamRow("CD4 분열주기", f"{REAL['cd4_doubling_h']}", "시간", "mempel2004",
510 |         "시간축소", f"{int(REAL['cd4_doubling_h']*Scale.TICKS_PER_HOUR)} tick", ""),
511 | ParamRow("B 세포 분열주기", f"{REAL['b_doubling_h']}", "시간", "wrammert2008",
512 |         "시간축소", f"{int(REAL['b_doubling_h']*Scale.TICKS_PER_HOUR)} tick", ""),
513 | ParamRow("CD8 나이브 전구세포수",
514 |         f"{REAL['cd8_precursor_systemic']:.1e}", "cells(전신)", "alanio2010",
515 |         "kappa=1/1000", "50 agent",
516 |         "인체 나이브 CD8 특이빈도 ~1/10^6 (전신 풀 기준). "
517 |         "1 mL 혈액만으로는 1 개 미만이므로 전신 풀에서 산출"),

```

```

518 ParamRow("CD4 나이트 전구세포수",
519         f"{REAL['cd4_precursor_systemic']:.1e}", "cells(전신)", "blattman2002",
520         "kappa=1/1000", "100 agent", ""),
521 ParamRow("B 나이트 전구세포수",
522         f"{REAL['b_precursor_systemic']:.1e}", "cells(전신)", "blattman2002",
523         "kappa=1/1000", "50 agent", ""),
524 ParamRow("항바이러스 상태 확립시간", f"{REAL['antiviral_state_h']}", "시간",
525         "ivashkiv2014", "시간축소",
526         f"{int(REAL['antiviral_state_h']*Scale.TICKS_PER_HOUR)} tick",
527         "IFN 노출 후 4~6h"),
528 ParamRow("보체 C3 혈장농도", f"{REAL['c3_mg_per_ml']}", "mg/mL", "janeway",
529         "정규화", "활성도 0~1 스케일",
530         "절대농도 대신 활성분율로 표현"),
531 ParamRow("IgM 반감기", f"{REAL['igm_half-life_d']}", "일", "janeway", "시간축소",
532         f"{Scale.per_tick_from_half-life_h(REAL['igm_half-life_d']*24):.5f}/tick", ""),
533 ParamRow("IgG 반감기", f"{REAL['igg_half-life_d']}", "일", "janeway", "시간축소",
534         f"{Scale.per_tick_from_half-life_h(REAL['igg_half-life_d']*24):.5f}/tick", ""),
535 ParamRow("기억세포 형성비율", "5~10", "% (정점 대비)", "kaech2002", "동일",
536         f"{REAL['memory_fraction']*100:.1f}% ", "수축기 후 잔존"),
537 # ---- 모델 가정값 ----
538 ParamRow("상피/간질 배치", "-", "-", MODEL_ASSUMPTION, "-",
539         "20 행 주기 중 8 행 상피(40%)",
540         "실제 점막의 상피-고유판 층상구조를 2D 로 단순화. "
541         "논문에 대응 수치 없음"),
542 ParamRow("식세포 접촉 반경", "-", "-", MODEL_ASSUMPTION, "-", "동일 격자칸(15um)",
543         "중성구 직경 ~12um, 격자 1 칸과 유사하여 '동일칸=접촉'으로 처리"),
544 ParamRow("중성구 1개 식균 용량", "-", "-", MODEL_ASSUMPTION, "-", "10 회",
545         "식균 후 기능소진. 논문 정량치 부재하여 가정"),
546 ParamRow("NK 접촉당 사멸확률", "-", "-", MODEL_ASSUMPTION, "-", "0.25/tick",
547         "NK-표적 접합 20~60 분(bhat2007) 을 6 분 tick 확률로 환산한 근사"),
548 ParamRow("CTL 접촉당 사멸확률", "-", "-", MODEL_ASSUMPTION, "-", "0.80/tick",
549         "agent 1개=CTL 1,000 개 이므로 접촉 시 해당 칸 제거 가능. "
550         "결과의 세포당 살상률을 regoes2007/ganusov2008 범위와 대조 검증"),
551 ParamRow("염증-동원 결합계수", "-", "-", MODEL_ASSUMPTION, "-",
552         "중성구 최대 4 배, 단핵구 10 배, NK 6 배",
553         "감염 조직 내 유입 배수 관측범위를 반영한 가정"),
554 ParamRow("인터페론 확산/감쇄", "-", "-", MODEL_ASSUMPTION, "-",
555         "5 점 스텔실 확산 + 반감기 2h",
556         "조직 내 IFN 농도장 논문값 부재"),
557 ParamRow("상피 재생속도", "-", "-", MODEL_ASSUMPTION, "-", "0.05/day",
558         "인플루엔자 후 상피 복구는 수일~수주. 감염 재점화 방지 위해 저속 설정"),
559 ]
560
561
562 # =====
563 # SECTION 4. SIMULATION PARAMETERS (실제값 -> tick/칸 단위로 환산)
564 # =====
565
566
567 @dataclass
568 class Params:
569     # ---- 초기 개체수 (kappa=1/1000) -----
570     n_virus0: int = 100

```

```

571 | n_neutrophil: int = 3000
572 | n_monocyte: int = 300
573 | n_nk: int = 195
574 | n_helper_t: int = 675
575 | n_killer_t: int = 375
576 | n_bcell: int = 180
577 |
578 | # ---- 항원특이 전구세포 (전신 풀 기준) -----
579 | precursor_cd8: float = 50.0
580 | precursor_cd4: float = 100.0
581 | precursor_b: float = 50.0
582 |
583 | # ---- 바이러스 동역학 -----
584 | eclipse_ticks: int = int(REAL["eclipse_h"] * Scale.TICKS_PER_HOUR) # 60
585 | productive_ticks: int = int(REAL["productive_h"] * Scale.TICKS_PER_HOUR) # 50
586 | virus_decay_p: float = Scale.per_tick_from_halflife_h(REAL["virus_halflife_h"])
587 | burst_agents_per_site: float = 25.0
588 | p_infect: float = 0.15 # 감수성 칸위 virion agent 의 tick 당 감염확률
589 | virus_sigma: float = Scale.motility_to_sigma_sites(REAL["virion_D_um2_s"] * 60.0)
590 | # 점액섬모 수송에 의한 광역 이동. 비강 점액섬모 청소율 4~10 mm/min 의
591 | # 1/100 수준(0.05 mm/min)만 '무방향 분산'으로 반영한 보수적 설정.
592 | # 방향성 없는 등방 분산이므로 표적 추적과 무관하다. (모델 가정값)
593 | virus_sigma_mucus: float = 30.0
594 |
595 | # ---- 인터페론 -----
596 | ifn_production: float = 0.02 # 생산 감염칸 1개가 tick 당 방출 (정규화)
597 | ifn_decay_p: float = Scale.per_tick_from_halflife_h(2.0)
598 | ifn_diffuse: float = 0.18 # 5 점 스텔실 계수
599 | antiviral_rate: float = 1.0 / (REAL["antiviral_state_h"] * Scale.TICKS_PER_HOUR)
600 | antiviral_decay: float = 1.0 / (24.0 * Scale.TICKS_PER_HOUR)
601 | ifn_block_release: float = 0.80 # 항바이러스 상태 최대 생산억제율
602 |
603 | # ---- 보체 -----
604 | complement_on: float = 0.06 # 활성화 속도상수 (tick^-1)
605 | complement_off: float = 0.02 # 불활성화
606 | complement_lysis: float = 0.010 # 활성화도 1.0 일 때 추가 virion 제거율/tick
607 | complement_opsonin: float = 0.35 # 식균확률 가산 최대치
608 |
609 | # ---- 항체 -----
610 | ab_per_pc_igm: float = 5.0e-6 # 단기 형질세포 agent 당 IgM 생산 (ug/mL/tick)
611 | ab_per_pc_igg: float = 2.5e-5 # GC 유래 형질세포 agent 당 IgG 생산
612 | igm_decay_p: float = Scale.per_tick_from_halflife_h(REAL["igm_halflife_d"] * 24)
613 | igg_decay_p: float = Scale.per_tick_from_halflife_h(REAL["igg_halflife_d"] * 24)
614 | ab_kd: float = 2.0 # 중화 반포화 농도 (ug/mL)
615 | ab_neutralize_max: float = 0.030 # 최대 중화율/tick
616 | ab_opsonin: float = 0.35 # 식균확률 가산 최대치
617 | ab_block_max: float = 0.95 # 중화항체의 최대 감염차단율
618 | ab_block_kd: float = 3.0 # 감염차단 반포화 농도 (ug/mL)
619 | ab_detect_threshold: float = 0.5 # 검출한계 (ug/mL) - 모델 가정값
620 |
621 | # ---- 선천 면역세포 기능 -----
622 | neut_phago_p: float = 0.45 # 접촉당 식균 성공확률 (기본)
623 | neut_capacity: int = 10

```

```

624 | mono_phago_p: float = 0.55
625 | mono_capacity: int = 25
626 | nk_kill_p: float = 0.25
627 | ctl_kill_p: float = 0.80
628 |
629 | # ---- 이동 (운동성계수 -> sigma 칸/tick) -----
630 | sigma_neut: float = Scale.motility_to_sigma_sites(
631 |     Scale.speed_to_motility(REAL["neut_speed"]))
632 | sigma_mono: float = Scale.motility_to_sigma_sites(
633 |     Scale.speed_to_motility(REAL["mono_speed"]))
634 | sigma_nk: float = Scale.motility_to_sigma_sites(
635 |     Scale.speed_to_motility(REAL["nk_speed"]))
636 | sigma_naive_t: float = Scale.motility_to_sigma_sites(
637 |     Scale.speed_to_motility(REAL["naiveT_speed"]))
638 | sigma_eff_t: float = Scale.motility_to_sigma_sites(
639 |     Scale.speed_to_motility(REAL["effT_speed"]))
640 | sigma_b: float = Scale.motility_to_sigma_sites(
641 |     Scale.speed_to_motility(REAL["b_speed"]))
642 |
643 | # ---- 수명 (tick) -----
644 | life_neut: int = int(REAL["neut_lifespan_d"] * Scale.TICKS_PER_DAY)
645 | life_mono: int = int(REAL["mono_lifespan_d"] * Scale.TICKS_PER_DAY)
646 | life_nk: int = int(REAL["nk_lifespan_d"] * Scale.TICKS_PER_DAY)
647 | life_naive: int = int(REAL["naive_lymph_lifespan_d"] * Scale.TICKS_PER_DAY)
648 | eff_t_death_expand: float = Scale.per_tick_from_halflife_h(
649 |     REAL["eff_t_halflife_expand_d"] * 24) # 확장기 (생존신호 존재)
650 | eff_t_death_contract: float = Scale.per_tick_from_halflife_h(
651 |     REAL["effector_T_halflife_d"] * 24) # 수축기
652 |
653 | # ---- 염증 / 동원 -----
654 | inflam_gain_virus: float = 1.0 / 8000.0
655 | inflam_gain_infected: float = 1.0 / 40000.0
656 | inflam_decay: float = 0.010
657 | recruit_neut_max: float = 3.0 # 기저 대비 추가 배수
658 | recruit_mono_max: float = 9.0
659 | recruit_nk_max: float = 5.0
660 | recruit_rate: float = 0.020 # tick 당 목표치 접근속도
661 | damage_rate: float = 0.0015 # 염증 1.0 에서 tick 당 조직손상 누적
662 | epi_regen_p: float = Scale.per_tick_from_per_day(0.02)
663 |
664 | # ---- 림프절 / 후천면역 -----
665 | dc_delay_ticks: int = int(REAL["dc_migration_h"] * Scale.TICKS_PER_HOUR)
666 | priming_ticks: int = int(REAL["priming_h"] * Scale.TICKS_PER_HOUR)
667 | cd8_growth: float = math.log(2.0) / (REAL["cd8_doubling_h"] * Scale.TICKS_PER_HOUR)
668 | cd4_growth: float = math.log(2.0) / (REAL["cd4_doubling_h"] * Scale.TICKS_PER_HOUR)
669 | b_growth: float = math.log(2.0) / (REAL["b_doubling_h"] * Scale.TICKS_PER_HOUR)
670 | cd8_max_expansion: float = 2.0 ** REAL["cd8_max_divisions"]
671 | cd4_max_expansion: float = 2.0 ** REAL["cd4_max_divisions"]
672 | b_max_expansion: float = 2.0 ** REAL["b_max_divisions"]
673 | net_expansion: float = REAL["net_expansion_factor"]
674 | program_ticks: int = int(REAL["expansion_program_d"] * Scale.TICKS_PER_DAY)
675 | contraction_p: float = Scale.per_tick_from_per_day(REAL["contraction_rate_d"])
676 | b_contraction_p: float = Scale.per_tick_from_per_day(0.30)

```

```

677 | # 형질세포 분화율 (tick 당 활성 B 세포 중 분화 비율) - 모델 가정값
678 | ef_diff_rate: float = 0.0016
679 | gc_diff_rate: float = 3.0e-5
680 | pc_short_decay: float = Scale.per_tick_from_per_day(
681 |     math.log(2.0) / REAL["plasmablast_half-life_d"])
682 | pc_gc_decay: float = Scale.per_tick_from_per_day(
683 |     math.log(2.0) / REAL["gc_pc_half-life_d"])
684 | gc_end_ticks: int = int(21.0 * Scale.TICKS_PER_DAY)
685 | memory_b_fraction: float = 0.10
686 | emigration_p: float = 1.6e-4      # 림프절 -> 조직 이주율/tick
687 | emigration_start_h: float = 96.0  # 조직 진입 개시 (~day 4, Lawrence 2005)
688 | antigen_threshold: float = 20.0   # 프라이밍 개시 항원량 (정규화)
689 | antigen_decay: float = Scale.per_tick_from_per_day(1.5)
690 | ef_delay_ticks: int = int(48.0 * Scale.TICKS_PER_HOUR) # 여포외 형질모세포-활성화+2 일
691 | gc_delay_ticks: int = int(144.0 * Scale.TICKS_PER_HOUR) # GC 유래 IgG ~활성화+6 일
692 | memory_fraction: float = REAL["memory_fraction"]
693 |
694 | # ---- 실행 제어 -----
695 | max_days: int = 30
696 | stop_after_clear_days: int = 4
697 | max_virions: int = 800_000      # 계산부하 상한 (초과 시 가중치 병합)
698 | seed: int = 20260809
699 |
700 |
701 | # =====
702 | # SECTION 5.  MOVEMENT ENGINE   (이동 알고리즘 - 교체 가능 지점)
703 | # =====
704 |
705 |
706 | class MovementEngine:
707 |     """
708 |     사양서 23/24/36/37 항 대응.
709 |
710 |     mode = "random"      : 순수 무작위 보행. 방향은 매 tick 균일난수.
711 |                          병원체 위치/농도/사이토카인 경사 일체 참조하지 않음.
712 |     mode = "systematic" : (미구현) 후속 버전에서 이 클래스만 교체하여
713 |                          동일 생물학 조건에서 비교.
714 |
715 |     중요: '이동방향'은 무작위, '이동속도(변위 크기)'는 논문값 축소치.
716 |           두 가지를 반드시 구분한다.
717 |     """
718 |
719 |     ALLOWED = ("random", "systematic")
720 |
721 |     def __init__(self, mode: str, rng: np.random.Generator):
722 |         if mode not in self.ALLOWED:
723 |             raise ValueError(f"movement_mode must be one of {self.ALLOWED}")
724 |         if mode == "systematic":
725 |             raise NotImplementedError(
726 |                 "체계적 이동 모델은 본 버전에서 사용하지 않는다(사양서 24 항). "
727 |                 "동일 생물학 파라미터로 후속 버전에서 구현할 것.")
728 |         self.mode = mode
729 |         self.rng = rng

```



```

730         self.n_calls = 0
731         self.n_agents_moved = 0
732
733     def move(self, x: np.ndarray, y: np.ndarray,
734             sigma_sites: np.ndarray | float) -> None:
735         """
736         제자리(in-place) 무작위 변위.
737         등방성 가우시안 변위 = 방향 균일난수 + 크기 Rayleigh 분포와 동등.
738         """
739         n = x.size
740         if n == 0:
741             return
742         self.n_calls += 1
743         self.n_agents_moved += n
744
745         # --- 방향: 완전 균일 난수 (외부 정보 미참조) ---
746         theta = self.rng.uniform(0.0, 2.0 * math.pi, size=n)
747         # --- 크기: 논문 운동성계수에서 유도된 스케일 ---
748         r = self.rng.normal(0.0, 1.0, size=n) * np.asarray(sigma_sites) * math.sqrt(2.0)
749
750         x += (r * np.cos(theta)).astype(x.dtype)
751         y += (r * np.sin(theta)).astype(y.dtype)
752
753         # 경계: 반사(reflecting) - 조직 슬랩 경계
754         lim = float(Scale.GRID) - 1e-3
755         np.abs(x, out=x)
756         np.abs(y, out=y)
757         over = x > lim
758         if over.any():
759             x[over] = 2.0 * lim - x[over]
760         over = y > lim
761         if over.any():
762             y[over] = 2.0 * lim - y[over]
763         np.clip(x, 0.0, lim, out=x)
764         np.clip(y, 0.0, lim, out=y)
765
766     # =====
767     # SECTION 6.  ENVIRONMENT   (조직 격자 / 인터페론 / 보체 / 염증)
768     # =====
769
770
771     STROMA, HEALTHY, ECLIPSE, PRODUCTIVE, DEAD = 0, 1, 2, 3, 4
772
773
774     class Environment:
775         """
776         조직 슬랩 격자 + 체액성 인자 필드.
777
778         격자 상태(uint8, 1,000,000 칸):
779         0 STROMA      : 간질/혈관/림프관 (표적 아님, 면역세포 통행로)
780         1 HEALTHY     : 정상 상피세포 (표적)
781         2 ECLIPSE     : 감염 직후 잠복기 (바이러스 미생산)
782         3 PRODUCTIVE  : 감염세포 (바이러스 생산 중)

```

```

783 |     4 DEAD      : 사멸 (세포자멸 또는 NK/CTL 살상) -> 바이러스 생산 중단
784 |     """
785 |
786 | def __init__(self, p: Params, rng: np.random.Generator):
787 |     self.p = p
788 |     self.rng = rng
789 |     n = Scale.GRID
790 |
791 |     # ---- 정적 조직 배치 (상피 밴드) ----
792 |     rows = np.arange(n)
793 |     epi_row = (rows % Scale.EPI_PERIOD) < Scale.EPI_ROWS
794 |     base = np.where(epi_row, HEALTHY, STROMA).astype(np.uint8)
795 |     self.state = np.repeat(base, n) # (1,000,000,)
796 |     self.is_epi = (self.state == HEALTHY).copy() # 정적 마스크
797 |     self.n_target = int(self.is_epi.sum())
798 |
799 |     # ---- 감염세포 컴팩트 리스트 ----
800 |     self.inf_site = np.empty(0, dtype=np.int32)
801 |     self.inf_timer = np.empty(0, dtype=np.int16)
802 |     self.inf_stage = np.empty(0, dtype=np.uint8) # 0=eclipse, 1=productive
803 |     self.inf_alive = np.empty(0, dtype=bool)
804 |
805 |     # ---- 분자 필드 (200 x 200, 1칸= 75 um) ----
806 |     f = Scale.FIELD_N
807 |     self.ifn = np.zeros((f, f), dtype=np.float32)
808 |     self.antiviral = np.zeros((f, f), dtype=np.float32)
809 |
810 |     # ---- 전신 스칼라 ----
811 |     self.complement = 0.0
812 |     self.inflammation = 0.0
813 |     self.tissue_damage = 0.0
814 |
815 |     # ---- 누적 ----
816 |     self.n_dead_epi = 0
817 |     self.n_regen = 0
818 |
819 |     # -- 좌표 변환 -----
820 |     @staticmethod
821 |     def site_of(x: np.ndarray, y: np.ndarray) -> np.ndarray:
822 |         col = x.astype(np.int32)
823 |         row = y.astype(np.int32)
824 |         np.clip(col, 0, Scale.GRID - 1, out=col)
825 |         np.clip(row, 0, Scale.GRID - 1, out=row)
826 |         return row * Scale.GRID + col
827 |
828 |     @staticmethod
829 |     def field_idx(site: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
830 |         row = site // Scale.GRID
831 |         col = site - row * Scale.GRID
832 |         return row // Scale.FIELD_BIN, col // Scale.FIELD_BIN
833 |
834 |     # -- 감염 등록 -----
835 |     def infect_sites(self, sites: np.ndarray) -> int:

```

```

836 |         if sites.size == 0:
837 |             return 0
838 |         self.state[sites] = ECLIPSE
839 |         k = sites.size
840 |         self.inf_site = np.concatenate([self.inf_site, sites.astype(np.int32)])
841 |         self.inf_timer = np.concatenate(
842 |             [self.inf_timer, np.zeros(k, dtype=np.int16)])
843 |         self.inf_stage = np.concatenate(
844 |             [self.inf_stage, np.zeros(k, dtype=np.uint8)])
845 |         self.inf_alive = np.concatenate([self.inf_alive, np.ones(k, dtype=bool)])
846 |         return k
847 |
848 | def kill_sites(self, sites: np.ndarray) -> int:
849 |     """NK/CTL 에 의한 감염세포 제거 (바이러스 생산 즉시 중단)"""
850 |     if sites.size == 0:
851 |         return 0
852 |     sites = np.unique(sites)
853 |     mask = (self.state[sites] == ECLIPSE) | (self.state[sites] == PRODUCTIVE)
854 |     sites = sites[mask]
855 |     if sites.size == 0:
856 |         return 0
857 |     self.state[sites] = DEAD
858 |     self.n_dead_epi += sites.size
859 |     return sites.size
860 |
861 | # -- 감염세포 시간진행 -----
862 | def advance_infected(self) -> Tuple[np.ndarray, int]:
863 |     """
864 |     반환: (현재 생산중인 감염칸 site 배열, 자연사멸한 감염세포 수)
865 |     """
866 |     if self.inf_site.size == 0:
867 |         return np.empty(0, dtype=np.int32), 0
868 |
869 |     # NK/CTL 로 이미 죽은 칸 정리
870 |     st = self.state[self.inf_site]
871 |     self.inf_alive &= (st == ECLIPSE) | (st == PRODUCTIVE)
872 |
873 |     alive = self.inf_alive
874 |     if not alive.any():
875 |         self._compact()
876 |         return np.empty(0, dtype=np.int32), 0
877 |
878 |     self.inf_timer[alive] += 1
879 |
880 |     # eclipse -> productive
881 |     to_prod = alive & (self.inf_stage == 0) & (self.inf_timer >= self.p.eclipse_ticks)
882 |     if to_prod.any():
883 |         self.inf_stage[to_prod] = 1
884 |         self.inf_timer[to_prod] = 0
885 |         self.state[self.inf_site[to_prod]] = PRODUCTIVE
886 |
887 |     # productive -> 자연 사멸(세포자멸)
888 |     to_dead = alive & (self.inf_stage == 1) & (self.inf_timer >= self.p.productive_ticks)

```

```

889 |         n_apoptosis = int(to_dead.sum())
890 |     if n_apoptosis:
891 |         self.state[self.inf_site[to_dead]] = DEAD
892 |         self.inf_alive[to_dead] = False
893 |         self.n_dead_epi += n_apoptosis
894 |
895 |     prod_mask = self.inf_alive & (self.inf_stage == 1)
896 |     prod_sites = self.inf_site[prod_mask]
897 |
898 |     if self.inf_alive.size > 20000 and self.inf_alive.mean() < 0.5:
899 |         self._compact()
900 |     return prod_sites, n_apoptosis
901 |
902 | def _compact(self):
903 |     keep = self.inf_alive
904 |     self.inf_site = self.inf_site[keep]
905 |     self.inf_timer = self.inf_timer[keep]
906 |     self.inf_stage = self.inf_stage[keep]
907 |     self.inf_alive = self.inf_alive[keep]
908 |
909 | # -- 인터페론 / 항바이러스 상태 -----
910 | def update_interferon(self, prod_sites: np.ndarray):
911 |     p = self.p
912 |     if prod_sites.size:
913 |         fr, fc = self.field_idx(prod_sites)
914 |         np.add.at(self.ifn, (fr, fc), p.ifn_production)
915 |
916 |     # 5점 스텐실 확산 (등방, 방향정보 없음)
917 |     a = self.ifn
918 |     lap = (np.roll(a, 1, 0) + np.roll(a, -1, 0) +
919 |           np.roll(a, 1, 1) + np.roll(a, -1, 1) - 4.0 * a)
920 |     a += p.ifn_diffuse * lap
921 |     a *= (1.0 - p.ifn_decay_p)
922 |     np.clip(a, 0.0, None, out=a)
923 |
924 |     # 항바이러스 상태 (포화형)
925 |     drive = np.clip(self.ifn * 6.0, 0.0, 1.0)
926 |     self.antiviral += (p.antiviral_rate * drive * (1.0 - self.antiviral)
927 |                      - p.antiviral_decay * self.antiviral)
928 |     np.clip(self.antiviral, 0.0, 1.0, out=self.antiviral)
929 |
930 | # -- 보체 / 염증 / 손상 -----
931 | def update_humoral(self, n_virus: int, n_infected: int):
932 |     p = self.p
933 |     drive = min(1.0, n_virus / 30000.0 + n_infected / 60000.0)
934 |     self.complement += p.complement_on * drive * (1.0 - self.complement)
935 |     self.complement -= p.complement_off * self.complement
936 |     self.complement = float(np.clip(self.complement, 0.0, 1.0))
937 |
938 |     target = min(1.0, n_virus * p.inflam_gain_virus
939 |                 + n_infected * p.inflam_gain_infected)
940 |     self.inflammation += 0.05 * (target - self.inflammation)
941 |     self.inflammation -= p.inflam_decay * self.inflammation

```

```

942         self.inflammation = float(np.clip(self.inflammation, 0.0, 1.0))
943
944         self.tissue_damage += p.damage_rate * self.inflammation
945
946     # -- 상피 재생 -----
947     def regenerate(self):
948         """
949         상피 복구는 염증이 해소되어야 진행된다. 급성 염증기의 IFN/TNF 환경은
950         기저세포 증식을 억제하므로 재생속도에 (1 - 염증) 계수를 적용한다.
951         (모델 가정값 - 정량 논문값 부재)
952         """
953         if self.n_dead_epi <= self.n_regen:
954             return
955         dead_idx = np.flatnonzero(self.state == DEAD)
956         if dead_idx.size == 0:
957             return
958         rate = self.p.epi_regen_p * max(0.0, 1.0 - self.inflammation)
959         if rate <= 0.0:
960             return
961         k = self.rng.binomial(dead_idx.size, rate)
962         if k <= 0:
963             return
964         pick = self.rng.choice(dead_idx, size=min(k, dead_idx.size), replace=False)
965         self.state[pick] = HEALTHY
966         self.n_regen += pick.size
967
968     # -- 집계 -----
969     def counts(self) -> Dict[str, int]:
970         st = self.state
971         return {
972             "healthy": int(np.count_nonzero(st == HEALTHY)),
973             "eclipse": int(np.count_nonzero(st == ECLIPSE)),
974             "productive": int(np.count_nonzero(st == PRODUCTIVE)),
975             "dead": int(np.count_nonzero(st == DEAD)),
976         }
977
978
979     # =====
980     # SECTION 7. VIRUS AGENT POOL (병원체: 확산 + 감염, 추적 기능 없음)
981     # =====
982
983
984     class VirusPool:
985         """
986         Influenza A 자유 바이러스 agent 집합.
987         - 이동: 등방성 확산 (혈류/조직액 이동 + 브라운 운동). 목표 탐색 없음.
988         - 감염: 자기가 위치한 칸이 감수성 상피이면 확률적으로 감염.
989         - 제거: 비특이 제거(반감기 3h) + 보체 + 항체 중화 + 식세포 포식.
990         """
991
992         def __init__(self, p: Params, rng: np.random.Generator):
993             self.p = p
994             self.rng = rng

```

```

995         self.x = np.empty(0, dtype=np.float32)
996         self.y = np.empty(0, dtype=np.float32)
997         self.thinning_events = 0
998         self.thin_weight = 1.0      # 상한 초과 시 1 agent 가 대표하는 배수
999
1000     @property
1001     def n(self) -> int:
1002         return self.x.size
1003
1004     @property
1005     def effective_n(self) -> float:
1006         return self.x.size * self.thin_weight
1007
1008     def seed_uniform(self, k: int):
1009         """흡입 에어로졸/비말은 상기도 표면 전체에 분산 침착하므로
1010         단일 병소가 아니라 다병소로 접촉한다 (Memoli 2015 인체감염모델 기준)."""
1011         self.x = self.rng.uniform(0, Scale.GRID - 1, k).astype(np.float32)
1012         self.y = self.rng.uniform(0, Scale.GRID - 1, k).astype(np.float32)
1013
1014     def add(self, x: np.ndarray, y: np.ndarray):
1015         self.x = np.concatenate([self.x, x.astype(np.float32)])
1016         self.y = np.concatenate([self.y, y.astype(np.float32)])
1017         if self.x.size > self.p.max_virions:
1018             self._thin()
1019
1020     def _thin(self):
1021         keep = self.rng.random(self.x.size) < 0.5
1022         self.x = self.x[keep]
1023         self.y = self.y[keep]
1024         self.thin_weight *= 2.0
1025         self.thinning_events += 1
1026
1027     def remove(self, mask_remove: np.ndarray) -> int:
1028         k = int(np.count_nonzero(mask_remove))
1029         if k:
1030             keep = ~mask_remove
1031             self.x = self.x[keep]
1032             self.y = self.y[keep]
1033         return k
1034
1035     def remove_idx(self, idx: np.ndarray) -> int:
1036         if idx.size == 0:
1037             return 0
1038         m = np.zeros(self.x.size, dtype=bool)
1039         m[idx] = True
1040         return self.remove(m)
1041
1042
1043 # =====
1044 # SECTION 8.  IMMUNE CELL POOL  (면역세포 agent - 구조체 배열)
1045 # =====
1046
1047 NEUT, MONO, NK, TH, TC, BC, MEM = 0, 1, 2, 3, 4, 5, 6

```

```

1048 CELL_NAME = {NEUT: "중성구", MONO: "단핵구/대식세포", NK: "NK 세포",
1049               TH: "조력 T 세포", TC: "살해 T 세포", BC: "B 세포", MEM: "기억세포"}
1050
1051
1052 class ImmuneCellPool:
1053     """면역세포 agent 를 numpy 구조체배열 (SoA)로 관리."""
1054
1055     def __init__(self, rng: np.random.Generator, cap: int = 8192):
1056         self.rng = rng
1057         self.cap = cap
1058         self.n = 0
1059         self.x = np.zeros(cap, np.float32)
1060         self.y = np.zeros(cap, np.float32)
1061         self ctype = np.zeros(cap, np.uint8)
1062         self.age = np.zeros(cap, np.int32)
1063         self.life = np.zeros(cap, np.int32)
1064         self.sigma = np.zeros(cap, np.float32)
1065         self.capacity = np.zeros(cap, np.int16)
1066         self.specific = np.zeros(cap, bool)      # 항원특이 효과기 여부
1067         self.alive = np.zeros(cap, bool)
1068
1069     def _grow(self, need: int):
1070         while self.n + need > self.cap:
1071             self.cap *= 2
1072         for name in ("x", "y", "ctype", "age", "life", "sigma",
1073                    "capacity", "specific", "alive"):
1074             arr = getattr(self, name)
1075             new = np.zeros(self.cap, arr.dtype)
1076             new[:arr.size] = arr
1077             setattr(self, name, new)
1078
1079     def add(self, k: int, ctype: int, sigma: float, life: int,
1080            capacity: int = 0, specific: bool = False,
1081            x: Optional[np.ndarray] = None, y: Optional[np.ndarray] = None,
1082            age_spread: bool = True):
1083         if k <= 0:
1084             return
1085         if self.n + k > self.cap:
1086             self._grow(k)
1087         s = slice(self.n, self.n + k)
1088         self.x[s] = (self.rng.uniform(0, Scale.GRID, k) if x is None else x)
1089         self.y[s] = (self.rng.uniform(0, Scale.GRID, k) if y is None else y)
1090         self.ctype[s] = ctype
1091         self.life[s] = life
1092         # 정상상태 연령분포: 균등하게 풀어 놓아야 초기에 몰살하지 않음
1093         self.age[s] = (self.rng.integers(0, max(1, life), k) if age_spread
1094                       else np.zeros(k, np.int32))
1095         self.sigma[s] = sigma
1096         self.capacity[s] = capacity
1097         self.specific[s] = specific
1098         self.alive[s] = True
1099         self.n += k
1100

```

```

1101 | # -- 조회 -----
1102 | def mask(self, ctype: int) -> np.ndarray:
1103 |     return self.alive[:self.n] & (self.ctype[:self.n] == ctype)
1104 |
1105 | def count(self, ctype: int) -> int:
1106 |     return int(np.count_nonzero(self.mask(ctype)))
1107 |
1108 | def count_specific(self, ctype: int) -> int:
1109 |     return int(np.count_nonzero(self.mask(ctype) & self.specific[:self.n]))
1110 |
1111 | def compact(self):
1112 |     keep = np.flatnonzero(self.alive[:self.n])
1113 |     k = keep.size
1114 |     for name in ("x", "y", "ctype", "age", "life", "sigma",
1115 |                 "capacity", "specific", "alive"):
1116 |         arr = getattr(self, name)
1117 |         arr[:k] = arr[keep]
1118 |         arr[k:self.n] = 0
1119 |     self.alive[:k] = True
1120 |     self.n = k
1121 |
1122 |
1123 | # =====
1124 | # SECTION 9.  LYMPH NODE   (림프절: 항원제시 -> 클론증식 -> 효과기/항체/기억)
1125 | # =====
1126 |
1127 |
1128 | class LymphNode:
1129 |     """
1130 |     배액 림프절을 well-mixed 구획으로 단순화 (사양서 25/26 항).
1131 |     개체 단위가 아닌 '개체수(count)' 로 클론증식을 계산하고,
1132 |     조직으로 이주하는 효과기만 agent 로 생성한다.
1133 |     (효과기 정점은 실제  $10^7 \sim 10^8$  세포 규모여서 전량 agent 화가 불가능)
1134 |     """
1135 |
1136 |     def __init__(self, p: Params, rng: np.random.Generator):
1137 |         self.p = p
1138 |         self.rng = rng
1139 |         self.dc_queue = np.zeros(p.dc_delay_ticks + 1, dtype=np.float64)
1140 |         self.qi = 0
1141 |
1142 |         self.antigen = 0.0
1143 |         self.antigen_total = 0.0
1144 |         self.primed = False
1145 |         self.prime_clock = 0
1146 |         self.activated = False
1147 |
1148 |         self.n_cd4 = p.precursor_cd4      # 림프절 내 항원특이 CD4
1149 |         self.n_cd8 = p.precursor_cd8
1150 |         self.n_b = p.precursor_b
1151 |         self.n_pc_short = 0.0             # 여포외 형질모세포 (IgM)
1152 |         self.n_pc_gc = 0.0               # 배중심 유래 형질세포 (IgG)
1153 |         self.memory_t = 0.0

```



```

1154 |         self.memory_b = 0.0
1155 |
1156 |         self.peak_cd8 = p.precursor_cd8
1157 |         self.peak_cd4 = p.precursor_cd4
1158 |         self.peak_b = p.precursor_b
1159 |         self.peak_pc = 0.0
1160 |         self.t_expansion_end: Optional[int] = None
1161 |         self.contracting = False
1162 |
1163 |         # 이벤트 시각 (tick)
1164 |         self.t_antigen_arrival: Optional[int] = None
1165 |         self.t_priming_done: Optional[int] = None
1166 |         self.t_cd4_activated: Optional[int] = None
1167 |         self.t_cd8_activated: Optional[int] = None
1168 |         self.t_b_activated: Optional[int] = None
1169 |         self.t_first_emigration: Optional[int] = None
1170 |
1171 |     def deposit_antigen(self, amount: float):
1172 |         """식세포가 항원을 획득 -> DC 이주지연 후 림프절 도착"""
1173 |         if amount <= 0:
1174 |             return
1175 |         j = (self.qi + self.p.dc_delay_ticks) % self.dc_queue.size
1176 |         self.dc_queue[j] += amount
1177 |
1178 |     def update(self, tick: int) -> None:
1179 |         p = self.p
1180 |         # 1) 지연 도착 항원
1181 |         arrive = self.dc_queue[self.qi]
1182 |         self.dc_queue[self.qi] = 0.0
1183 |         self.qi = (self.qi + 1) % self.dc_queue.size
1184 |         if arrive > 0 and self.t_antigen_arrival is None:
1185 |             self.t_antigen_arrival = tick
1186 |         self.antigen += arrive
1187 |         self.antigen_total += arrive
1188 |         self.antigen *= (1.0 - p.antigen_decay)
1189 |
1190 |         # 2) 프라이밍 개시
1191 |         if not self.primed and self.antigen >= p.antigen_threshold:
1192 |             self.primed = True
1193 |             self.prime_clock = 0
1194 |         if self.primed and not self.activated:
1195 |             self.prime_clock += 1
1196 |             if self.prime_clock >= p.priming_ticks:
1197 |                 self.activated = True
1198 |                 self.t_priming_done = tick
1199 |                 self.t_cd4_activated = tick
1200 |                 self.t_cd8_activated = tick
1201 |                 self.t_b_activated = tick
1202 |
1203 |         if not self.activated:
1204 |             return
1205 |
1206 |         elapsed = tick - (self.t_priming_done or tick)

```

```

1207 |
1208 | # -----
1209 | # 프로그램된 클론증식 (van Stipdonk 2001; Kaech & Ahmed 2002)
1210 | # 항원은 '개시 신호'이며, 일단 개시되면 증식은 자율적으로 진행되고
1211 | # 정해진 기간 경과 후 반드시 수축기로 전환된다.
1212 | # -> 항원 잔존 여부와 무관하게 정점이 형성되므로 무한증식이 불가능.
1213 | # -----
1214 | expansion_on = elapsed < p.program_ticks
1215 | gr = p.net_expansion
1216 | help_f = self.n_cd4 / (self.n_cd4 + 5.0 * p.precursor_cd4)
1217 |
1218 | if expansion_on:
1219 |     if self.n_cd4 < p.precursor_cd4 * p.cd4_max_expansion:
1220 |         self.n_cd4 *= math.exp(p.cd4_growth * gr)
1221 |     if self.n_cd8 < p.precursor_cd8 * p.cd8_max_expansion:
1222 |         self.n_cd8 *= math.exp(p.cd8_growth * gr * (0.55 + 0.9 * help_f))
1223 |     if self.n_b < p.precursor_b * p.b_max_expansion:
1224 |         self.n_b *= math.exp(p.b_growth * gr * (0.45 + 1.0 * help_f))
1225 | else:
1226 |     # ---- 수축기: 정점의 memory_fraction 까지 감소 후 기억세포로 잔존 ----
1227 |     if not self.contracting:
1228 |         self.contracting = True
1229 |         self.t_expansion_end = tick
1230 |     self.n_cd8 = max(self.n_cd8 * (1.0 - p.contraction_p),
1231 |                     self.peak_cd8 * p.memory_fraction)
1232 |     self.n_cd4 = max(self.n_cd4 * (1.0 - p.contraction_p),
1233 |                     self.peak_cd4 * p.memory_fraction)
1234 |     self.n_b = max(self.n_b * (1.0 - p.b_contraction_p),
1235 |                   self.peak_b * p.memory_b_fraction)
1236 |     self.memory_t = min(self.n_cd8 + self.n_cd4,
1237 |                       (self.peak_cd8 + self.peak_cd4) * p.memory_fraction)
1238 |     self.memory_b = min(self.n_b, self.peak_b * p.memory_b_fraction)
1239 |
1240 | # -----
1241 | # 형질세포 분화 (사양서 19/20 항)
1242 | #   - 여포외 반응(IgM): 활성화 +2 일 부터, 증식기 동안만 신규 분화
1243 | #   - 배종심 반응(IgG): 활성화 +4 일 부터 3 주까지, CD4 도움 의존
1244 | # 두 경로 모두 '신규 분화 창(window)'이 닫히면 소멸만 남으므로
1245 | # 항체 농도가 무한히 상승하지 않는다.
1246 | # -----
1247 | if expansion_on and elapsed >= p.ef_delay_ticks:
1248 |     self.n_pc_short += p.ef_diff_rate * self.n_b
1249 |     self.n_pc_short *= (1.0 - p.pc_short_decay)
1250 |
1251 | if p.gc_delay_ticks <= elapsed < p.gc_end_ticks:
1252 |     self.n_pc_gc += p.gc_diff_rate * self.n_b * min(1.0, help_f * 2.0)
1253 |     self.n_pc_gc *= (1.0 - p.pc_gc_decay)
1254 |
1255 | self.peak_cd8 = max(self.peak_cd8, self.n_cd8)
1256 | self.peak_cd4 = max(self.peak_cd4, self.n_cd4)
1257 | self.peak_b = max(self.peak_b, self.n_b)
1258 | self.peak_pc = max(self.peak_pc, self.n_pc_short + self.n_pc_gc)
1259 |

```

```

1260 | def emigrate(self, tick: int) -> Tuple[int, int]:
1261 |     """조작으로 이주하는 (CTL, Th) agent 수 산출"""
1262 |     p = self.p
1263 |     if not self.activated:
1264 |         return 0, 0
1265 |     if tick * Scale.DT_HOUR < p.emigration_start_h:
1266 |         return 0, 0
1267 |     # 조직 귀소는 림프절 풀 크기에 비례하므로 수축기에는 자동으로 감소한다.
1268 |     n_ctl = self.rng.poisson(p.emigration_p * self.n_cd8)
1269 |     n_th = self.rng.poisson(p.emigration_p * 0.30 * self.n_cd4)
1270 |     if (n_ctl or n_th) and self.t_first_emigration is None:
1271 |         self.t_first_emigration = tick
1272 |     return int(n_ctl), int(n_th)
1273 |
1274 |
1275 | # =====
1276 | # SECTION 10.  ANTIBODY & COMPLEMENT  (체액성 면역)
1277 | # =====
1278 |
1279 |
1280 | class Antibody:
1281 |     """
1282 |     IgM(초기) / IgG(후기) 를 ug/mL 로 추적.
1283 |     효과: 중화(감염력 감소) / 옹소닌화(식균 보조) / 보체 활성화 보조.
1284 |     접촉 즉시 사멸이 아니라 농도의존적 확률로 작용한다 (사양서 20 항).
1285 |     """
1286 |
1287 |     def __init__(self, p: Params):
1288 |         self.p = p
1289 |         self.igm = 0.0
1290 |         self.igg = 0.0
1291 |         self.t_igm_detect: Optional[int] = None
1292 |         self.t_igg_detect: Optional[int] = None
1293 |         self.peak_total = 0.0
1294 |         self.neutralized = 0
1295 |
1296 |     @property
1297 |     def total(self) -> float:
1298 |         return self.igm + self.igg
1299 |
1300 |     def update(self, ln: LymphNode, tick: int):
1301 |         p = self.p
1302 |         self.igm += p.ab_per_pc_igm * ln.n_pc_short
1303 |         self.igg += p.ab_per_pc_igg * ln.n_pc_gc
1304 |         self.igm *= (1.0 - p.igm_decay_p)
1305 |         self.igg *= (1.0 - p.igg_decay_p)
1306 |         if self.t_igm_detect is None and self.igm >= p.ab_detect_threshold:
1307 |             self.t_igm_detect = tick
1308 |         if self.t_igg_detect is None and self.igg >= p.ab_detect_threshold:
1309 |             self.t_igg_detect = tick
1310 |         self.peak_total = max(self.peak_total, self.total)
1311 |
1312 |     def neutralization_rate(self) -> float:

```

```

1313 |         """tick 당 바이러스 중화 확률 (포화형)"""
1314 |         c = self.total
1315 |         return self.p.ab_neutralize_max * c / (c + self.p.ab_kd)
1316 |
1317 |     def opsonin_bonus(self) -> float:
1318 |         c = self.total
1319 |         return self.p.ab_opsonin * c / (c + self.p.ab_kd)
1320 |
1321 |     def infection_block(self) -> float:
1322 |         """
1323 |         중화항체가 바이러스의 '세포 침입 능력' 자체를 차단하는 비율.
1324 |         HA 결합부위를 가려 부착/융합을 저해하는 기전이며, 유리 바이러스를
1325 |         제거하는 것과는 별개의 효과이다 (사양서 20 항).
1326 |         접촉 즉시 사멸이 아니라 농도의존적 확률로 작용한다.
1327 |         """
1328 |         c = self.total
1329 |         return self.p.ab_block_max * c / (c + self.p.ab_block_kd)
1330 |
1331 |
1332 | # =====
1333 | # SECTION 11. SIMULATION CONTROLLER
1334 | # =====
1335 |
1336 |
1337 | class ImmuneSimulation:
1338 |
1339 |     def __init__(self, p: Optional[Params] = None, movement_mode: str = "random",
1340 |                  seed: Optional[int] = None, verbose: bool = True):
1341 |         self.p = p or Params()
1342 |         if seed is not None:
1343 |             self.p.seed = seed
1344 |         self.rng = np.random.default_rng(self.p.seed)
1345 |         self.movement_mode = movement_mode
1346 |         self.mover = MovementEngine(movement_mode, self.rng)
1347 |         self.verbose = verbose
1348 |
1349 |         self.env = Environment(self.p, self.rng)
1350 |         self.virus = VirusPool(self.p, self.rng)
1351 |         self.cells = ImmuneCellPool(self.rng)
1352 |         self.ln = LymphNode(self.p, self.rng)
1353 |         self.ab = Antibody(self.p)
1354 |
1355 |         self.tick = 0
1356 |         self.history: List[dict] = []
1357 |         self.daily: List[dict] = []
1358 |
1359 |         # 누적 통계
1360 |         self.cleared_virus = 0          # 총 제거 병원체
1361 |         self.cleared_by_phago = 0
1362 |         self.cleared_by_decay = 0
1363 |         self.cleared_by_complement = 0
1364 |         self.cleared_by_antibody = 0
1365 |         self.killed_infected = 0        # NK+CTL 살상

```

```

1366 |     self.killed_by_nk = 0
1367 |     self.killed_by_ctl = 0
1368 |     self.apoptosis_infected = 0          # 세포자멸 (면역세포 무관)
1369 |     self.dead_immune = 0
1370 |     self.total_infections = 0          # 누적 감염 발생칸
1371 |     self.ctl_contact = 0
1372 |     self.ctl_kill_events = 0
1373 |     self.ctl_agent_ticks = 0
1374 |     self.consumed_first_gen = 0
1375 |
1376 |     self._recruit_target = {NEUT: float(self.p.n_neutrophil),
1377 |                             MONO: float(self.p.n_monocyte),
1378 |                             NK: float(self.p.n_nk)}
1379 |     self._init_population()
1380 |
1381 | # -----
1382 | def _init_population(self):
1383 |     p = self.p
1384 |     # 병원체: 도메인 중앙부에 국소 집중 (비강 집중 부위 모사)
1385 |     self.virus.seed_uniform(p.n_virus0)
1386 |
1387 |     c = self.cells
1388 |     c.add(p.n_neutrophil, NEUT, p.sigma_neut, p.life_neut, p.neut_capacity)
1389 |     c.add(p.n_monocyte, MONO, p.sigma_mono, p.life_mono, p.mono_capacity)
1390 |     c.add(p.n_nk, NK, p.sigma_nk, p.life_nk)
1391 |     c.add(p.n_helper_t, TH, p.sigma_naive_t, p.life_naive)
1392 |     c.add(p.n_killer_t, TC, p.sigma_naive_t, p.life_naive)
1393 |     c.add(p.n_bcell, BC, p.sigma_b, p.life_naive)
1394 |
1395 | # -----
1396 | # 헬퍼: 같은 격자칸에 있는 (식세포, 바이러스) 매칭
1397 | # -----
1398 | def _match_same_site(self, p_site: np.ndarray, v_site: np.ndarray
1399 |                     ) -> Tuple[np.ndarray, np.ndarray]:
1400 |     """반환 (매칭된 식세포 지역인덱스, 매칭된 바이러스 전역인덱스)"""
1401 |     if p_site.size == 0 or v_site.size == 0:
1402 |         return np.empty(0, np.int64), np.empty(0, np.int64)
1403 |     pu = np.unique(p_site)
1404 |     inmask = np.isin(v_site, pu, kind="table")
1405 |     cand = np.flatnonzero(inmask)
1406 |     if cand.size == 0:
1407 |         return np.empty(0, np.int64), np.empty(0, np.int64)
1408 |     cs = v_site[cand]
1409 |     order = np.argsort(cs, kind="stable")
1410 |     cs_s = cs[order]
1411 |     cand_s = cand[order]
1412 |     uniq, start, counts = np.unique(cs_s, return_index=True, return_counts=True)
1413 |
1414 |     pos = np.searchsorted(uniq, p_site)
1415 |     pos_c = np.minimum(pos, uniq.size - 1)
1416 |     has = uniq[pos_c] == p_site
1417 |     idx_p = np.flatnonzero(has)
1418 |     if idx_p.size == 0:

```

```

1419         return np.empty(0, np.int64), np.empty(0, np.int64)
1420     g = pos_c[idx_p]
1421     off = self.rng.integers(0, counts[g])
1422     v_idx = cand_s[start[g] + off]
1423     # 동일 바이러스 중복 포식 방지
1424     _, keep = np.unique(v_idx, return_index=True)
1425     return idx_p[keep], v_idx[keep]
1426
1427 # -----
1428 def step(self):
1429     p = self.p
1430     env = self.env
1431     self.tick += 1
1432     t = self.tick
1433
1434     # ----- A. 감염세포 시간진행 -----
1435     prod_sites, n_apop = env.advance_infected()
1436     self.apoptosis_infected += n_apop
1437
1438     # ----- B. 바이러스 생산 (인터페론 억제 반영) -----
1439     if prod_sites.size:
1440         fr, fc = Environment.field_idx(prod_sites)
1441         av = env.antiviral[fr, fc]
1442         rate = (p.burst_agents_per_site / p.productive_ticks) * \
1443             (1.0 - p.ifn_block_release * av) / max(1.0, self.virus.thin_weight)
1444         k = self.rng.poisson(np.maximum(rate, 0.0))
1445         tot = int(k.sum())
1446         if tot:
1447             src = np.repeat(prod_sites, k)
1448             row = src // Scale.GRID
1449             col = src - row * Scale.GRID
1450             nx = (col + self.rng.random(tot)).astype(np.float32)
1451             ny = (row + self.rng.random(tot)).astype(np.float32)
1452             self.virus.add(nx, ny)
1453
1454     # ----- C. 바이러스 이동 (무작위 확산) -----
1455     if self.virus.n:
1456         self.mover.move(self.virus.x, self.virus.y,
1457             math.hypot(p.virus_sigma, p.virus_sigma_mucus))
1458
1459     # ----- D. 감염 시도 -----
1460     n_new_inf = 0
1461     if self.virus.n:
1462         v_site = Environment.site_of(self.virus.x, self.virus.y)
1463         st = env.state[v_site]
1464         sus = np.flatnonzero(st == HEALTHY)
1465         if sus.size:
1466             s_sites = v_site[sus]
1467             row = s_sites // Scale.GRID
1468             col = s_sites - row * Scale.GRID
1469             av = env.antiviral[row // Scale.FIELD_BIN, col // Scale.FIELD_BIN]
1470             # 감염확률 = 기본감염력 x (1 - 항바이러스상태) x (1 - 항체차단)
1471             pe = p.p_infect * (1.0 - av) * (1.0 - self.ab.infection_block())

```

```

1472         if self.virus.thin_weight > 1.0:
1473             pe = 1.0 - (1.0 - pe) ** self.virus.thin_weight
1474         ok = self.rng.random(sus.size) < pe
1475         if ok.any():
1476             hit_sites = np.unique(s_sites[ok])
1477             n_new_inf = env.infect_sites(hit_sites)
1478             self.total_infections += n_new_inf
1479             self.virus.remove_idx(sus[ok])
1480     else:
1481         v_site = np.empty(0, np.int32)
1482
1483     # ----- E. 면역세포 노화/사멸 -----
1484     c = self.cells
1485     n = c.n
1486     if n:
1487         alive = c.alive[:n]
1488         c.age[:n][alive] += 1
1489         died = alive & (c.age[:n] >= c.life[:n])
1490         # 효과기 T 세포 사멸: 확장기에도 상시 적용, 수축기에 가속
1491         # (확장기 조직 효과기도 지속적 교체가 일어나므로 무한 누적 불가)
1492         eff = alive & c.specific[:n] & ((c.ctype[:n] == TC) | (c.ctype[:n] == TH))
1493         if eff.any():
1494             pd_eff = (p.eff_t_death_contract if self.ln.contracting
1495                      else p.eff_t_death_expand)
1496             r = self.rng.random(n) < pd_eff
1497             died |= eff & r
1498         k = int(np.count_nonzero(died))
1499         if k:
1500             c.alive[:n][died] = False
1501             self.dead_immune += k
1502         if n > 5000 and c.alive[:n].mean() < 0.85:
1503             c.compact()
1504             n = c.n
1505
1506     # ----- F. 면역세포 이동 (핵심: 무작위 보행) -----
1507     n = c.n
1508     alive_idx = np.flatnonzero(c.alive[:n])
1509     if alive_idx.size:
1510         xa = c.x[alive_idx]
1511         ya = c.y[alive_idx]
1512         self.mover.move(xa, ya, c.sigma[alive_idx])
1513         c.x[alive_idx] = xa
1514         c.y[alive_idx] = ya
1515         cell_site = Environment.site_of(xa, ya)
1516     else:
1517         cell_site = np.empty(0, np.int32)
1518
1519     ct = c.ctype[alive_idx] if alive_idx.size else np.empty(0, np.uint8)
1520
1521     antigen_in = 0.0
1522
1523     # ----- G. 식균작용 (중성구 / 단핵구) -----
1524     if alive_idx.size and self.virus.n:

```

```

1525 |         v_site = Environment.site_of(self.virus.x, self.virus.y)
1526 |         opson = min(0.95, self.ab.opsonin_bonus()
1527 |                     + p.complement_opsonin * env.complement)
1528 |
1529 |         for ctype, base_p in ((NEUT, p.neut_phago_p), (MONO, p.mono_phago_p)):
1530 |             sel = np.flatnonzero((ct == ctype) & (c.capacity[alive_idx] > 0))
1531 |             if sel.size == 0:
1532 |                 continue
1533 |             loc, vidx = self._match_same_site(cell_site[sel], v_site)
1534 |             if loc.size == 0:
1535 |                 continue
1536 |             succ = self.rng.random(loc.size) < min(0.98, base_p + opson)
1537 |             if not succ.any():
1538 |                 continue
1539 |             gi = alive_idx[sel[loc[succ]]]
1540 |             c.capacity[gi] -= 1
1541 |             spent = gi[c.capacity[gi] <= 0]
1542 |             if spent.size:
1543 |                 c.alive[spent] = False
1544 |                 self.dead_immune += spent.size
1545 |             k = self.virus.remove_idx(vidx[succ])
1546 |             self.cleared_by_phago += k
1547 |             self.cleared_virus += k
1548 |             if ctype == MONO:
1549 |                 antigen_in += k * 1.0
1550 |             v_site = Environment.site_of(self.virus.x, self.virus.y)
1551 |
1552 | # ----- H. NK 세포: 감염세포 응해 -----
1553 | if alive_idx.size:
1554 |     sel = np.flatnonzero(ct == NK)
1555 |     if sel.size:
1556 |         s = cell_site[sel]
1557 |         stt = env.state[s]
1558 |         tgt = (stt == PRODUCTIVE) | (stt == ECLIPSE)
1559 |         if tgt.any():
1560 |             boost = 1.0 + 0.5 * float(env.inflammation)
1561 |             ok = self.rng.random(int(tgt.sum())) < min(0.95, p.nk_kill_p * boost)
1562 |             ks = s[tgt][ok]
1563 |             k = env.kill_sites(ks)
1564 |             self.killed_by_nk += k
1565 |             self.killed_infected += k
1566 |
1567 | # ----- I. 살해 T 세포 (항원특이 효과기) -----
1568 | if alive_idx.size:
1569 |     spec = c.specific[alive_idx]
1570 |     sel = np.flatnonzero((ct == TC) & spec)
1571 |     self.ctl_agent_ticks += sel.size
1572 |     if sel.size:
1573 |         s = cell_site[sel]
1574 |         stt = env.state[s]
1575 |         tgt = (stt == PRODUCTIVE) | (stt == ECLIPSE)
1576 |         nt = int(tgt.sum())
1577 |         self.ctl_contact += nt

```



```

1578         if nt:
1579             ok = self.rng.random(nt) < p.ctl_kill_p
1580             ks = s[tgt][ok]
1581             k = env.kill_sites(ks)
1582             self.killed_by_ctl += k
1583             self.killed_infected += k
1584             self.ctl_kill_events += k
1585
1586         # ----- J. 상주 DC/대식세포 항원 표본추출 -----
1587         n_infected_now = int(env.inf_alive.sum()) if env.inf_alive.size else 0
1588         antigen_in += 0.005 * n_infected_now
1589         self.ln.deposit_antigen(antigen_in)
1590
1591         # ----- K. 바이러스 제거 (비특이/보체/항체) -----
1592         if self.virus.n:
1593             pd = p.virus_decay_p
1594             pc = p.complement_lysis * env.complement
1595             pa = self.ab.neutralization_rate()
1596             ptot = 1.0 - (1.0 - pd) * (1.0 - pc) * (1.0 - pa)
1597             rem = self.rng.random(self.virus.n) < ptot
1598             k = self.virus.remove(rem)
1599             if k:
1600                 tot = pd + pc + pa
1601                 self.cleared_by_decay += int(k * pd / tot)
1602                 self.cleared_by_complement += int(k * pc / tot)
1603                 self.cleared_by_antibody += int(k * pa / tot)
1604                 self.ab.neutralized += int(k * pa / tot)
1605                 self.cleared_virus += k
1606
1607         # ----- L. 림프절 / 항체 -----
1608         self.ln.update(t)
1609         n_ctl, n_th = self.ln.emigrate(t)
1610         if n_ctl:
1611             c.add(n_ctl, TC, p.sigma_eff_t, p.life_naive, specific=True,
1612                  age_spread=False)
1613         if n_th:
1614             c.add(n_th, TH, p.sigma_eff_t, p.life_naive, specific=True,
1615                  age_spread=False)
1616         self.ab.update(self.ln, t)
1617
1618         # ----- M. 체액인자 / 염증 / 동원 / 재생 -----
1619         env.update_interferon(prod_sites)
1620         env.update_humoral(self.virus.n, n_infected_now)
1621         self._recruit()
1622         env.regenerate()
1623
1624         self._record()
1625
1626         # -----
1627         def _recruit(self):
1628             """염증에 비례한 골수/혈류로부터의 선천면역세포 동원 (사양서 15 항)"""
1629             p = self.p
1630             infl = self.env.inflammation

```

```

1631 spec = {NEUT: (p.n_neutrophil, p.recruit_neut_max, p.sigma_neut,
1632               p.life_neut, p.neut_capacity),
1633         MONO: (p.n_monocyte, p.recruit_mono_max, p.sigma_mono,
1634               p.life_mono, p.mono_capacity),
1635         NK: (p.n_nk, p.recruit_nk_max, p.sigma_nk, p.life_nk, 0)}
1636 for ctype, (base, mx, sg, lf, cap) in spec.items():
1637     target = base * (1.0 + mx * infl)
1638     cur = self.cells.count(ctype)
1639     gap = target - cur
1640     if gap <= 0:
1641         continue
1642     k = self.rng.poisson(gap * p.recruit_rate)
1643     if k:
1644         self.cells.add(int(k), ctype, sg, lf, cap, age_spread=False)
1645
1646 # -----
1647 def _snapshot(self) -> dict:
1648     env, c, ln, ab = self.env, self.cells, self.ln, self.ab
1649     cn = env.counts()
1650     infected = cn["eclipse"] + cn["productive"]
1651     return {
1652         "tick": self.tick,
1653         "hours": self.tick * Scale.DT_HOUR,
1654         "day": self.tick / Scale.TICKS_PER_DAY,
1655         "virus": self.virus.n,
1656         "virus_eff": self.virus.effective_n,
1657         "infected": infected,
1658         "eclipse": cn["eclipse"],
1659         "productive": cn["productive"],
1660         "healthy": cn["healthy"],
1661         "dead_epi": cn["dead"],
1662         "neutrophil": c.count(NEUT),
1663         "monocyte": c.count(MONO),
1664         "nk": c.count(NK),
1665         "helper_t": c.count(TH),
1666         "helper_t_spec": c.count_specific(TH),
1667         "killer_t": c.count(TC),
1668         "killer_t_spec": c.count_specific(TC),
1669         "bcell": c.count(BC),
1670         "memory": ln.memory_t + ln.memory_b,
1671         "antibody": ab.total,
1672         "igm": ab.igm,
1673         "igg": ab.igg,
1674         "complement": env.complement,
1675         "interferon": float(env.ifn.mean()),
1676         "interferon_max": float(env.ifn.max()),
1677         "antiviral": float(env.antiviral.mean()),
1678         "inflammation": env.inflammation,
1679         "damage": env.tissue_damage,
1680         "cleared_virus": self.cleared_virus,
1681         "killed_infected": self.killed_infected,
1682         "apoptosis": self.apoptosis_infected,
1683         "dead_immune": self.dead_immune,

```

```

1684         "ln_cd8": ln.n_cd8,
1685         "ln_cd4": ln.n_cd4,
1686         "ln_pc": ln.n_pc_short + ln.n_pc_gc,
1687     }
1688
1689     RECORD_EVERY = 5      # 30 분 간격 기록 (전 격자 스캔 비용 절감)
1690
1691     def _record(self):
1692         if (self.tick % self.RECORD_EVERY == 0
1693             or self.tick % Scale.TICKS_PER_DAY == 0):
1694             self.history.append(self._snapshot())
1695
1696     # =====
1697     # SECTION 12. DAILY SNAPSHOT (사양서 29 항 - 하루마다 필수 출력)
1698     # =====
1699     REQUIRED_FIELDS = [
1700         ("현재 시뮬레이션 시간", "time"),
1701         ("병원체 수", "virus"),
1702         ("감염세포 수", "infected"),
1703         ("정상세포 수", "healthy"),
1704         ("중성구 수", "neutrophil"),
1705         ("단핵구/대식세포 수", "monocyte"),
1706         ("NK 세포 수", "nk"),
1707         ("조력 T 세포 수", "helper_t"),
1708         ("살해 T 세포 수", "killer_t"),
1709         ("B 세포 수", "bcell"),
1710         ("기억세포 수", "memory"),
1711         ("항체 농도", "antibody"),
1712         ("보체 활성도", "complement"),
1713         ("인터페론 농도", "interferon"),
1714         ("염증 정도", "inflammation"),
1715         ("제거된 병원체 수", "cleared_virus"),
1716         ("제거된 감염세포 수", "killed_infected"),
1717         ("사망한 면역세포 수", "dead_immune"),
1718     ]
1719
1720     @staticmethod
1721     def format_block(s: dict, title: str) -> str:
1722         """18 개 필수 항목 블록. 하루 경과 시 및 최종 출력에 항상 사용."""
1723         L = []
1724         L.append("=" * 78)
1725         L.append(f" {title}")
1726         L.append("-" * 78)
1727         d = s["day"]
1728         L.append(f" 현재 시뮬레이션 시간 : Day {d:5.2f} "
1729                 f"({s['hours']:7.1f} h, tick {s['tick']})")
1730         L.append(f" 병원체 수 : {s['virus']:>12,} agent"
1731                 f" (= {s['virus_eff']*Scale.INV_KAPPA:.3e} 감염단위 상당)")
1732         L.append(f" 감염세포 수 : {s['infected']:>12,} 칸"
1733                 f" (잠복 {s['eclipse']:,} / 생산 {s['productive']:,})")
1734         L.append(f" 정상세포 수 : {s['healthy']:>12,} 칸"
1735                 f" (사멸상피 {s['dead_epi']:,})")
1736         L.append(f" 중성구 수 : {s['neutrophil']:>12,} agent")

```

```

1737 | L.append(f" 단핵구/대식세포 수 : {s['monocyte']:>12,} agent")
1738 | L.append(f" NK 세포 수 : {s['nk']:>12,} agent")
1739 | L.append(f" 조력 T 세포 수 : {s['helper_t']:>12,} agent"
1740 | f" (항원특이 효과기 {s['helper_t_spec']:,})")
1741 | L.append(f" 살해 T 세포 수 : {s['killer_t']:>12,} agent"
1742 | f" (항원특이 효과기 {s['killer_t_spec']:,})")
1743 | L.append(f" B 세포 수 : {s['bcell']:>12,} agent"
1744 | f" (림프절 형질세포 {s['ln_pc']:,}.0f}"))
1745 | L.append(f" 기억세포 수 : {s['memory']:>12,.0f} agent 상당")
1746 | L.append(f" 항체 농도 : {s['antibody']:>12.3f} ug/mL"
1747 | f" (IgM {s['igm']:.2f} / IgG {s['igg']:.2f}))")
1748 | L.append(f" 보체 활성화도 : {s['complement']*100:>12.2f} %")
1749 | L.append(f" 인터페론 농도 : {s['interferon']*500:>12.2f} pg/mL 상당"
1750 | f" (국소최대 {s['interferon_max']*500:.1f}, "
1751 | f"항바이러스상태 평균 {s['antiviral']*100:.1f}%)")
1752 | L.append(f" 염증 정도 : {s['inflammation']*100:>12.2f} %"
1753 | f" (누적 조직손상 {s['damage']:.3f}))")
1754 | L.append(f" 제거된 병원체 수 : {s['cleared_virus']:>12,} 누적")
1755 | L.append(f" 제거된 감염세포 수 : {s['killed_infected']:>12,} 누적"
1756 | f"(NK/CTL 살상, 별도 세포자멸 {s['apoptosis']:,})")
1757 | L.append(f" 사망한 면역세포 수 : {s['dead_immune']:>12,} 누적")
1758 | L.append(f" * 78)
1759 | return "\n".join(L)
1760 |
1761 | # =====
1762 | # RUN LOOP
1763 | # =====
1764 | def run(self) -> None:
1765 |     p = self.p
1766 |     t0 = time.time()
1767 |
1768 |     snap0 = self._snapshot()
1769 |     self.daily.append(snap0)
1770 |     if self.verbose:
1771 |         print(self.format_block(snap0, "Day 0 | 초기 상태 (감염 직후)")
1772 |               print())
1773 |
1774 |     clear_streak = 0
1775 |     max_ticks = p.max_days * Scale.TICKS_PER_DAY
1776 |     while self.tick < max_ticks:
1777 |         self.step()
1778 |
1779 |         if self.tick % Scale.TICKS_PER_DAY == 0:
1780 |             s = self._snapshot()
1781 |             self.daily.append(s)
1782 |             if self.verbose:
1783 |                 day = self.tick // Scale.TICKS_PER_DAY
1784 |                 print(self.format_block(s, f"Day {day} | 경과 {day}일")
1785 |                       print())
1786 |
1787 |             # 완전 제거 판정
1788 |             if self.virus.n == 0 and (self.env.inf_alive.sum() if
1789 |                                     self.env.inf_alive.size else 0) == 0:

```

```

1790         clear_streak += 1
1791     else:
1792         clear_streak = 0
1793     if clear_streak >= p.stop_after_clear_days * Scale.TICKS_PER_DAY:
1794         break
1795
1796     # 마지막 시점이 일 단위가 아니면 추가 기록
1797     if self.daily[-1]["tick"] != self.tick:
1798         self.daily.append(self._snapshot())
1799
1800     self.wall_time = time.time() - t0
1801
1802
1803 # =====
1804 # SECTION 13. STATISTICS & LITERATURE VALIDATION
1805 # =====
1806
1807
1808 def _series(hist: List[dict], key: str) -> np.ndarray:
1809     return np.array([h[key] for h in hist], dtype=float)
1810
1811
1812 def _days(hist: List[dict]) -> np.ndarray:
1813     return np.array([h["day"] for h in hist], dtype=float)
1814
1815
1816 def _peak(hist, key) -> Tuple[float, float]:
1817     v = _series(hist, key)
1818     if v.size == 0:
1819         return 0.0, float("nan")
1820     i = int(np.argmax(v))
1821     return float(v[i]), float(hist[i]["day"])
1822
1823
1824 def _first_below_after_peak(hist, key, frac) -> Optional[float]:
1825     v = _series(hist, key)
1826     if v.size == 0 or v.max() <= 0:
1827         return None
1828     i = int(np.argmax(v))
1829     thr = v[i] * frac
1830     for j in range(i, v.size):
1831         if v[j] <= thr:
1832             return float(hist[j]["day"])
1833     return None
1834
1835
1836 def _first_zero_after_peak(hist, key) -> Optional[float]:
1837     v = _series(hist, key)
1838     if v.size == 0:
1839         return None
1840     i = int(np.argmax(v))
1841     for j in range(i, v.size):
1842         if v[j] <= 0:

```

```

1843         return float(hist[j]["day"])
1844     return None
1845
1846
1847 def analyze(sim: "ImmuneSimulation") -> dict:
1848     h = sim.history
1849     R = {}
1850
1851     R["virus_peak"], R["virus_peak_day"] = _peak(h, "virus")
1852     R["virus_final"] = h[-1]["virus"] if h else 0
1853     R["virus_t50"] = _first_below_after_peak(h, "virus", 0.5)
1854     R["virus_t90"] = _first_below_after_peak(h, "virus", 0.1)
1855     R["virus_clear_day"] = _first_zero_after_peak(h, "virus")
1856     # 문헌의 '바이러스 배출기간'은 배양/PCR 검출한계 이하가 되는 시점이므로
1857     # agent 가 정확히 0 이 되는 시점이 아니라 정점 대비 상대역치로 비교한다.
1858     R["virus_shed_end_1pct"] = _first_below_after_peak(h, "virus", 0.01)
1859     R["virus_shed_end_01pct"] = _first_below_after_peak(h, "virus", 0.001)
1860
1861     R["inf_peak"], R["inf_peak_day"] = _peak(h, "infected")
1862     R["inf_t50"] = _first_below_after_peak(h, "infected", 0.5)
1863     R["inf_clear_day"] = _first_zero_after_peak(h, "infected")
1864
1865     R["neut_peak"], R["neut_peak_day"] = _peak(h, "neutrophil")
1866     R["mono_peak"], R["mono_peak_day"] = _peak(h, "monocyte")
1867     R["nk_peak"], R["nk_peak_day"] = _peak(h, "nk")
1868     R["ifn_peak"], R["ifn_peak_day"] = _peak(h, "interferon")
1869     R["comp_peak"], R["comp_peak_day"] = _peak(h, "complement")
1870     R["inflam_peak"], R["inflam_peak_day"] = _peak(h, "inflammation")
1871     R["damage_final"] = h[-1]["damage"] if h else 0.0
1872
1873     R["ctl_peak"], R["ctl_peak_day"] = _peak(h, "killer_t_spec")
1874     R["th_peak"], R["th_peak_day"] = _peak(h, "helper_t_spec")
1875     R["pc_peak"], R["pc_peak_day"] = _peak(h, "ln_pc")
1876     R["ab_peak"], R["ab_peak_day"] = _peak(h, "antibody")
1877
1878     tpd = Scale.TICKS_PER_DAY
1879     ln = sim.ln
1880     R["t_antigen_arrival"] = (ln.t_antigen_arrival / tpd
1881                             if ln.t_antigen_arrival else None)
1882     R["t_priming_done"] = (ln.t_priming_done / tpd if ln.t_priming_done else None)
1883     R["t_cd4_act"] = (ln.t_cd4_activated / tpd if ln.t_cd4_activated else None)
1884     R["t_cd8_act"] = (ln.t_cd8_activated / tpd if ln.t_cd8_activated else None)
1885     R["t_b_act"] = (ln.t_b_activated / tpd if ln.t_b_activated else None)
1886     R["t_emigration"] = (ln.t_first_emigration / tpd
1887                         if ln.t_first_emigration else None)
1888     R["t_igm"] = (sim.ab.t_igm_detect / tpd if sim.ab.t_igm_detect else None)
1889     R["t_igg"] = (sim.ab.t_igg_detect / tpd if sim.ab.t_igg_detect else None)
1890
1891     R["memory_final"] = ln.memory_t + ln.memory_b
1892     R["ln_cd8_peak"] = ln.peak_cd8
1893     R["memory_ratio"] = (ln.memory_t / ln.peak_cd8 if ln.peak_cd8 > 0 else 0.0)
1894
1895     # 초기 지수증식률 -> 체내 R0 추정

```

```

1896 |     days = _days(h)
1897 |     inf = _series(h, "infected")
1898 |     m = (days >= 0.4) & (days <= 1.3) & (inf > 5)
1899 |     if m.sum() > 10:
1900 |         r = float(np.polyfit(days[m], np.log(inf[m]), 1)[0]) # per day
1901 |         Tg = (sim.p.eclipse_ticks + 0.5 * sim.p.productive_ticks) / Scale.TICKS_PER_HOUR
1902 |         R["growth_rate_per_day"] = r
1903 |         R["R0_est"] = float(math.exp(r * Tg / 24.0))
1904 |     else:
1905 |         R["growth_rate_per_day"] = float("nan")
1906 |         R["R0_est"] = float("nan")
1907 |
1908 |     # CTL 세포당 살상률 (cells / CTL / day) * kappa 균일축소이므로 직접 비교 가능
1909 |     if sim.ctl_agent_ticks > 0:
1910 |         R["ctl_kill_per_day"] = sim.killed_by_ctl / sim.ctl_agent_ticks * tpd
1911 |     else:
1912 |         R["ctl_kill_per_day"] = float("nan")
1913 |     if sim.ctl_agent_ticks > 0:
1914 |         R["ctl_contact_per_day"] = sim.ctl_contact / sim.ctl_agent_ticks * tpd
1915 |     else:
1916 |         R["ctl_contact_per_day"] = float("nan")
1917 |
1918 |     R["target_depletion"] = 1.0 - (h[-1]["healthy"] / sim.env.n_target) if h else 0.0
1919 |     R["max_target_depletion"] = 1.0 - float(_series(h, "healthy").min()) / sim.env.n_target
1920 |
1921 |     R["killed_by_nk"] = sim.killed_by_nk
1922 |     R["killed_by_ctl"] = sim.killed_by_ctl
1923 |     R["apoptosis"] = sim.apoptosis_infected
1924 |     R["cleared_by_phago"] = sim.cleared_by_phago
1925 |     R["cleared_by_decay"] = sim.cleared_by_decay
1926 |     R["cleared_by_complement"] = sim.cleared_by_complement
1927 |     R["cleared_by_antibody"] = sim.cleared_by_antibody
1928 |     R["total_infections"] = sim.total_infections
1929 |     return R
1930 |
1931 |
1932 | @dataclass
1933 | class Check:
1934 |     item: str
1935 |     lit: str
1936 |     ref: str
1937 |     sim: str
1938 |     verdict: str
1939 |
1940 |
1941 | def _judge(val: Optional[float], lo: float, hi: float,
1942 |           tol: float = 0.35) -> str:
1943 |     if val is None or (isinstance(val, float) and math.isnan(val)):
1944 |         return "판정불가"
1945 |     if lo <= val <= hi:
1946 |         return "일치"
1947 |     span = max(hi - lo, 1e-9)
1948 |     if lo - span * tol <= val <= hi + span * tol:

```

```

1949         return "근접"
1950     return "불일치"
1951
1952
1953 def validate(sim: "ImmuneSimulation", R: dict) -> List[Check]:
1954     def f(v, unit="일"):
1955         if v is None or (isinstance(v, float) and math.isnan(v)):
1956             return "미발생"
1957         return f"Day {v:.2f}" if unit == "일" else f"{v:.3g}"
1958
1959     C: List[Check] = []
1960     C.append(Check("바이러스 정점 시각", "Day 2 (0.5~1 일 급증 후 2 일 정점)",
1961         "carrat2008", f(R["virus_peak_day"]),
1962         _judge(R["virus_peak_day"], 1.5, 2.5)))
1963     C.append(Check("감염세포 정점 시각", "Day 1~3 (표적세포제한 모델)",
1964         "baccam2006", f(R["inf_peak_day"]),
1965         _judge(R["inf_peak_day"], 1.0, 3.0)))
1966     C.append(Check("바이러스 배출 종료(정점의 1%)", "Day 4.3~5.3 (평균 4.8 일)",
1967         "carrat2008", f(R["virus_shed_end_1pct"]),
1968         _judge(R["virus_shed_end_1pct"], 4.3, 6.5)))
1969     C.append(Check("바이러스 완전 제거 시각", "배출 종료 후 수일, 최대 Day 10 내외",
1970         "carrat2008", f(R["virus_clear_day"]),
1971         _judge(R["virus_clear_day"], 5.0, 12.0)))
1972     C.append(Check("바이러스 50% 감소 시각", "정점 후 1~2 일 내",
1973         "carrat2008", f(R["virus_t50"]),
1974         _judge((R["virus_t50"] - R["virus_peak_day"])
1975             if R["virus_t50"] else None, 0.2, 2.0)))
1976     C.append(Check("체내 기초재생산수 R0", "약 22 (감염세포 1 개당 신규 생산감염)",
1977         "baccam2006", f("{R['R0_est']:.1f} (초기 증식을 추정)",
1978         _judge(R["R0_est"], 8.0, 40.0)))
1979     C.append(Check("인터페론 정점 시각", "Day 2 (비강세척액 IFN 정점)",
1980         "hayden1998", f(R["ifn_peak_day"]),
1981         _judge(R["ifn_peak_day"], 1.5, 3.0)))
1982     C.append(Check("염증/증상 정점 시각", "Day 2~3 (전신증상 2 일, 총증상 3 일)",
1983         "carrat2008", f(R["inflam_peak_day"]),
1984         _judge(R["inflam_peak_day"], 1.5, 3.5)))
1985     C.append(Check("중성구 조직유입 정점", "Day 2~3",
1986         "hayden1998", f(R["neut_peak_day"]),
1987         _judge(R["neut_peak_day"], 1.5, 3.5)))
1988     C.append(Check("NK 세포 유입 정점", "Day 2~3",
1989         "zhang2007", f(R["nk_peak_day"]),
1990         _judge(R["nk_peak_day"], 1.5, 3.5)))
1991     C.append(Check("T 세포 프라이밍 완료", "Day 1.5~2.5 (DC 이주 12~24h + 프라이밍 24h)",
1992         "mempel2004", f(R["t_priming_done"]),
1993         _judge(R["t_priming_done"], 1.2, 3.0)))
1994     C.append(Check("항원특이 CD8 조직 정점", "Day 8~10 (일차 인플루엔자 감염)",
1995         "lawrence2005", f(R["ctl_peak_day"]),
1996         _judge(R["ctl_peak_day"], 7.0, 11.0)))
1997     C.append(Check("형질모세포 정점 시각", "Day 7",
1998         "wrammert2008", f(R["pc_peak_day"]),
1999         _judge(R["pc_peak_day"], 6.0, 9.0)))
2000     C.append(Check("IgM 검출 시각", "Day 5~7",
2001         "clements1986", f(R["t_igm"]),

```



```

2002 |         _judge(R["t_igm"], 4.5, 8.0)))
2003 |     C.append(Check("IgG 검출 시각", "Day 10~14",
2004 |         "clements1986", f(R["t_igg"]),
2005 |         _judge(R["t_igg"], 9.0, 15.0)))
2006 |     C.append(Check("기억세포 형성비율", "정점 대비 5~10%",
2007 |         "kaech2002", f"{R['memory_ratio']*100:.1f}%",
2008 |         _judge(R["memory_ratio"] * 100, 5.0, 10.0)))
2009 |     C.append(Check("CTL 세포당 살상률", "2~16 표적세포/CTL/일 (모델의존적)",
2010 |         "regoes2007/ganusov2008",
2011 |         f"{R['ctl_kill_per_day']:.2f} 칸/CTL/일",
2012 |         _judge(R["ctl_kill_per_day"], 2.0, 16.0)))
2013 |     C.append(Check("표적 상피세포 소모율", "표적세포제한 모델에서 상당 비율 소모",
2014 |         "baccam2006", f"{R['max_target_depletion']*100:.1f}%",
2015 |         _judge(R["max_target_depletion"] * 100, 20.0, 95.0)))
2016 |     return C
2017 |
2018 |
2019 | # =====
2020 | # SECTION 14. REPORT PRINTERS
2021 | # =====
2022 |
2023 |
2024 | def print_param_table():
2025 |     print()
2026 |     print("#" * 100)
2027 |     print(" [논문 근거 표]   실제값 -> 논문출처 -> 축소비율 -> 시뮬레이션값 (사양서 34 항)")
2028 |     print("#" * 100)
2029 |     hdr = f"{'항목':<24}{'실제값':>14} {'단위':<18}{'출처':<16}{'축소비율':<14}{'시뮬레이션값'}"
2030 |     print(hdr)
2031 |     print("-" * 130)
2032 |     for r in PARAM_TABLE:
2033 |         src = REFS[r.source].short() if r.source in REFS else r.source
2034 |         print(f"{'r.item':<24}{r.real:>14} {r.unit:<18}{src:<16}{r.scaling:<14}{r.sim}")
2035 |         if r.note:
2036 |             print(f"{'':<24}   L {r.note}")
2037 |     print("-" * 130)
2038 |     print(" * '모델 가정값' 표기 항목은 논문에서 직접 대응 수치를 찾지 못한 값이며,")
2039 |     print("   실제값처럼 제시하지 않고 가정임을 명시하였다 (사양서 35 항).")
2040 |     print()
2041 |     print(" [인용 문헌 전체]")
2042 |     for k in sorted(REFS):
2043 |         print(f"   - {REFS[k].full()}")
2044 |     print(" * DOI/PMID 는 작성 시점 기준. 보고서 인용 전 PubMed 재확인 권장.")
2045 |     print()
2046 |
2047 |
2048 | def print_final_report(sim: "ImmuneSimulation", R: dict, checks: List[Check]):
2049 |     p = sim.p
2050 |     last = sim.history[-1] if sim.history else sim._snapshot()
2051 |
2052 |     print()
2053 |     print("#" * 100)
2054 |     print(" 최종 결과 (무작위 이동 조건, 1 회 실행)")

```

```

2055 | print("#" * 100)
2056 |
2057 | # ---- (1) 필수 18 항목 블록 재출력 ----
2058 | print(ImmuneSimulation.format_block(
2059 |     last, f"FINAL | 종료 시점 Day {last['day']:.2f}  "
2060 |     f"(병원체 소멸 후 관찰기간 포함)"))
2061 | print()
2062 |
2063 | # ---- (2) 상세 결과 (사양서 33 항) ----
2064 | def d(v):
2065 |     return "미발생" if v is None or (isinstance(v, float) and math.isnan(v)) \
2066 |         else f"Day {v:.2f}"
2067 |
2068 | print("-" * 78)
2069 | print(" [병원체]")
2070 | print(f"   · 최대 병원체 수       : {R['virus_peak']:.0f} agent "
2071 |       f" (= {R['virus_peak']*Scale.INV_KAPPA:.2e} 감염단위 상당) @ {d(R['virus_peak_day'])}")
2072 | print(f"   · 최종 병원체 수       : {R['virus_final']:.0f} agent")
2073 | print(f"   · 병원체 50% 감소시간    : {d(R['virus_t50'])}")
2074 | print(f"   · 병원체 90% 감소시간    : {d(R['virus_t90'])}")
2075 | print(f"   · 병원체 배출종료(정점 1%) : {d(R['virus_shed_end_1pct'])}"
2076 |       f" [문헌 '배출기간'과 동일 기준]")
2077 | print(f"   · 병원체 배출종료(정점 0.1%) : {d(R['virus_shed_end_01pct'])}")
2078 | print(f"   · 병원체 완전 제거시간    : {d(R['virus_clear_day'])}"
2079 |       f" [agent 0 도달, 문헌보다 엄격한 기준]")
2080 | print(f"   · 폭발적 증가 최대값      : {R['virus_peak']:.0f} agent "
2081 |       f"(초기 {p.n_virus0} agent 대비 {R['virus_peak']/max(1,p.n_virus0):.0f}배)")
2082 | print(f"   · 초기 지수증식률        : {R['growth_rate_per_day']:.2f} /일 "
2083 |       f"f"-> 추정 R0 = {R['R0_est']:.1f}")
2084 | print()
2085 | print(" [감염세포]")
2086 | print(f"   · 최대 감염세포 수        : {R['inf_peak']:.0f} 칸 @ {d(R['inf_peak_day'])}")
2087 | print(f"   · 감염세포 50% 감소시간    : {d(R['inf_t50'])}")
2088 | print(f"   · 감염세포 제거시간        : {d(R['inf_clear_day'])}")
2089 | print(f"   · 누적 감염 발생칸         : {R['total_infections']:,}")
2090 | print(f"   · 표적세포 최대 소모율     : {R['max_target_depletion']*100:.1f}% "
2091 |       f"(종료시점 {R['target_depletion']*100:.1f}%)")
2092 | print()
2093 | print(" [선천면역]")
2094 | print(f"   · 중성구 정점 / 소모량     : {R['neut_peak']:.0f} agent @ {d(R['neut_peak_day'])}"
2095 |       f" / 식균제거 {R['cleared_by_phago']:,} agent")
2096 | print(f"   · 단핵구/대식세포 정점    : {R['mono_peak']:.0f} agent @ {d(R['mono_peak_day'])}")
2097 | print(f"   · NK세포 정점 / 살상량     : {R['nk_peak']:.0f} agent @ {d(R['nk_peak_day'])}"
2098 |       f" / 감염세포 {R['killed_by_nk']:,} 칸 제거")
2099 | print(f"   · 사망 면역세포 총계       : {sim.dead_immune:,} agent")
2100 | print(f"   · 보체 활성화도 정점      : {R['comp_peak']*100:.1f}% @ {d(R['comp_peak_day'])}"
2101 |       f" (보체 제거 {R['cleared_by_complement']:,} agent)")
2102 | print(f"   · 인터페론 정점           : {R['ifn_peak']*500:.1f} pg/mL 상당 @ {d(R['ifn_peak_day'])}")
2103 | print(f"   · 염증 정점 / 조직손상     : {R['inflam_peak']*100:.1f}% @ {d(R['inflam_peak_day'])}"
2104 |       f" / 누적손상 {R['damage_final']:.3f}")
2105 | print()
2106 | print(" [후천면역]")
2107 | print(f"   · 항원 림프절 도착        : {d(R['t_antigen_arrival'])}")

```

```

2108 | print(f"    · 조력 T 세포 활성화 시간 : {d(R['t_cd4_act'])}")
2109 | print(f"    · 살해 T 세포 활성화 시간 : {d(R['t_cd8_act'])}")
2110 | print(f"    · B 세포 활성화 시간 : {d(R['t_b_act'])}")
2111 | print(f"    · 조직 진입 개시 : {d(R['t_emigration'])}")
2112 | print(f"    · 항원특이 CTL 조직 정점 : {R['ctl_peak']:,.0f} agent @ {d(R['ctl_peak_day'])}")
2113 | print(f"    · 항원특이 Th 조직 정점 : {R['th_peak']:,.0f} agent @ {d(R['th_peak_day'])}")
2114 | print(f"    · 형질세포 정점 : {R['pc_peak']:,.0f} agent @ {d(R['pc_peak_day'])}")
2115 | print(f"    · 항체 생성 시작(IgM/IgG) : {d(R['t_igm'])} / {d(R['t_igg'])}")
2116 | print(f"    · 최대 항체 농도 : {R['ab_peak']:,.2f} ug/mL @ {d(R['ab_peak_day'])}")
2117 | print(f"    · 항체에 의한 병원체 감소 : {R['cleared_by_antibody']:,.} agent")
2118 | print(f"    · CTL 감염세포 제거 : {R['killed_by_ctl']:,.} 칸")
2119 | print(f"    · 기억세포 형성량 : {R['memory_final']:,.0f} agent 상당 "
2120 |         f"(정점의 {R['memory_ratio']*100:.1f}%)")
2121 | print("-" * 78)
2122 | print()
2123 |
2124 | # ---- (3) 병원체 제거 기여도 ----
2125 | tot = max(1, sim.cleared_virus)
2126 | print("    [병원체 제거 기여도]")
2127 | print(f"    · 비특이 제거(점액섬모 등) : {R['cleared_by_decay']:>10,} "
2128 |         f"({R['cleared_by_decay']/tot*100:5.1f}%)")
2129 | print(f"    · 식세포 포식 : {R['cleared_by_phago']:>10,} "
2130 |         f"({R['cleared_by_phago']/tot*100:5.1f}%)")
2131 | print(f"    · 보체 : {R['cleared_by_complement']:>10,} "
2132 |         f"({R['cleared_by_complement']/tot*100:5.1f}%)")
2133 | print(f"    · 항체 중화 : {R['cleared_by_antibody']:>10,} "
2134 |         f"({R['cleared_by_antibody']/tot*100:5.1f}%)")
2135 | print()
2136 | tot_i = max(1, sim.killed_infected + sim.apoptosis_infected)
2137 | print("    [감염세포 제거 기여도]")
2138 | print(f"    · NK 세포 살상 : {R['killed_by_nk']:>10,} "
2139 |         f"({R['killed_by_nk']/tot_i*100:5.1f}%)")
2140 | print(f"    · 살해 T 세포(CTL) 살상 : {R['killed_by_ctl']:>10,} "
2141 |         f"({R['killed_by_ctl']/tot_i*100:5.1f}%)")
2142 | print(f"    · 바이러스 유도 세포자멸 : {R['apoptosis']:>10,} "
2143 |         f"({R['apoptosis']/tot_i*100:5.1f}%)")
2144 | print()
2145 |
2146 | # ---- (4) 이론(문헌) 대비 검증표 ----
2147 | print("#" * 100)
2148 | print("    [이론(문헌) 대비 검증표] - 입력값이 아니라 '창발 결과'만 대조")
2149 | print("#" * 100)
2150 | print(f"{'검증 항목':<26}{'문헌 기준값':<42}{'시뮬레이션 결과':<24}{'판정'}")
2151 | print("-" * 110)
2152 | n_ok = n_near = n_bad = 0
2153 | for ck in checks:
2154 |     print(f"{'ck.item':<26}{'ck.lit':<42}{'ck.sim':<24}{'ck.verdict'}")
2155 |     print(f"{'':<26}└─ 출처: "
2156 |           f"{'REFS[ck.ref].short() if ck.ref in REFS else ck.ref}')"
2157 |     if ck.verdict == "일치":
2158 |         n_ok += 1
2159 |     elif ck.verdict == "근접":
2160 |         n_near += 1

```

```

2161         elif ck.verdict == "불일치":
2162             n_bad += 1
2163     print("-" * 110)
2164     n_tot = n_ok + n_near + n_bad
2165     print(f"    종합: 일치 {n_ok} / 근접 {n_near} / 불일치 {n_bad}    "
2166           f"(판정대상 {n_tot}개, 일치+근접 = {(n_ok+n_near)/max(1,n_tot)*100:.0f}%)")
2167     print()
2168
2169     # ---- (5) 무작위 이동 조건에 대한 결론 (선입견 없이 결과 기반) ----
2170     print("#" * 100)
2171     print("    [무작위 이동 조건 실험 결론 - 결과 기반]")
2172     print("#" * 100)
2173     cleared = R["virus_clear_day"] is not None
2174     print(f"    · 이동 방식          : {sim.movement_mode.upper()} "
2175           f"(방향 = 균일난수, 위치/농도 경사 일체 미참조)")
2176     print(f"    · 이동 호출 횟수        : {sim.mover.n_calls:,}회, "
2177           f"누적 이동 agent {sim.mover.n_agents_moved:,}")
2178     if cleared:
2179         print(f"    · 감염 결과          : 병원체 완전 제거 성공 "
2180               f"({d(R['virus_clear_day'])})")
2181     else:
2182         print(f"    · 감염 결과          : 관찰기간({p.max_days}일) 내 "
2183               f"완전 제거 실패, 잔존 {R['virus_final':, .0f} agent")
2184         print(f"    · CTL 접촉률          : {R['ctl_contact_per_day']:.2f} 회/CTL/일 "
2185               f"(무작위 조우 기반)")
2186         print(f"    · 조직 손상            : 표적세포 최대 {R['max_target_depletion']*100:.1f}% 소모")
2187         print()
2188         print("    ※ 본 실험은 1 회 시행 결과이다. 무작위 보행은 확률과정이므로")
2189         print("    결론 확정에는 반복시행 (사양서 32 항)이 필요하며,")
2190         print("    체계적 이동과의 우열은 동일 생물학 조건에서 이동함수만 교체한")
2191         print("    비교실험 이후에 판단한다.")
2192         print()
2193     if sim.virus.thinning_events:
2194         print(f"    ! 계산부하 상한 도달로 바이러스 agent 가중치 병합 "
2195               f"{sim.virus.thinning_events}회 발생 (1 agent = {sim.virus.thin_weight:.0f} 배 대표)")
2196         print()
2197
2198
2199     # =====
2200     # SECTION 15.  VISUALIZATION   (0.5 초 갱신, 사양서 29 항)
2201     # =====
2202
2203
2204     def run_interactive(sim: "ImmuneSimulation", ticks_per_frame: int = 12):
2205         import matplotlib
2206         import matplotlib.pyplot as plt
2207         import matplotlib.animation as animation
2208         try:
2209             matplotlib.rcParams["font.family"] = "DejaVu Sans"
2210         except Exception:
2211             pass
2212
2213         fig = plt.figure(figsize=(16, 9))

```

```

2214 | gs = fig.add_gridspec(2, 3, width_ratios=[1.2, 1, 1])
2215 | ax_map = fig.add_subplot(gs[:, 0])
2216 | ax_v = fig.add_subplot(gs[0, 1])
2217 | ax_i = fig.add_subplot(gs[1, 1])
2218 | ax_txt = fig.add_subplot(gs[:, 2])
2219 | ax_txt.axis("off")
2220 | fig.suptitle("Influenza A ABM | RANDOM WALK immune search "
2221 |             "(no gradient / no target tracking)", fontweight="bold")
2222 |
2223 | def upd(_):
2224 |     for _ in range(ticks_per_frame):
2225 |         if sim.tick >= sim.p.max_days * Scale.TICKS_PER_DAY:
2226 |             break
2227 |         sim.step()
2228 |     s = sim._snapshot()
2229 |
2230 |     ax_map.clear()
2231 |     ax_map.set_xlim(0, Scale.GRID); ax_map.set_ylim(0, Scale.GRID)
2232 |     ax_map.set_title(f"Tissue Day {s['day']:.2f}")
2233 |     inf = sim.env.inf_site[sim.env.inf_alive] if sim.env.inf_alive.size else np.empty(0, int)
2234 |     if inf.size:
2235 |         sub = inf[:max(1, inf.size // 4000)]
2236 |         ax_map.scatter(sub % Scale.GRID, sub // Scale.GRID, s=2,
2237 |                        c="magenta", alpha=.5, label=f"infected({s['infected']:,})")
2238 |
2239 |     if sim.virus.n:
2240 |         k = max(1, sim.virus.n // 4000)
2241 |         ax_map.scatter(sim.virus.x[:k], sim.virus.y[:k], s=1.5, c="red",
2242 |                        alpha=.35, label=f"virus({s['virus']:,})")
2243 |
2244 |     c = sim.cells
2245 |     for ctype, col, lab in ((NEUT, "royalblue", "neutrophil"),
2246 |                             (NK, "orange", "NK"),
2247 |                             (TC, "lime", "killerT")):
2248 |         m = np.flatnonzero(c.mask(ctype))
2249 |         if m.size:
2250 |             m = m[:max(1, m.size // 800)]
2251 |             ax_map.scatter(c.x[m], c.y[m], s=4, c=col, alpha=.5, label=lab)
2252 |
2253 |     ax_map.legend(loc="upper right", fontsize=7)
2254 |
2255 |     d = _days(sim.history)
2256 |     ax_v.clear()
2257 |     ax_v.plot(d, _series(sim.history, "virus"), "r-", label="virus")
2258 |     ax_v.plot(d, _series(sim.history, "infected"), "m--", label="infected cells")
2259 |     ax_v.set_yscale("symlog"); ax_v.legend(fontsize=8); ax_v.grid(alpha=.3)
2260 |     ax_v.set_xlabel("day")
2261 |
2262 |     ax_i.clear()
2263 |     ax_i.plot(d, _series(sim.history, "neutrophil"), label="neutrophil")
2264 |     ax_i.plot(d, _series(sim.history, "nk"), label="NK")
2265 |     ax_i.plot(d, _series(sim.history, "killer_t_spec"), label="specific CTL")
2266 |     ax_i.plot(d, _series(sim.history, "antibody") * 100, label="Ab x100")
2267 |     ax_i.plot(d, _series(sim.history, "interferon") * 5e4, label="IFN(scaled)")
2268 |     ax_i.legend(fontsize=7); ax_i.grid(alpha=.3); ax_i.set_xlabel("day")

```

```

2267         ax_txt.clear(); ax_txt.axis("off")
2268         ax_txt.text(0, 1, ImmuneSimulation.format_block(s, "LIVE"),
2269                   fontsize=6.5, family="monospace", va="top")
2270
2271     ani = animation.FuncAnimation(fig, upd, frames=100000, interval=500,
2272                                 repeat=False, cache_frame_data=False)
2273     plt.tight_layout()
2274     plt.show()
2275     return ani
2276
2277
2278 def save_figures(sim: "ImmuneSimulation", outdir: str):
2279     import matplotlib
2280     matplotlib.use("Agg")
2281     import matplotlib.pyplot as plt
2282
2283     h = sim.history
2284     d = _days(h)
2285     fig, ax = plt.subplots(2, 2, figsize=(15, 9))
2286     fig.suptitle("Influenza A ABM - RANDOM WALK immune search (single run)",
2287                 fontweight="bold")
2288
2289     a = ax[0, 0]
2290     a.plot(d, _series(h, "virus"), "r-", lw=1.6, label="free virus (agents)")
2291     a.plot(d, _series(h, "infected"), "m--", lw=1.4, label="infected cells")
2292     a.plot(d, _series(h, "healthy"), "g-", lw=1.0, alpha=.6, label="healthy epithelium")
2293     a.set_yscale("log"); a.set_ylim(1, None)
2294     a.set_xlabel("day"); a.set_title("Infection dynamics"); a.grid(alpha=.3)
2295     a.legend(fontsize=8)
2296
2297     a = ax[0, 1]
2298     a.plot(d, _series(h, "neutrophil"), label="neutrophil")
2299     a.plot(d, _series(h, "monocyte"), label="monocyte/macrophage")
2300     a.plot(d, _series(h, "nk"), label="NK")
2301     a.set_xlabel("day"); a.set_title("Innate immune cells"); a.grid(alpha=.3)
2302     a.legend(fontsize=8)
2303
2304     a = ax[1, 0]
2305     a.plot(d, _series(h, "killer_t_spec"), "g-", label="specific CTL (tissue)")
2306     a.plot(d, _series(h, "helper_t_spec"), "b-", label="specific Th (tissue)")
2307     a.plot(d, _series(h, "ln_cd8"), "g--", alpha=.5, label="CD8 in lymph node")
2308     a.set_yscale("symlog")
2309     a.set_xlabel("day"); a.set_title("Adaptive cellular response"); a.grid(alpha=.3)
2310     a.legend(fontsize=8)
2311
2312     a = ax[1, 1]
2313     a.plot(d, _series(h, "antibody"), "b-", label="antibody (ug/mL)")
2314     a.plot(d, _series(h, "interferon") * 500, "orange", label="IFN (pg/mL eq.)")
2315     a.plot(d, _series(h, "complement") * 100, "purple", label="complement (%)")
2316     a.plot(d, _series(h, "inflammation") * 100, "r--", label="inflammation (%)")
2317     a.set_xlabel("day"); a.set_title("Humoral factors"); a.grid(alpha=.3)
2318     a.legend(fontsize=8)
2319

```

```

2320 | plt.tight_layout()
2321 | path = os.path.join(outdir, "immune_random_walk_result.png")
2322 | fig.savefig(path, dpi=130)
2323 | return path
2324 |
2325 |
2326 | # =====
2327 | # SECTION 16.  MAIN
2328 | # =====
2329 |
2330 |
2331 | def main(argv=None):
2332 |     ap = argparse.ArgumentParser(
2333 |         description="면역 무작위이동 ABM (Influenza A)")
2334 |     ap.add_argument("--mode", default="random", choices=["random", "systematic"])
2335 |     ap.add_argument("--seed", type=int, default=Params.seed)
2336 |     ap.add_argument("--days", type=int, default=Params.max_days)
2337 |     ap.add_argument("--runs", type=int, default=1,
2338 |         help="반복 시행 횟수 (이번 실험은 1 회)")
2339 |     ap.add_argument("--live", action="store_true", help="0.5 초 갱신 실시간 화면")
2340 |     ap.add_argument("--outdir", default=".")
2341 |     ap.add_argument("--no-params", action="store_true")
2342 |     args = ap.parse_args(argv)
2343 |
2344 |     print("=" * 100)
2345 |     print("  면역 시뮬레이션 - 무작위 이동 (RANDOM WALK) 조건")
2346 |     print("  Influenza A / 상기도 상피 슬랩 + 배액 림프절 / kappa = 1/1000")
2347 |     print(f"  격자 {Scale.GRID}x{Scale.GRID} = {Scale.SITES:,}칸 "
2348 |         f"{Scale.N_TARGET_SITES:,}칸, 1 tick = {Scale.DT_MIN:.0f}분, "
2349 |         f"1일 = {Scale.TICKS_PER_DAY} tick")
2350 |     print(f"  seed = {args.seed}, 시행 횟수 = {args.runs}")
2351 |     print("=" * 100)
2352 |
2353 |     if not args.no_params:
2354 |         print_param_table()
2355 |
2356 |     p = Params()
2357 |     p.max_days = args.days
2358 |     p.seed = args.seed
2359 |
2360 |     sim = ImmuneSimulation(p, movement_mode=args.mode, seed=args.seed)
2361 |
2362 |     if args.live:
2363 |         run_interactive(sim)
2364 |         return 0
2365 |
2366 |     sim.run()
2367 |     R = analyze(sim)
2368 |     checks = validate(sim, R)
2369 |     print_final_report(sim, R, checks)
2370 |
2371 |     try:
2372 |         path = save_figures(sim, args.outdir)

```

```

2373 |         print(f" [그래프 저장] {path}")
2374 |     except Exception as e:
2375 |         print(f" (그래프 저장 실패: {e})")
2376 |
2377 |     with open(os.path.join(args.outdir, "immune_random_walk_history.json"),
2378 |               "w", encoding="utf-8") as f:
2379 |         json.dump({"daily": sim.daily, "summary": {k: (None if (isinstance(v, float)
2380 |               and math.isnan(v)) else v) for k, v in R.items()}},
2381 |                   f, ensure_ascii=False, indent=1)
2382 |     print(f" [원자료 저장] immune_random_walk_history.json")
2383 |     print(f" [실행시간] {sim.wall_time:.1f}초")
2384 |     return 0
2385 |
2386 |
2387 | if __name__ == "__main__":
2388 |     sys.exit(main())
2389 |

```


부록 B. 인플루엔자 A 감염 모형 — 염증성 환경 (주화성 유도 이동)

면역세포가 자기 수용체 축의 케모카인 농도구배를 따라 이동하는 조건의 Agent-Based Model 전체 소스코드. 부록 A 와 이동 규칙만 다르며, 격자 구조·시간 축척·면역세포 초기 개체수·병원체 파라미터는 완전히 동일하다. 혈관 네트워크, 림프절 공간, 케모카인·사이토카인·인터페론 농도장을 추가로 구현하였다.

파일명	immune_influenza_inflammatory.py	총 줄 수	2,365 줄
실행 환경	Python 3.12 / NumPy · SciPy · Matplotlib	난수 시드	700000, 700037, 700074, 700111, 700148

```

1 | #!/usr/bin/env python3
2 | # -*- coding: utf-8 -*-
3 | """
4 | =====
5 | immune_systematic_abm.py
6 | 면역세포 "체계적 이동(Systematic / Chemotaxis-only)" 감염-면역 반응 ABM
7 | -----
8 | 실험 조건 : 면역세포의 이동 방향은 100% 화학신호 (케모카인 농도구배) +
9 |             혈관구조 + 혈류로만 결정된다.
10 |             무작위 방향 선택, 확률적 이동, 랜덤 탐색을 일체 사용하지 않는다.
11 |             동시에 면역세포는 병원체/감염세포의 좌표를 직접 참조하지 못한다.
12 |             (오직 자신이 있는 지점 주변 8 방향의 신호 농도만 감지)
13 |
14 | 대표 병원체 : Influenza A virus (H1N1)
15 | 모델 도메인 : 상기도 상피 슬랩 + 혈관 네트워크 + 배액 림프절
16 |
17 | 축소 원칙 : 실제값 -> 논문 출처 -> 축소비율 -> 시뮬레이션값
18 |             개체 축소비 kappa = 1/1000 (상피세포/면역세포/바이러스 공통)
19 |             논문에서 값을 찾지 못한 항목은 "모델 가정값"으로 명시
20 |
21 | ※ 본 파일은 무작위이동 모델과 코드/상태를 전혀 공유하지 않는 독립 실행 파일이다.
22 |     생물학 파라미터는 동일 문헌에서 독립적으로 재산출하였다.
23 | =====
24 | """
25 |
26 | from __future__ import annotations
27 |
28 | import argparse
29 | import json
30 | import math
31 | import os
32 | import sys
33 | import time
34 | from dataclasses import dataclass
35 | from typing import Dict, List, Optional, Tuple
36 |
37 | import numpy as np
38 |
39 | # =====
40 | # SECTION 1. LITERATURE REFERENCE DATABASE

```

```

41 | # =====
42 |
43 |
44 | @dataclass(frozen=True)
45 | class Ref:
46 |     tag: str
47 |     authors: str
48 |     title: str
49 |     journal: str
50 |     year: int
51 |     doi: str = ""
52 |     pmid: str = ""
53 |
54 |     def short(self) -> str:
55 |         return f"{self.authors.split(',')[0]} et al. {self.year}"
56 |
57 |     def full(self) -> str:
58 |         ident = []
59 |         if self.doi:
60 |             ident.append(f"DOI:{self.doi}")
61 |         if self.pmid:
62 |             ident.append(f"PMID:{self.pmid}")
63 |         return (f"{self.authors}. \"{self.title}\" {self.journal} "
64 |                f"({self.year}). {' '.join(ident)}")
65 |
66 |
67 | REFS: Dict[str, Ref] = {
68 |     # ---- 바이러스 동역학 ----
69 |     "baccam2006": Ref(
70 |         "baccam2006",
71 |         "Baccam P, Beauchemin C, Macken CA, Hayden FG, Perelson AS",
72 |         "Kinetics of influenza A virus infection in humans",
73 |         "Journal of Virology 80(15):7590-7599", 2006,
74 |         "10.1128/JVI.01623-05", "16840338"),
75 |     "carrat2008": Ref(
76 |         "carrat2008",
77 |         "Carrat F, Vergu E, Ferguson NM, Lemaitre M, Cauchemez S, Leach S, Valleron AJ",
78 |         "Time lines of infection and disease in human influenza: a review of "
79 |         "volunteer challenge studies",
80 |         "American Journal of Epidemiology 167(7):775-785", 2008,
81 |         "10.1093/aje/kwm375", "18230677"),
82 |     "mohler2005": Ref(
83 |         "mohler2005", "Mohler L, Flockerzi D, Sann H, Reichl U",
84 |         "Mathematical model of influenza A virus production in large-scale "
85 |         "microcarrier culture",
86 |         "Biotechnology and Bioengineering 90(1):46-58", 2005,
87 |         "10.1002/bit.20363", "15736163"),
88 |     "harris2006": Ref(
89 |         "harris2006",
90 |         "Harris A, Cardone G, Winkler DC, Heymann JB, Brecher M, White JM, Steven AC",
91 |         "Influenza virus pleiomorphy characterized by cryoelectron tomography",
92 |         "PNAS 103(50):19123-19127", 2006, "10.1073/pnas.0607614103", "17146053"),
93 |     "memoli2015": Ref(

```

```

94 |         "memoli2015",
95 |         "Memoli MJ, Czajkowski L, Reed S, Athota R, Bristol T, Proudfoot K, et al.",
96 |         "Validation of the wild-type influenza A human challenge model H1N1pdMIST",
97 |         "Clinical Infectious Diseases 60(5):693-702", 2015,
98 |         "10.1093/cid/ciu924", "25416753"),
99 |     "olmsted2001": Ref(
100 |         "olmsted2001",
101 |         "Olmsted SS, Padgett JL, Yudin AI, Whaley KJ, Moench TR, Cone RA",
102 |         "Diffusion of macromolecules and virus-like particles in human cervical mucus",
103 |         "Biophysical Journal 81(4):1930-1937", 2001,
104 |         "10.1016/S0006-3495(01)75844-4", "11566767"),
105 |
106 | # ---- 혈액 세포 조성 ----
107 | "dacie": Ref("dacie", "Bain BJ, Bates I, Laffan MA",
108 |             "Dacie and Lewis Practical Haematology, 12th ed.",
109 |             "Elsevier (textbook, reference intervals)", 2017),
110 | "bisset2004": Ref(
111 |     "bisset2004", "Bisset LR, Lung TL, Kaelin M, Ludwig E, Dubs RW",
112 |     "Reference values for peripheral blood lymphocyte phenotypes applicable "
113 |     "to the healthy adult population in Switzerland",
114 |     "European Journal of Haematology 72(3):203-212", 2004,
115 |     "10.1046/j.0902-4441.2003.00199.x", "14962239"),
116 |
117 | # ---- 세포 운동성 / 주화성 ----
118 | "lammermann2013": Ref(
119 |     "lammermann2013",
120 |     "Lammermann T, Afonso PV, Angermann BR, Wang JM, Kastenmuller W, "
121 |     "Parent CA, Germain RN",
122 |     "Neutrophil swarms require LTB4 and integrins at sites of cell death in vivo",
123 |     "Nature 498(7454):371-375", 2013, "10.1038/nature12175", "23708969"),
124 | "miller2002": Ref(
125 |     "miller2002", "Miller MJ, Wei SH, Parker I, Cahalan MD",
126 |     "Two-photon imaging of lymphocyte motility and antigen response in "
127 |     "intact lymph node", "Science 296(5574):1869-1873", 2002,
128 |     "10.1126/science.1070051", "12016203"),
129 | "ariotti2012": Ref(
130 |     "ariotti2012",
131 |     "Ariotti S, Beltman JB, Chodaczek G, Hoekstra ME, van Beek AE, et al.",
132 |     "Tissue-resident memory CD8+ T cells continuously patrol skin epithelia "
133 |     "to quickly recognize local antigen", "PNAS 109(48):19739-19744", 2012,
134 |     "10.1073/pnas.1208927109", "23150545"),
135 | "bhat2007": Ref(
136 |     "bhat2007", "Bhat R, Watzl C",
137 |     "Serial killing of tumor cells by human natural killer cells - "
138 |     "enhancement by therapeutic antibodies", "PLoS ONE 2(3):e326", 2007,
139 |     "10.1371/journal.pone.0000326", "17389917"),
140 | "sackmann2012": Ref(
141 |     "sackmann2012",
142 |     "Sackmann EK, Berthier E, Schwantes EA, Fichtinger PS, Evans MD, et al.",
143 |     "Characterizing bacteria-neutrophil chemotaxis and migration velocity "
144 |     "using a microfluidic device",
145 |     "PNAS 109(15):5813-5818", 2012, "10.1073/pnas.1119578109", "22451913"),
146 | "boneschanker2014": Ref(

```

```

147 |         "boneschansker2014",
148 |         "Boneschansker L, Yan J, Wong E, Briscoe DM, Irimia D",
149 |         "Microfluidic platform for the quantitative analysis of leukocyte "
150 |         "migration signatures", "Nature Communications 5:4787", 2014,
151 |         "10.1038/ncomms5787", "25183261"),
152 |     "beltman2007": Ref(
153 |         "beltman2007", "Beltman JB, Maree AFM, Lynch JN, Miller MJ, de Boer RJ",
154 |         "Lymph node topology dictates T cell migration behavior",
155 |         "Journal of Experimental Medicine 204(4):771-780", 2007,
156 |         "10.1084/jem.20061278", "17389236"),
157 |
158 | # ---- 케모카인 / 사이토카인 ----
159 |     "hayden1998": Ref(
160 |         "hayden1998",
161 |         "Hayden FG, Fritz R, Lobo MC, Alvord W, Strober W, Straus SE",
162 |         "Local and systemic cytokine responses during experimental human "
163 |         "influenza A virus infection",
164 |         "Journal of Clinical Investigation 101(3):643-649", 1998,
165 |         "10.1172/JCI1355", "9449698"),
166 |     "proudfoot2003": Ref(
167 |         "proudfoot2003",
168 |         "Proudfoot AEI, Handel TM, Johnson Z, Lau EK, LiWang P, et al.",
169 |         "Glycosaminoglycan binding and oligomerization are essential for the "
170 |         "in vivo activity of certain chemokines",
171 |         "PNAS 100(4):1885-1890", 2003, "10.1073/pnas.0334864100", "12571364"),
172 |     "weber2013": Ref(
173 |         "weber2013",
174 |         "Weber M, Hauschild R, Schwarz J, Moussion C, de Vries I, et al.",
175 |         "Interstitial dendritic cell guidance by haptotactic chemokine gradients",
176 |         "Science 339(6117):328-332", 2013, "10.1126/science.1228456", "23329049"),
177 |     "ivashkiv2014": Ref(
178 |         "ivashkiv2014", "Ivashkiv LB, Donlin LT",
179 |         "Regulation of type I interferon responses",
180 |         "Nature Reviews Immunology 14(1):36-49", 2014, "10.1038/nri3581",
181 |         "24362405"),
182 |
183 | # ---- 혈관 / 혈관외 유출 ----
184 |     "ley2007": Ref(
185 |         "ley2007", "Ley K, Laudanna C, Cybulsky MI, Nourshargh S",
186 |         "Getting to the site of inflammation: the leukocyte adhesion cascade "
187 |         "updated", "Nature Reviews Immunology 7(9):678-689", 2007,
188 |         "10.1038/nri2156", "17717539"),
189 |     "nourshargh2010": Ref(
190 |         "nourshargh2010", "Nourshargh S, Hordijk PL, Sixt M",
191 |         "Breaching multiple barriers: leukocyte motility through venular walls "
192 |         "and the interstitium", "Nature Reviews Molecular Cell Biology 11(5):366-378",
193 |         2010, "10.1038/nrm2889", "20414258"),
194 |
195 | # ---- 세포 수명 ----
196 |     "pillay2010": Ref(
197 |         "pillay2010",
198 |         "Pillay J, den Braber I, Vrizekoop N, Kwast LM, de Boer RJ, Borghans JAM, "
199 |         "Tesselaar K, Koenderman L",

```

200 | "In vivo labeling with 2H2O reveals a human neutrophil lifespan of 5.4 days",
 201 | "Blood 116(4):625-627", 2010, "10.1182/blood-2010-01-259028", "20410504"),
 202 | "patel2017": Ref(
 203 | "patel2017",
 204 | "Patel AA, Zhang Y, Fullerton JN, Boelen L, Rongvaux A, Maini AA, et al.",
 205 | "The fate and lifespan of human monocyte subsets in steady state and "
 206 | "systemic inflammation",
 207 | "Journal of Experimental Medicine 214(7):1913-1923", 2017,
 208 | "10.1084/jem.20170355", "28606987"),
 209 | "zhang2007": Ref(
 210 | "zhang2007",
 211 | "Zhang Y, Wallace DL, de Lara CM, Ghattas H, Asquith B, Worth A, et al.",
 212 | "In vivo kinetics of human natural killer cells: the effects of ageing "
 213 | "and acute and chronic viral infection",
 214 | "Immunology 121(2):258-265", 2007,
 215 | "10.1111/j.1365-2567.2007.02573.x", "17346281"),
 216 |
 217 | # ---- 후천면역 ----
 218 | "mempel2004": Ref(
 219 | "mempel2004", "Mempel TR, Henrickson SE, von Andrian UH",
 220 | "T-cell priming by dendritic cells in lymph nodes occurs in three "
 221 | "distinct phases", "Nature 427(6970):154-159", 2004,
 222 | "10.1038/nature02238", "14712275"),
 223 | "lawrence2005": Ref(
 224 | "lawrence2005", "Lawrence CW, Ream RM, Braciale TJ",
 225 | "Frequency, specificity, and sites of expansion of CD8+ T cells during "
 226 | "primary pulmonary influenza virus infection",
 227 | "Journal of Immunology 174(9):5332-5340", 2005,
 228 | "10.4049/jimmunol.174.9.5332", "15843530"),
 229 | "wrammert2008": Ref(
 230 | "wrammert2008",
 231 | "Wrammert J, Smith K, Miller J, Langley WA, Kokko K, Larsen C, et al.",
 232 | "Rapid cloning of high-affinity human monoclonal antibodies against "
 233 | "influenza virus", "Nature 453(7195):667-671", 2008,
 234 | "10.1038/nature06890", "18449194"),
 235 | "kaech2002": Ref(
 236 | "kaech2002", "Kaech SM, Wherry EJ, Ahmed R",
 237 | "Effector and memory T-cell differentiation: implications for vaccine "
 238 | "development", "Nature Reviews Immunology 2(4):251-262", 2002,
 239 | "10.1038/nri778", "11970378"),
 240 | "vanstipdonk2001": Ref(
 241 | "vanstipdonk2001", "van Stipdonk MJB, Lemmens EE, Schoenberger SP",
 242 | "Naive CTLs require a single brief period of antigenic stimulation for "
 243 | "clonal expansion and differentiation",
 244 | "Nature Immunology 2(5):423-429", 2001, "10.1038/87730", "11323696"),
 245 | "alanio2010": Ref(
 246 | "alanio2010", "Alanio C, Lemaitre F, Law HKW, Hasan M, Albert ML",
 247 | "Enumeration of human antigen-specific naive CD8+ T cells reveals "
 248 | "conserved precursor frequencies", "Blood 115(18):3718-3725", 2010,
 249 | "10.1182/blood-2009-10-251124", "20220118"),
 250 | "blattman2002": Ref(
 251 | "blattman2002",
 252 | "Blattman JN, Antia R, Sourdive DJD, Wang X, Kaech SM, et al.",

```

253 |         "Estimating the precursor frequency of naive antigen-specific CD8 T cells",
254 |         "Journal of Experimental Medicine 195(5):657-664", 2002,
255 |         "10.1084/jem.20001021", "11877489"),
256 |     "clements1986": Ref(
257 |         "clements1986", "Clements ML, Betts RF, Tierney EL, Murphy BR",
258 |         "Serum and nasal wash antibodies associated with resistance to "
259 |         "experimental challenge with influenza A wild-type virus",
260 |         "Journal of Clinical Microbiology 24(1):157-160", 1986,
261 |         "10.1128/jcm.24.1.157-160.1986", "3722363"),
262 |     "regoes2007": Ref(
263 |         "regoes2007", "Regoes RR, Barber DL, Ahmed R, Antia R",
264 |         "Estimation of the rate of killing by cytotoxic T lymphocytes in vivo",
265 |         "PNAS 104(5):1599-1603", 2007, "10.1073/pnas.0508830104", "17242364"),
266 |     "ganusov2008": Ref(
267 |         "ganusov2008", "Ganusov VV, De Boer RJ",
268 |         "Estimating in vivo death rates of targets due to CD8 T-cell-mediated "
269 |         "killing", "Journal of Virology 82(23):11749-11757", 2008,
270 |         "10.1128/JVI.01128-08", "18815293"),
271 |     "janeway": Ref("janeway", "Murphy K, Weaver C",
272 |         "Janeway's Immunobiology, 9th ed.",
273 |         "Garland Science (textbook)", 2016),
274 |     "histology": Ref("histology", "Mescher AL",
275 |         "Junqueira's Basic Histology (respiratory epithelium, "
276 |         "microvascular dimensions)", "McGraw-Hill (textbook)", 2018),
277 | }
278 |
279 | MODEL_ASSUMPTION = "모델 가정값"
280 |
281 |
282 | # =====
283 | # SECTION 2.  SCALE
284 | # =====
285 |
286 |
287 | class Scale:
288 |     """
289 |     [공간] 1,000 x 1,000 = 1,000,000 칸.  1칸= 15 um (상피세포 직경 10~20um 중앙값)
290 |     -> 도메인 15 mm x 15 mm 상기도 상피 슬랩
291 |     [개체] kappa = 1/1000 (상피세포/면역세포/바이러스 공통)
292 |     [시간] 1 tick = 6 분 = 0.1 시간, 1일= 240 tick
293 |     """
294 |     GRID = 1000
295 |     SITES = GRID * GRID
296 |     SITE_UM = 15.0
297 |     DOMAIN_MM = GRID * SITE_UM / 1000.0
298 |
299 |     KAPPA = 1.0 / 1000.0
300 |     INV_KAPPA = 1000.0
301 |
302 |     DT_MIN = 6.0
303 |     DT_HOUR = 0.1
304 |     TICKS_PER_HOUR = 10
305 |     TICKS_PER_DAY = 240

```

```

306 |
307 | # 조직 배치: 20 행 주기 중 8 행 상피(40%), 12 행 간질(혈관/림프관 포함)
308 | EPI_PERIOD = 20
309 | EPI_ROWS = 8
310 |
311 | # 화학신호 농도장 해상도: 5x5 칸 = 75 um 단위 -> 200 x 200
312 | FIELD_BIN = 5
313 | FIELD_N = GRID // FIELD_BIN
314 | FIELD_UM = SITE_UM * FIELD_BIN
315 |
316 | @staticmethod
317 | def per_tick_from_per_day(rate_per_day: float) -> float:
318 |     return 1.0 - math.exp(-rate_per_day / Scale.TICKS_PER_DAY)
319 |
320 | @staticmethod
321 | def per_tick_from_half-life_h(half-life_hours: float) -> float:
322 |     k = math.log(2.0) / half-life_hours
323 |     return 1.0 - math.exp(-k * Scale.DT_HOUR)
324 |
325 | @staticmethod
326 | def speed_to_sites_per_tick(speed_um_per_min: float) -> float:
327 |     """
328 |     논문의 이동 '속도' (um/min) -> tick 당 이동 거리(칸).
329 |     체계적 이동에서는 방향이 신호로 결정되므로 속도는 그대로 경로 길이가 된다.
330 |     (무작위보행처럼 운동성계수로 환산하지 않는다 - 이것이 두 모델의 정의 차이)
331 |     """
332 |     return speed_um_per_min * Scale.DT_MIN / Scale.SITE_UM
333 |
334 |
335 | # =====
336 | # SECTION 3. REAL VALUES (논문값) & PARAMETER TABLE
337 | # =====
338 |
339 | REAL = {
340 |     # 혈액 조성
341 |     "wbc_per_ml": 5.0e6, "neut_frac": 0.60, "mono_frac": 0.06, "lymph_frac": 0.30,
342 |     "cd4_of_lymph": 0.45, "cd8_of_lymph": 0.25, "bcell_of_lymph": 0.12,
343 |     "nk_of_lymph": 0.13,
344 |
345 |     # 바이러스
346 |     "virion_nm": 100.0, "eclipse_h": 6.0, "productive_h": 5.0,
347 |     "infected_lifetime_h": 11.0, "virus_half-life_h": 3.0, "R0_within_host": 22.0,
348 |     "burst_particles": 5000.0, "target_cells_ur": 4.0e8,
349 |     "inoculum_tcid50": 1.0e5, "virion_D_um2_s": 1.0,
350 |
351 |     # 이동속도 (um/min)
352 |     "neut_speed": 15.0, "mono_speed": 2.5, "nk_speed": 8.0,
353 |     "naiveT_speed": 11.0, "effT_speed": 5.0, "b_speed": 6.0,
354 |     "neut_chemotaxis_speed": 7.0, # IL-8 구배 하 3D 이동속도 2~7 um/min
355 |     "rolling_velocity_um_s": 10.0, # 세정맥 내 rolling 속도 5~50 um/s
356 |
357 |     # 주화성 지수 (실제)
358 |     "chemotactic_index_real": 0.35, # 0.25~0.5

```

```

359 |
360 | # 수명 (일)
361 | "neut_lifespan_d": 1.5, "mono_lifespan_d": 3.0, "nk_lifespan_d": 14.0,
362 | "naive_lymph_lifespan_d": 180.0, "effector_T_half-life_d": 1.5,
363 | "eff_t_half-life_expand_d": 4.0,
364 |
365 | # 후천면역
366 | "dc_migration_h": 18.0, "priming_h": 24.0,
367 | "cd8_doubling_h": 7.0, "cd4_doubling_h": 9.0, "b_doubling_h": 10.0,
368 | "cd8_max_divisions": 11.0, "cd4_max_divisions": 9.0, "b_max_divisions": 10.0,
369 | "expansion_program_d": 6.0, "net_expansion_factor": 0.53,
370 | "contraction_rate_d": 0.75, "memory_fraction": 0.075,
371 | "plasmablast_half-life_d": 3.5, "gc_pc_half-life_d": 7.0,
372 | "igm_half-life_d": 5.0, "igg_half-life_d": 21.0,
373 | "cd8_precursor_systemic": 5.0e4, "cd4_precursor_systemic": 1.0e5,
374 | "b_precursor_systemic": 5.0e4,
375 |
376 | # 화학신호
377 | "chemokine_D_um2_s": 45.0,      # GAG 결합으로 자유확산보다 크게 저해됨
378 | "chemokine_half-life_h": 3.5,
379 | "cytokine_D_um2_s": 120.0,
380 | "cytokine_half-life_h": 1.5,
381 | "ifn_peak_day": 2.0, "antiviral_state_h": 5.0,
382 | "cytokine_peak_day": 2.0,
383 | "c3_mg_per_ml": 1.2,
384 | "symptom_peak_day": 3.0, "shedding_days": 4.8,
385 |
386 | # 혈관
387 | "capillary_spacing_um": 300.0,   # 조직 내 모세혈관 간격 20~300 um
388 | "diapedesis_min": 5.0,          # 혈관벽 통과 2~10 분
389 | }
390 |
391 |
392 | def _sp(v: float) -> float:
393 |     return Scale.speed_to_sites_per_tick(v)
394 |
395 |
396 | @dataclass(frozen=True)
397 | class ParamRow:
398 |     item: str
399 |     real: str
400 |     unit: str
401 |     source: str
402 |     scaling: str
403 |     sim: str
404 |     note: str = ""
405 |
406 |
407 | PARAM_TABLE: List[ParamRow] = [
408 |     ParamRow("기준 부피", "1", "mL 상당", "본 모델 정의", "기준",
409 |             f"{Scale.SITES:,}칸 (1000x1000)", "사양서 ①②"),
410 |     ParamRow("격자 1칸", "10~20", "um (상피세포 직경)", "histology", "공간축소",
411 |             f"{Scale.SITE_UM:.0f} um -> 도메인 {Scale.DOMAIN_MM:.0f}x{Scale.DOMAIN_MM:.0f} mm", ""),

```



```

412 ParamRow("표적 상피세포", f"{REAL['target_cells_urt']:.1e}", "cells", "baccam2006",
413         "kappa=1/1000", "약 397,000 칸 (격자의 40%, 림프절 영역 제외)",
414         "Baccam 2006 T0 = 4x10^8"),
415 ParamRow("총 백혈구", f"{REAL['wbc_per_ml']:.1e}", "cells/mL", "dacie",
416         "kappa=1/1000", "5,000 agent", "4.0~11.0 x10^9/L 중앙 부근"),
417 ParamRow("중성구 비율", "40~70 (60 채택)", "%", "dacie", "동일", "3,000 agent", ""),
418 ParamRow("단핵구 비율", "2~10 (6 채택)", "%", "dacie", "동일", "300 agent", ""),
419 ParamRow("NK 세포 비율", "림프구의 5~20 (13 채택)", "%", "bisset2004", "동일",
420         "195 agent", ""),
421 ParamRow("조력 T 세포 비율", "림프구의 45%", "%", "bisset2004", "동일", "675 agent", ""),
422 ParamRow("살해 T 세포 비율", "림프구의 25%", "%", "bisset2004", "동일", "375 agent", ""),
423 ParamRow("B 세포 비율", "림프구의 12%", "%", "bisset2004", "동일", "180 agent", ""),
424 ParamRow("병원체 초기량", f"{REAL['inoculum_tcid50']:.1e}", "TCID50", "memoli2015",
425         "kappa=1/1000", "100 agent", "인체감염모델 접종량 10^4~10^6"),
426 ParamRow("바이러스 크기", "80~120", "nm", "harris2006", "공간축소",
427         "점입자 (1 칸 15um 대비 무시)", "세포>>바이러스 관계 유지"),
428 ParamRow("바이러스 확산", f"{REAL['virion_D_um2_s']}", "um^2/s", "olmsted2001",
429         "시간/공간축소", "확산+점액수송 (목표탐색 없음)", "사양서 ㉑"),
430 ParamRow("바이러스 잠복기", f"{REAL['eclipse_h']}", "시간", "baccam2006", "시간축소",
431         f"{int(REAL['eclipse_h']*Scale.TICKS_PER_HOUR)} tick", ""),
432 ParamRow("감염세포 생산기간", f"{REAL['productive_h']}", "시간", "baccam2006",
433         "시간축소", f"{int(REAL['productive_h']*Scale.TICKS_PER_HOUR)} tick",
434         "감염세포 평균수명 11h = 6h + 5h"),
435 ParamRow("바이러스 반감기", f"{REAL['virus_half-life_h']}", "시간", "baccam2006",
436         "시간축소",
437         f"{Scale.per_tick_from_half-life_h(REAL['virus_half-life_h']):.4f}/tick", ""),
438 ParamRow("감염세포당 생산량", "10^3~10^4", "particles/cell", "mohler2005",
439         "kappa+감염성비율", "25 agent/칸", "입자:감염단위 ~100:1"),
440 ParamRow("체내 R0", f"{REAL['R0_within_host']}", "-", "baccam2006", "동일",
441         "창발 결과로 검증", ""),
442 # ---- 이동 ----
443 ParamRow("중성구 이동속도 (주화성)", "2~7 (7 채택)", "um/min", "sackmann2012",
444         "시간/공간축소", f"{_sp(REAL['neut_chemotaxis_speed']):.1f} 칸/tick",
445         "IL-8 구배 하 3D 콜라겐 내 속도. 무구배 대조군은 0~1 um/min"),
446 ParamRow("중성구 최대속도", f"{REAL['neut_speed']}", "um/min", "lammermann2013",
447         "시간/공간축소", f"{_sp(REAL['neut_speed']):.1f} 칸/tick", "생체 내 10~20"),
448 ParamRow("단핵구/대식세포 속도", f"{REAL['mono_speed']}", "um/min",
449         "lammermann2013", "시간/공간축소", f"{_sp(REAL['mono_speed']):.1f} 칸/tick", ""),
450 ParamRow("NK 세포 속도", f"{REAL['nk_speed']}", "um/min", "bhat2007",
451         "시간/공간축소", f"{_sp(REAL['nk_speed']):.1f} 칸/tick", ""),
452 ParamRow("나이프 T 세포 속도", f"{REAL['naiveT_speed']}", "um/min", "miller2002",
453         "시간/공간축소", f"{_sp(REAL['naiveT_speed']):.1f} 칸/tick", "림프절 2 광자"),
454 ParamRow("효과기 CD8 조직내 속도", f"{REAL['effT_speed']}", "um/min", "ariotti2012",
455         "시간/공간축소", f"{_sp(REAL['effT_speed']):.1f} 칸/tick", ""),
456 ParamRow("B 세포 속도", f"{REAL['b_speed']}", "um/min", "miller2002",
457         "시간/공간축소", f"{_sp(REAL['b_speed']):.1f} 칸/tick", ""),
458 ParamRow("혈관 내 rolling 속도", "5~50 (10 채택)", "um/s", "ley2007",
459         "시간/공간축소", f"{10.0*60*Scale.DT_MIN/Scale.SITE_UM:.0f} 칸/tick",
460         "혈관외유출 가능한 marginated 세포 기준"),
461 ParamRow("혈관벽 통과 (diapedesis)", "2~10", "분", "nourshargh2010", "시간축소",
462         "1 tick 내 완료", ""),
463 ParamRow("모세혈관 간격", "20~300", "um", "histology", "공간축소",
464         "혈관 격자 간격 300 um (20 칸)", ""),

```

```

465 | # ---- 화학신호 ----
466 | ParamRow("케모카인 확산계수", "GAG 결합으로 크게 저해", "um^2/s", "proudfoot2003",
467 |         "시간/공간축소",
468 |         f"{REAL['chemokine_D_um2_s']:.0f} um^2/s -> 구배 감식거리 약 0.9 mm",
469 |         "자유확산(~150)이 아닌 haptotactic 고정형 구배 (weber2013)"),
470 | ParamRow("케모카인 반감기", f"{REAL['chemokine_half-life_h']}", "시간",
471 |         MODEL_ASSUMPTION, "시간축소",
472 |         f"{Scale.per_tick_from_half-life_h(REAL['chemokine_half-life_h']):.4f}/tick",
473 |         "조직 내 케모카인 turnover 정량치 부재"),
474 | ParamRow("사이토카인 반감기", f"{REAL['cytokine_half-life_h']}", "시간",
475 |         "hayden1998", "시간축소",
476 |         f"{Scale.per_tick_from_half-life_h(REAL['cytokine_half-life_h']):.4f}/tick",
477 |         "IL-6/TNF 비강세척액 동태"),
478 | ParamRow("사이토카인 정점", f"{REAL['cytokine_peak_day']}", "일", "hayden1998",
479 |         "시간축소", "창발 결과로 검증", ""),
480 | ParamRow("인터페론 정점", f"{REAL['ifn_peak_day']}", "일", "hayden1998",
481 |         "시간축소", "창발 결과로 검증", ""),
482 | ParamRow("항바이러스 상태 확립", f"{REAL['antiviral_state_h']}", "시간",
483 |         "ivashkiv2014", "시간축소",
484 |         f"{int(REAL['antiviral_state_h']*Scale.TICKS_PER_HOUR)} tick", ""),
485 | ParamRow("보체 C3 혈장농도", f"{REAL['c3_mg_per_ml']}", "mg/mL", "janeway",
486 |         "정규화", "활성도 0~1", ""),
487 | # ---- 수명 ----
488 | ParamRow("중성구 수명", f"{REAL['neut_lifespan_d']}", "일", "pillay2010",
489 |         "시간축소", f"{int(REAL['neut_lifespan_d']*Scale.TICKS_PER_DAY)} tick",
490 |         "혈중 t1/2 7~19h vs 5.4 일 보고 상충, 조직 1~2 일 채택"),
491 | ParamRow("단핵구 수명", f"{REAL['mono_lifespan_d']}", "일", "patel2017",
492 |         "시간축소", f"{int(REAL['mono_lifespan_d']*Scale.TICKS_PER_DAY)} tick", ""),
493 | ParamRow("NK 세포 수명", f"{REAL['nk_lifespan_d']}", "일", "zhang2007",
494 |         "시간축소", f"{int(REAL['nk_lifespan_d']*Scale.TICKS_PER_DAY)} tick", ""),
495 | # ---- 후천면역 ----
496 | ParamRow("DC 항원수송 지연", f"{REAL['dc_migration_h']}", "시간", "lawrence2005",
497 |         "시간축소", f"{int(REAL['dc_migration_h']*Scale.TICKS_PER_HOUR)} tick",
498 |         "12~24h 범위 중앙"),
499 | ParamRow("T 세포 프라이밍", f"{REAL['priming_h']}", "시간", "mempel2004",
500 |         "시간축소", f"{int(REAL['priming_h']*Scale.TICKS_PER_HOUR)} tick",
501 |         "3 단계 priming 후 첫 분열까지"),
502 | ParamRow("CD8 분열주기", f"{REAL['cd8_doubling_h']}", "시간", "lawrence2005",
503 |         "시간축소", f"{int(REAL['cd8_doubling_h']*Scale.TICKS_PER_HOUR)} tick", ""),
504 | ParamRow("CD4 분열주기", f"{REAL['cd4_doubling_h']}", "시간", "mempel2004",
505 |         "시간축소", f"{int(REAL['cd4_doubling_h']*Scale.TICKS_PER_HOUR)} tick", ""),
506 | ParamRow("B 세포 분열주기", f"{REAL['b_doubling_h']}", "시간", "wrammert2008",
507 |         "시간축소", f"{int(REAL['b_doubling_h']*Scale.TICKS_PER_HOUR)} tick", ""),
508 | ParamRow("자율증식 프로그램", f"{REAL['expansion_program_d']}", "일",
509 |         "vanstipdonk2001", "시간축소",
510 |         f"{int(REAL['expansion_program_d']*Scale.TICKS_PER_DAY)} tick",
511 |         "항원은 개시신호, 이후 자율 증식 후 수축"),
512 | ParamRow("CD8 나이브 전구세포", f"{REAL['cd8_precursor_systemic']:.1e}",
513 |         "cells(전신)", "alanio2010", "kappa=1/1000", "50 agent", ""),
514 | ParamRow("CD4 나이브 전구세포", f"{REAL['cd4_precursor_systemic']:.1e}",
515 |         "cells(전신)", "blattman2002", "kappa=1/1000", "100 agent", ""),
516 | ParamRow("B 나이브 전구세포", f"{REAL['b_precursor_systemic']:.1e}",
517 |         "cells(전신)", "blattman2002", "kappa=1/1000", "50 agent", ""),

```

```

518 | ParamRow("IgM 반감기", f"{REAL['igm_half-life_d']}", "일", "janeway", "시간축소",
519 |         f"{Scale.per_tick_from_half-life_h(REAL['igm_half-life_d']*24):.5f}/tick", ""),
520 | ParamRow("IgG 반감기", f"{REAL['igg_half-life_d']}", "일", "janeway", "시간축소",
521 |         f"{Scale.per_tick_from_half-life_h(REAL['igg_half-life_d']*24):.5f}/tick", ""),
522 | ParamRow("기억세포 형성비율", "5~10", "% (정점 대비)", "kaech2002", "동일",
523 |         f"{REAL['memory_fraction']*100:.1f}%", ""),
524 | # ---- 모델 가정값 ----
525 | ParamRow("주화성 지수(CI)", "0.25~0.5 (실제 호중구)", "-",
526 |         "sackmann2012/boneschansker2014", "실험조건으로 대체", "1.0 (완전 지향)",
527 |         "★ 실제 면역세포는 '편향된 무작위보행'이며 CI<1 이다. ",
528 |         "본 실험은 '체계적으로만' 이동하는 조건이므로 CI=1 의 이상적 상한을 사용한다"),
529 | ParamRow("혈관 면적비", "-", "-", MODEL_ASSUMPTION, "-", "격자의 약 7%",
530 |         "간질층 내 수평 세정맥 + 수직 연결혈관 네트워크"),
531 | ParamRow("혈관외유출 위치", "-", "-", MODEL_ASSUMPTION, "-", "케모카인 0.02 (정규화)",
532 |         "국소 케모카인이 역시 초과 시 결정론적으로 유출"),
533 | ParamRow("신호 감지 반경", "-", "-", MODEL_ASSUMPTION, "-", "75 um (필드 1 칸)",
534 |         "세포는 자기 주변 8 방향만 감지. 전역 좌표 접근 불가 (사양서 ㉔)"),
535 | ParamRow("식세포 접촉 반경", "-", "-", MODEL_ASSUMPTION, "-", "동일 격자칸 (15um)",
536 |         "중성구 직경 ~12um"),
537 | ParamRow("중성구 식균 용량", "-", "-", MODEL_ASSUMPTION, "-", "10 회", "기능소진"),
538 | ParamRow("NK 접촉당 사멸확률", "-", "-", MODEL_ASSUMPTION, "-", "0.25/tick",
539 |         "NK-표적 접합 20~60 분(bhat2007) 을 6 분 tick 으로 환산"),
540 | ParamRow("CTL 접촉당 사멸확률", "-", "-", MODEL_ASSUMPTION, "-", "0.80/tick",
541 |         "결과를 regoes2007/ganusov2008 범위와 대조 검증"),
542 | ParamRow("상피 재생속도", "-", "-", MODEL_ASSUMPTION, "-",
543 |         "0.02/일 x (1-염증)", "급성 염증기에는 기저세포 증식 억제"),
544 | ]
545 |
546 |
547 | # =====
548 | # SECTION 4.  PARAMS
549 | # =====
550 |
551 |
552 | @dataclass
553 | class Params:
554 |     # 초기 개체수 (kappa = 1/1000)
555 |     n_virus0: int = 100
556 |     n_neutrophil: int = 3000
557 |     n_monocyte: int = 300
558 |     n_nk: int = 195
559 |     n_helper_t: int = 675
560 |     n_killer_t: int = 375
561 |     n_bcell: int = 180
562 |
563 |     precursor_cd8: float = 50.0
564 |     precursor_cd4: float = 100.0
565 |     precursor_b: float = 50.0
566 |
567 |     # 바이러스
568 |     eclipse_ticks: int = int(REAL["eclipse_h"] * Scale.TICKS_PER_HOUR)
569 |     productive_ticks: int = int(REAL["productive_h"] * Scale.TICKS_PER_HOUR)
570 |     virus_decay_p: float = Scale.per_tick_from_half-life_h(REAL["virus_half-life_h"])

```

```

571 | burst_agents_per_site: float = 25.0
572 | p_infect: float = 0.15
573 | virus_sigma: float = 30.0          # 확산 + 점액섬모 수송 (등방, 목표탐색 없음)
574 | max_virions: int = 800_000
575 |
576 | # ---- 화학신호 필드 ----
577 | # 케모카인: 이동 방향 결정 (사양서 ⑭⑯)
578 | chemo_from_infected: float = 0.020      # 생산 감염칸 1개가 tick 당 방출
579 | chemo_from_phagocyte: float = 0.004     # 활성 식세포의 2차 방출 (사양서 19)
580 | chemo_decay: float = Scale.per_tick_from_half-life_h(REAL["chemokine_half-life_h"])
581 | chemo_sigma_field: float = 2.4          # tick 당 확산 sigma (필드칸)
582 | # 사이토카인: 면역반응 조절 (사양서 ㉓) - 이동방향 결정에 사용하지 않음
583 | cyto_from_infected: float = 0.020
584 | cyto_decay: float = Scale.per_tick_from_half-life_h(REAL["cytokine_half-life_h"])
585 | cyto_sigma_field: float = 3.8
586 | cyto_activation_threshold: float = 0.004 # 식세포 활성화 역치
587 | # 인터페론: 항바이러스 상태
588 | ifn_production: float = 0.02
589 | ifn_decay_p: float = Scale.per_tick_from_half-life_h(2.0)
590 | ifn_sigma_field: float = 1.6
591 | antiviral_rate: float = 1.0 / (REAL["antiviral_state_h"] * Scale.TICKS_PER_HOUR)
592 | antiviral_decay: float = 1.0 / (24.0 * Scale.TICKS_PER_HOUR)
593 | ifn_block_release: float = 0.80
594 |
595 | # ---- 이동 (체계적) ----
596 | speed_neut: float = Scale.speed_to_sites_per_tick(REAL["neut_chemotaxis_speed"])
597 | speed_mono: float = Scale.speed_to_sites_per_tick(REAL["mono_speed"])
598 | speed_nk: float = Scale.speed_to_sites_per_tick(REAL["nk_speed"])
599 | speed_naive_t: float = Scale.speed_to_sites_per_tick(REAL["naiveT_speed"])
600 | speed_eff_t: float = Scale.speed_to_sites_per_tick(REAL["effT_speed"])
601 | speed_b: float = Scale.speed_to_sites_per_tick(REAL["b_speed"])
602 | vessel_speed: float = REAL["rolling_velocity_um_s"] * 60.0 * Scale.DT_MIN \
603 |     / Scale.SITE_UM          # 240 칸/tick
604 | extravasation_threshold: float = 0.02    # 국소 케모카인 역치 (정규화)
605 | reentry_threshold: float = 0.002         # 이하로 떨어지면 혈관 복귀(역주행 해소)
606 |
607 | # ---- 보체 / 항체 ----
608 | complement_on: float = 0.06
609 | complement_off: float = 0.02
610 | complement_lysis: float = 0.010
611 | complement_opsonin: float = 0.35
612 | ab_per_pc_igm: float = 5.0e-6
613 | ab_per_pc_igg: float = 2.5e-5
614 | igm_decay_p: float = Scale.per_tick_from_half-life_h(REAL["igm_half-life_d"] * 24)
615 | igg_decay_p: float = Scale.per_tick_from_half-life_h(REAL["igg_half-life_d"] * 24)
616 | ab_kd: float = 2.0
617 | ab_neutralize_max: float = 0.030
618 | ab_opsonin: float = 0.35
619 | ab_block_max: float = 0.95
620 | ab_block_kd: float = 3.0
621 | ab_detect_threshold: float = 0.5
622 |
623 | # ---- 세포 기능 ----

```

```

624 | neut_phago_p: float = 0.45
625 | neut_capacity: int = 10
626 | mono_phago_p: float = 0.55
627 | mono_capacity: int = 25
628 | nk_kill_p: float = 0.25
629 | ctl_kill_p: float = 0.80
630 |
631 | # ---- 수명 (tick) ----
632 | life_neut: int = int(REAL["neut_lifespan_d"] * Scale.TICKS_PER_DAY)
633 | life_mono: int = int(REAL["mono_lifespan_d"] * Scale.TICKS_PER_DAY)
634 | life_nk: int = int(REAL["nk_lifespan_d"] * Scale.TICKS_PER_DAY)
635 | life_naive: int = int(REAL["naive_lymph_lifespan_d"] * Scale.TICKS_PER_DAY)
636 | eff_t_death_expand: float = Scale.per_tick_from_half-life_h(
637 |     REAL["eff_t_half-life_expand_d"] * 24)
638 | eff_t_death_contract: float = Scale.per_tick_from_half-life_h(
639 |     REAL["effector_T_half-life_d"] * 24)
640 |
641 | # ---- 염증 / 동원 / 재생 ----
642 | inflam_gain_virus: float = 1.0 / 8000.0
643 | inflam_gain_infected: float = 1.0 / 40000.0
644 | inflam_decay: float = 0.010
645 | recruit_neut_max: float = 3.0
646 | recruit_mono_max: float = 9.0
647 | recruit_nk_max: float = 5.0
648 | recruit_rate: float = 0.020
649 | damage_rate: float = 0.0015
650 | epi_regen_p: float = Scale.per_tick_from_per_day(0.02)
651 |
652 | # ---- 림프절 / 후천면역 ----
653 | dc_delay_ticks: int = int(REAL["dc_migration_h"] * Scale.TICKS_PER_HOUR)
654 | priming_ticks: int = int(REAL["priming_h"] * Scale.TICKS_PER_HOUR)
655 | cd8_growth: float = math.log(2.0) / (REAL["cd8_doubling_h"] * Scale.TICKS_PER_HOUR)
656 | cd4_growth: float = math.log(2.0) / (REAL["cd4_doubling_h"] * Scale.TICKS_PER_HOUR)
657 | b_growth: float = math.log(2.0) / (REAL["b_doubling_h"] * Scale.TICKS_PER_HOUR)
658 | cd8_max_expansion: float = 2.0 ** REAL["cd8_max_divisions"]
659 | cd4_max_expansion: float = 2.0 ** REAL["cd4_max_divisions"]
660 | b_max_expansion: float = 2.0 ** REAL["b_max_divisions"]
661 | net_expansion: float = REAL["net_expansion_factor"]
662 | program_ticks: int = int(REAL["expansion_program_d"] * Scale.TICKS_PER_DAY)
663 | contraction_p: float = Scale.per_tick_from_per_day(REAL["contraction_rate_d"])
664 | b_contraction_p: float = Scale.per_tick_from_per_day(0.30)
665 | ef_delay_ticks: int = int(48.0 * Scale.TICKS_PER_HOUR)
666 | gc_delay_ticks: int = int(144.0 * Scale.TICKS_PER_HOUR)
667 | gc_end_ticks: int = int(21.0 * Scale.TICKS_PER_DAY)
668 | ef_diff_rate: float = 0.0016
669 | gc_diff_rate: float = 3.0e-5
670 | pc_short_decay: float = Scale.per_tick_from_per_day(
671 |     math.log(2.0) / REAL["plasmablast_half-life_d"])
672 | pc_gc_decay: float = Scale.per_tick_from_per_day(
673 |     math.log(2.0) / REAL["gc_pc_half-life_d"])
674 | memory_fraction: float = REAL["memory_fraction"]
675 | memory_b_fraction: float = 0.10
676 | emigration_p: float = 1.6e-4

```

```

677 |     emigration_start_h: float = 96.0
678 |     antigen_threshold: float = 20.0
679 |     antigen_decay: float = Scale.per_tick_from_per_day(1.5)
680 |
681 |     max_days: int = 30
682 |     stop_after_clear_days: int = 4
683 |     seed: int = 20260809
684 |
685 |
686 | # =====
687 | # SECTION 5.  SYSTEMATIC MOVEMENT ENGINE    (★ 본 실험의 핵심)
688 | # =====
689 |
690 | # 8방향 단위벡터 (사양서 ⑥: 주변 8방향 농도 측정 후 최대 방향으로 이동)
691 | _R2 = 1.0 / math.sqrt(2.0)
692 | DIR8 = np.array([
693 |     (0.0, 1.0), ( _R2,  _R2), ( 1.0, 0.0), ( _R2, -_R2),
694 |     (0.0, -1.0), (-_R2, -_R2), (-1.0, 0.0), (-_R2,  _R2)], dtype=np.float32)
695 | DIR8_OFF = np.array([
696 |     (0, 1), (1, 1), (1, 0), (1, -1),
697 |     (0, -1), (-1, -1), (-1, 0), (-1, 1)], dtype=np.int32)  # (dcol, drow) 필드단위
698 |
699 |
700 | class SystematicMovementEngine:
701 |     """
702 |     100% 체계적 이동.  난수 발생기를 일절 사용하지 않는다.
703 |
704 |     이동 방향 결정 규칙 (우선순위):
705 |     1) 조직 내 세포 : 자기 위치 주변 8방향의 '케모카인' 농도를 측정하여
706 |         가장 높은 방향으로 이동한다. (사양서 ⑥)
707 |         - 전역 좌표/병원체 위치는 참조하지 않는다. (사양서 ②③⑨)
708 |     2) 구배가 없을 때 : 직전 이동방향 (세포 극성)을 유지한다.
709 |         - 무작위 방향 재선택을 하지 않는다. (사양서 ③⑩)
710 |     3) 혈관 내 세포 : 혈류 방향으로 이동하되, 혈관을 따라 케모카인이
711 |         증가하는 방향이 있으면 그쪽으로 이동한다. (사양서 ③⑪)
712 |     4) 경로의 굴곡 : 8방향 양자화 + 시간에 따라 변하는 농도장 + 혈관 구조
713 |         때문에 자연스럽게 굽는다.  별도의 난수를 넣지 않는다. (사양서 ⑦)
714 |     """
715 |
716 |     MODE = "systematic"
717 |
718 |     def __init__(self):
719 |         self.n_calls = 0
720 |         self.n_agents_moved = 0
721 |         self.n_gradient_guided = 0      # 구배로 방향이 결정된 횟수
722 |         self.n_persistence = 0         # 구배 없어 극성 유지한 횟수
723 |         self.n_flow = 0                # 혈류로 이동한 횟수
724 |
725 |     # -----
726 |     @staticmethod
727 |     def sample_8(field_stack: np.ndarray, fr: np.ndarray, fc: np.ndarray
728 |         ) -> np.ndarray:
729 |         """

```

```

730 |         미리 shift 해 둔 8 장의 농도장에서 각 세포 위치의 8 방향 값을 읽는다.
731 |         field_stack shape = (8, FIELD_N, FIELD_N)
732 |         반환 shape = (N, 8)
733 |         """
734 |         return field_stack[:, fr, fc].T
735 |
736 | # -----
737 | def move_tissue(self, x, y, heading, speed, chemo_stack, fr, fc):
738 |     """조직 내 세포: 8 방향 케모카인 argmax 로 방향 결정"""
739 |     n = x.size
740 |     if n == 0:
741 |         return
742 |     self.n_calls += 1
743 |     self.n_agents_moved += n
744 |
745 |     vals = self.sample_8(chemo_stack, fr, fc)          # (N,8)
746 |     best = np.argmax(vals, axis=1)
747 |     vmax = vals[np.arange(n), best]
748 |     vmin = vals.min(axis=1)
749 |     has_grad = (vmax - vmin) > 1e-9                  # 감지 가능한 구배 존재
750 |
751 |     d = DIR8[best]                                    # (N,2)
752 |     new_head = np.where(has_grad[:, None], d, heading)
753 |     # 극성이 아직 정해지지 않은 세포(heading=0)는 신호가 생길 때까지 정지
754 |     zero = (np.abs(new_head).sum(axis=1) < 1e-9)
755 |     heading[:] = new_head
756 |
757 |     self.n_gradient_guided += int(has_grad.sum())
758 |     self.n_persistence += int((~has_grad & ~zero).sum())
759 |
760 |     step = np.where(zero, 0.0, speed)
761 |     x += (new_head[:, 0] * step).astype(x.dtype)
762 |     y += (new_head[:, 1] * step).astype(y.dtype)
763 |     self._bound(x, y)
764 |
765 | # -----
766 | def move_vessel(self, x, y, heading, flow_dx, flow_dy, speed,
767 |                 chemo_stack, fr, fc):
768 |     """
769 |     혈관 내 세포: 기본은 혈류 방향.
770 |     단, 혈관을 따라 케모카인이 뚜렷이 증가하는 방향이 있으면 그쪽 우선.
771 |     """
772 |     n = x.size
773 |     if n == 0:
774 |         return
775 |     self.n_calls += 1
776 |     self.n_agents_moved += n
777 |     self.n_flow += n
778 |
779 |     vals = self.sample_8(chemo_stack, fr, fc)
780 |     best = np.argmax(vals, axis=1)
781 |     vmax = vals[np.arange(n), best]
782 |     here = vals[:, 0] * 0.0

```

```

783 |         # 현재 위치 농도는 8 방향 중앙값 대응으로 min 을 사용 (구배 유무 판정용)
784 |         vmin = vals.min(axis=1)
785 |         strong = (vmax - vmin) > 1e-6
786 |
787 |         d = DIR8[best]
788 |         dx = np.where(strong, d[:, 0], flow_dx)
789 |         dy = np.where(strong, d[:, 1], flow_dy)
790 |         heading[:, 0] = dx
791 |         heading[:, 1] = dy
792 |         x += (dx * speed).astype(x.dtype)
793 |         y += (dy * speed).astype(y.dtype)
794 |         self._bound(x, y)
795 |
796 |         # -----
797 |         @staticmethod
798 |         def _bound(x, y):
799 |             lim = float(Scale.GRID) - 1e-3
800 |             np.abs(x, out=x)
801 |             np.abs(y, out=y)
802 |             over = x > lim
803 |             if over.any():
804 |                 x[over] = 2.0 * lim - x[over]
805 |             over = y > lim
806 |             if over.any():
807 |                 y[over] = 2.0 * lim - y[over]
808 |             np.clip(x, 0.0, lim, out=x)
809 |             np.clip(y, 0.0, lim, out=y)
810 |
811 |
812 | # =====
813 | # SECTION 6.  ENVIRONMENT  (조직 + 혈관 네트워크 + 림프절 + 화학신호장)
814 | # =====
815 |
816 | STROMA, HEALTHY, ECLIPSE, PRODUCTIVE, DEAD, VESSEL, LYMPH = 0, 1, 2, 3, 4, 5, 6
817 |
818 | # 림프절 영역 (사양서 ⑩㉓)
819 | LN_R0, LN_R1 = 460, 540
820 | LN_C0, LN_C1 = 40, 120
821 |
822 |
823 | class Environment:
824 |
825 |     def __init__(self, p: Params, rng: np.random.Generator):
826 |         self.p = p
827 |         self.rng = rng
828 |         n = Scale.GRID
829 |
830 |         # ---- 1. 조직 기본 배치 (상피 밴드 40%) ----
831 |         rows = np.arange(n)
832 |         epi_row = (rows % Scale.EPI_PERIOD) < Scale.EPI_ROWS
833 |         base = np.where(epi_row, HEALTHY, STROMA).astype(np.uint8)
834 |         state = np.repeat(base, n).reshape(n, n)
835 |

```



```

836 | # ---- 2. 혈관 네트워크 (사양서 10⑳) ----
837 | # 수평 세정맥: 간질층 내 20 행 주기로 1 행 (간격 300 um)
838 | # 수직 연결혈관: 50 열 주기로 1 열 (간격 750 um)
839 | # 혈류 방향: 수평혈관은 행 인덱스에 따라 좌우 교대, 수직은 상행
840 | vessel_rows = np.flatnonzero((rows % Scale.EPI_PERIOD) == 13)
841 | state[vessel_rows, :] = VESSEL
842 | vessel_cols = np.arange(25, n, 50)
843 | for c in vessel_cols:
844 |     col_mask = ~epi_row                # 간질층에서만 수직혈관 통과
845 |     state[col_mask, c] = VESSEL
846 |
847 | # 혈류 방향 필드 (혈관 칸에만 의미)
848 | self.flow_dx = np.zeros((n, n), dtype=np.float32)
849 | self.flow_dy = np.zeros((n, n), dtype=np.float32)
850 | for i, r in enumerate(vessel_rows):
851 |     self.flow_dx[r, :] = 1.0 if (i % 2 == 0) else -1.0
852 | for c in vessel_cols:
853 |     m = ~epi_row
854 |     self.flow_dx[m, c] = 0.0
855 |     self.flow_dy[m, c] = 1.0
856 |
857 | # ---- 3. 림프절 영역 ----
858 | state[LN_R0:LN_R1, LN_C0:LN_C1] = LYMPH
859 | self.flow_dx[LN_R0:LN_R1, LN_C0:LN_C1] = 0.0
860 | self.flow_dy[LN_R0:LN_R1, LN_C0:LN_C1] = 0.0
861 |
862 | self.state = state.reshape(-1)
863 | self.flow_dx = self.flow_dx.reshape(-1)
864 | self.flow_dy = self.flow_dy.reshape(-1)
865 |
866 | self.is_vessel = (self.state == VESSEL)
867 | self.vessel_sites = np.flatnonzero(self.is_vessel)
868 | self.n_target = int(np.count_nonzero(self.state == HEALTHY))
869 | self.n_vessel = int(self.is_vessel.sum())
870 |
871 | # 림프절 출구(혈관) 지점: 림프절에 인접한 혈관 칸
872 | ln_rows = np.arange(LN_R0 - 25, LN_R1 + 25)
873 | vr = self.vessel_sites // Scale.GRID
874 | vc = self.vessel_sites - vr * Scale.GRID
875 | near = ((vr >= ln_rows[0]) & (vr <= ln_rows[-1])) &
876 |         (vc >= LN_C0 - 30) & (vc <= LN_C1 + 60)
877 | self.ln_exit_sites = self.vessel_sites[near]
878 | if self.ln_exit_sites.size == 0:
879 |     self.ln_exit_sites = self.vessel_sites[:1000]
880 |
881 | # ---- 4. 감염세포 리스트 ----
882 | self.inf_site = np.empty(0, dtype=np.int32)
883 | self.inf_timer = np.empty(0, dtype=np.int16)
884 | self.inf_stage = np.empty(0, dtype=np.uint8)
885 | self.inf_alive = np.empty(0, dtype=bool)
886 |
887 | # ---- 5. 화학신호 농도장 (200 x 200, 1칸 = 75 um) ----
888 | f = Scale.FIELD_N

```

```

889 |         self.chemokine = np.zeros((f, f), dtype=np.float32)
890 |         self.cytokine = np.zeros((f, f), dtype=np.float32)
891 |         self.ifn = np.zeros((f, f), dtype=np.float32)
892 |         self.antiviral = np.zeros((f, f), dtype=np.float32)
893 |
894 |         # ---- 6. 전신 스칼라 ----
895 |         self.complement = 0.0
896 |         self.inflammation = 0.0
897 |         self.tissue_damage = 0.0
898 |         self.n_dead_epi = 0
899 |         self.n_regen = 0
900 |
901 |     # -----
902 |     @staticmethod
903 |     def site_of(x: np.ndarray, y: np.ndarray) -> np.ndarray:
904 |         col = x.astype(np.int32)
905 |         row = y.astype(np.int32)
906 |         np.clip(col, 0, Scale.GRID - 1, out=col)
907 |         np.clip(row, 0, Scale.GRID - 1, out=row)
908 |         return row * Scale.GRID + col
909 |
910 |     @staticmethod
911 |     def field_of_xy(x: np.ndarray, y: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
912 |         fc = (x / Scale.FIELD_BIN).astype(np.int32)
913 |         fr = (y / Scale.FIELD_BIN).astype(np.int32)
914 |         np.clip(fc, 0, Scale.FIELD_N - 1, out=fc)
915 |         np.clip(fr, 0, Scale.FIELD_N - 1, out=fr)
916 |         return fr, fc
917 |
918 |     @staticmethod
919 |     def field_of_site(site: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
920 |         row = site // Scale.GRID
921 |         col = site - row * Scale.GRID
922 |         return row // Scale.FIELD_BIN, col // Scale.FIELD_BIN
923 |
924 |     # -----
925 |     def infect_sites(self, sites: np.ndarray) -> int:
926 |         if sites.size == 0:
927 |             return 0
928 |         self.state[sites] = ECLIPSE
929 |         k = sites.size
930 |         self.inf_site = np.concatenate([self.inf_site, sites.astype(np.int32)])
931 |         self.inf_timer = np.concatenate([self.inf_timer, np.zeros(k, np.int16)])
932 |         self.inf_stage = np.concatenate([self.inf_stage, np.zeros(k, np.uint8)])
933 |         self.inf_alive = np.concatenate([self.inf_alive, np.ones(k, bool)])
934 |         return k
935 |
936 |     def kill_sites(self, sites: np.ndarray) -> int:
937 |         if sites.size == 0:
938 |             return 0
939 |         sites = np.unique(sites)
940 |         m = (self.state[sites] == ECLIPSE) | (self.state[sites] == PRODUCTIVE)
941 |         sites = sites[m]

```

```

942 |         if sites.size == 0:
943 |             return 0
944 |         self.state[sites] = DEAD
945 |         self.n_dead_epi += sites.size
946 |         return sites.size
947 |
948 | def advance_infected(self) -> Tuple[np.ndarray, int]:
949 |     if self.inf_site.size == 0:
950 |         return np.empty(0, np.int32), 0
951 |     st = self.state[self.inf_site]
952 |     self.inf_alive &= (st == ECLIPSE) | (st == PRODUCTIVE)
953 |     alive = self.inf_alive
954 |     if not alive.any():
955 |         self._compact()
956 |         return np.empty(0, np.int32), 0
957 |
958 |     self.inf_timer[alive] += 1
959 |     to_prod = alive & (self.inf_stage == 0) & (self.inf_timer >= self.p.eclipse_ticks)
960 |     if to_prod.any():
961 |         self.inf_stage[to_prod] = 1
962 |         self.inf_timer[to_prod] = 0
963 |         self.state[self.inf_site[to_prod]] = PRODUCTIVE
964 |     to_dead = alive & (self.inf_stage == 1) & (self.inf_timer >= self.p.productive_ticks)
965 |     n_apop = int(to_dead.sum())
966 |     if n_apop:
967 |         self.state[self.inf_site[to_dead]] = DEAD
968 |         self.inf_alive[to_dead] = False
969 |         self.n_dead_epi += n_apop
970 |
971 |     prod_sites = self.inf_site[self.inf_alive & (self.inf_stage == 1)]
972 |     if self.inf_alive.size > 20000 and self.inf_alive.mean() < 0.5:
973 |         self._compact()
974 |     return prod_sites, n_apop
975 |
976 | def _compact(self):
977 |     k = self.inf_alive
978 |     self.inf_site = self.inf_site[k]
979 |     self.inf_timer = self.inf_timer[k]
980 |     self.inf_stage = self.inf_stage[k]
981 |     self.inf_alive = self.inf_alive[k]
982 |
983 | # -----
984 | @staticmethod
985 | def _diffuse(a: np.ndarray, sigma: float) -> np.ndarray:
986 |     from scipy.ndimage import gaussian_filter
987 |     return gaussian_filter(a, sigma=sigma, mode="nearest")
988 |
989 | def update_signals(self, prod_sites: np.ndarray,
990 |                   phago_field: Optional[Tuple[np.ndarray, np.ndarray]]):
991 |     """
992 |     케모카인 / 사이토카인 / 인터페론 농도장 갱신.
993 |     - 케모카인: 감염세포 + 활성 식세포가 생성 (사양서 ⑭⑲)
994 |     - 사이토카인: 감염세포가 생성, 면역세포 활성화 조절 (사양서 ㉓)

```

```

995 | - 인터페론: 감염세포가 생성, 항바이러스 상태 유도 (사양서 ㉔)
996 | """
997 | p = self.p
998 | if prod_sites.size:
999 |     fr, fc = self.field_of_site(prod_sites)
1000 |     np.add.at(self.chemokine, (fr, fc), p.chemo_from_infected)
1001 |     np.add.at(self.cytokine, (fr, fc), p.cyto_from_infected)
1002 |     np.add.at(self.ifn, (fr, fc), p.ifn_production)
1003 | if phago_field is not None and phago_field[0].size:
1004 |     np.add.at(self.chemokine, phago_field, p.chemo_from_phagocyte)
1005 |
1006 | self.chemokine = self._diffuse(self.chemokine, p.chemo_sigma_field) \
1007 |     * (1.0 - p.chemo_decay)
1008 | self.cytokine = self._diffuse(self.cytokine, p.cyto_sigma_field) \
1009 |     * (1.0 - p.cyto_decay)
1010 | self.ifn = self._diffuse(self.ifn, p.ifn_sigma_field) * (1.0 - p.ifn_decay_p)
1011 |
1012 | drive = np.clip(self.ifn * 6.0, 0.0, 1.0)
1013 | self.antiviral += (p.antiviral_rate * drive * (1.0 - self.antiviral)
1014 |     - p.antiviral_decay * self.antiviral)
1015 | np.clip(self.antiviral, 0.0, 1.0, out=self.antiviral)
1016 |
1017 | def build_chemo_stack(self) -> np.ndarray:
1018 |     """8방향 shift 스택 (세포가 '주변 8방향'을 감지하기 위한 자료)"""
1019 |     st = np.empty((8, Scale.FIELD_N, Scale.FIELD_N), dtype=np.float32)
1020 |     for i, (dc, dr) in enumerate(DIR8_OFF):
1021 |         st[i] = np.roll(np.roll(self.chemokine, -dr, axis=0), -dc, axis=1)
1022 |     return st
1023 |
1024 | # -----
1025 | def update_humoral(self, n_virus: int, n_infected: int):
1026 |     p = self.p
1027 |     drive = min(1.0, n_virus / 30000.0 + n_infected / 60000.0)
1028 |     self.complement += p.complement_on * drive * (1.0 - self.complement)
1029 |     self.complement -= p.complement_off * self.complement
1030 |     self.complement = float(np.clip(self.complement, 0.0, 1.0))
1031 |
1032 |     target = min(1.0, n_virus * p.inflam_gain_virus
1033 |         + n_infected * p.inflam_gain_infected)
1034 |     self.inflammation += 0.05 * (target - self.inflammation)
1035 |     self.inflammation -= p.inflam_decay * self.inflammation
1036 |     self.inflammation = float(np.clip(self.inflammation, 0.0, 1.0))
1037 |     self.tissue_damage += p.damage_rate * self.inflammation
1038 |
1039 | def regenerate(self):
1040 |     if self.n_dead_epi <= self.n_regen:
1041 |         return
1042 |     dead_idx = np.flatnonzero(self.state == DEAD)
1043 |     if dead_idx.size == 0:
1044 |         return
1045 |     rate = self.p.epi_regen_p * max(0.0, 1.0 - self.inflammation)
1046 |     if rate <= 0:
1047 |         return

```

```

1048         k = self.rng.binomial(dead_idx.size, rate)
1049         if k <= 0:
1050             return
1051         pick = self.rng.choice(dead_idx, size=min(k, dead_idx.size), replace=False)
1052         self.state[pick] = HEALTHY
1053         self.n_regen += pick.size
1054
1055     def counts(self) -> Dict[str, int]:
1056         st = self.state
1057         return {"healthy": int(np.count_nonzero(st == HEALTHY)),
1058                 "eclipse": int(np.count_nonzero(st == ECLIPSE)),
1059                 "productive": int(np.count_nonzero(st == PRODUCTIVE)),
1060                 "dead": int(np.count_nonzero(st == DEAD))}
1061
1062
1063 # =====
1064 # SECTION 7.  VIRUS POOL  (혈류/확산 이동, 목표탐색 없음 - 사양서 ㉠)
1065 # =====
1066
1067
1068 class VirusPool:
1069
1070     def __init__(self, p: Params, rng: np.random.Generator):
1071         self.p = p
1072         self.rng = rng
1073         self.x = np.empty(0, np.float32)
1074         self.y = np.empty(0, np.float32)
1075         self.thin_weight = 1.0
1076         self.thinning_events = 0
1077
1078     @property
1079     def n(self) -> int:
1080         return self.x.size
1081
1082     def seed_uniform(self, k: int):
1083         self.x = self.rng.uniform(0, Scale.GRID - 1, k).astype(np.float32)
1084         self.y = self.rng.uniform(0, Scale.GRID - 1, k).astype(np.float32)
1085
1086     def diffuse(self):
1087         """물리적 확산 + 점액섬모 수송. 등방성이며 목적지를 찾지 않는다. """
1088         n = self.n
1089         if n == 0:
1090             return
1091         s = self.p.virus_sigma
1092         self.x += self.rng.normal(0.0, s, n).astype(np.float32)
1093         self.y += self.rng.normal(0.0, s, n).astype(np.float32)
1094         SystematicMovementEngine._bound(self.x, self.y)
1095
1096     def add(self, x, y):
1097         self.x = np.concatenate([self.x, x.astype(np.float32)])
1098         self.y = np.concatenate([self.y, y.astype(np.float32)])
1099         if self.x.size > self.p.max_virions:
1100             keep = self.rng.random(self.x.size) < 0.5

```

```

1101         self.x = self.x[keep]
1102         self.y = self.y[keep]
1103         self.thin_weight *= 2.0
1104         self.thinning_events += 1
1105
1106     def remove(self, mask) -> int:
1107         k = int(np.count_nonzero(mask))
1108         if k:
1109             self.x = self.x[~mask]
1110             self.y = self.y[~mask]
1111         return k
1112
1113     def remove_idx(self, idx) -> int:
1114         if idx.size == 0:
1115             return 0
1116         m = np.zeros(self.x.size, bool)
1117         m[idx] = True
1118         return self.remove(m)
1119
1120
1121 # =====
1122 # SECTION 8. IMMUNE CELL POOL
1123 # =====
1124
1125 NEUT, MONO, NK, TH, TC, BC = 0, 1, 2, 3, 4, 5
1126 CELL_NAME = {NEUT: "중성구", MONO: "단핵구/대식세포", NK: "NK 세포",
1127              TH: "조력 T 세포", TC: "살해 T 세포", BC: "B 세포"}
1128
1129
1130 class ImmuneCellPool:
1131
1132     FIELDS = ("x", "y", "hx", "hy", "ctype", "age", "life", "speed",
1133              "capacity", "specific", "in_vessel", "activated", "alive")
1134
1135     def __init__(self, cap: int = 16384):
1136         self.cap = cap
1137         self.n = 0
1138         self.x = np.zeros(cap, np.float32)
1139         self.y = np.zeros(cap, np.float32)
1140         self.hx = np.zeros(cap, np.float32) # 극성(직전 이동방향)
1141         self.hy = np.zeros(cap, np.float32)
1142         self.ctype = np.zeros(cap, np.uint8)
1143         self.age = np.zeros(cap, np.int32)
1144         self.life = np.zeros(cap, np.int32)
1145         self.speed = np.zeros(cap, np.float32)
1146         self.capacity = np.zeros(cap, np.int16)
1147         self.specific = np.zeros(cap, bool)
1148         self.in_vessel = np.zeros(cap, bool)
1149         self.activated = np.zeros(cap, bool)
1150         self.alive = np.zeros(cap, bool)
1151
1152     def _grow(self, need: int):
1153         while self.n + need > self.cap:

```

```

1154 |         self.cap *= 2
1155 |     for name in self.FIELDS:
1156 |         arr = getattr(self, name)
1157 |         new = np.zeros(self.cap, arr.dtype)
1158 |         new[:arr.size] = arr
1159 |         setattr(self, name, new)
1160 |
1161 | def add(self, k, ctype, speed, life, capacity=0, specific=False,
1162 |        x=None, y=None, in_vessel=True, age=None):
1163 |     if k <= 0:
1164 |         return
1165 |     if self.n + k > self.cap:
1166 |         self._grow(k)
1167 |     s = slice(self.n, self.n + k)
1168 |     self.x[s] = x
1169 |     self.y[s] = y
1170 |     self.hx[s] = 0.0
1171 |     self.hy[s] = 0.0
1172 |     self.ctype[s] = ctype
1173 |     self.life[s] = life
1174 |     self.age[s] = 0 if age is None else age
1175 |     self.speed[s] = speed
1176 |     self.capacity[s] = capacity
1177 |     self.specific[s] = specific
1178 |     self.in_vessel[s] = in_vessel
1179 |     self.activated[s] = False
1180 |     self.alive[s] = True
1181 |     self.n += k
1182 |
1183 | def mask(self, ctype) -> np.ndarray:
1184 |     return self.alive[:self.n] & (self.ctype[:self.n] == ctype)
1185 |
1186 | def count(self, ctype) -> int:
1187 |     return int(np.count_nonzero(self.mask(ctype)))
1188 |
1189 | def count_specific(self, ctype) -> int:
1190 |     return int(np.count_nonzero(self.mask(ctype) & self.specific[:self.n]))
1191 |
1192 | def compact(self):
1193 |     keep = np.flatnonzero(self.alive[:self.n])
1194 |     k = keep.size
1195 |     for name in self.FIELDS:
1196 |         arr = getattr(self, name)
1197 |         arr[:k] = arr[keep]
1198 |         arr[k:self.n] = 0
1199 |     self.alive[:k] = True
1200 |     self.n = k
1201 |
1202 |
1203 | # =====
1204 | # SECTION 9.  LYMPH NODE  (사양서 ㉔~㉕)
1205 | # =====
1206 |

```

```

1207 |
1208 | class LymphNode:
1209 |
1210 |     def __init__(self, p: Params, rng: np.random.Generator):
1211 |         self.p = p
1212 |         self.rng = rng
1213 |         self.dc_queue = np.zeros(p.dc_delay_ticks + 1, np.float64)
1214 |         self.qi = 0
1215 |         self.antigen = 0.0
1216 |         self.primed = False
1217 |         self.prime_clock = 0
1218 |         self.activated = False
1219 |         self.contracting = False
1220 |
1221 |         self.n_cd4 = p.precursor_cd4
1222 |         self.n_cd8 = p.precursor_cd8
1223 |         self.n_b = p.precursor_b
1224 |         self.n_pc_short = 0.0
1225 |         self.n_pc_gc = 0.0
1226 |         self.memory_t = 0.0
1227 |         self.memory_b = 0.0
1228 |         self.peak_cd4 = p.precursor_cd4
1229 |         self.peak_cd8 = p.precursor_cd8
1230 |         self.peak_b = p.precursor_b
1231 |         self.peak_pc = 0.0
1232 |
1233 |         self.t_antigen_arrival: Optional[int] = None
1234 |         self.t_priming_done: Optional[int] = None
1235 |         self.t_cd4_act: Optional[int] = None
1236 |         self.t_cd8_act: Optional[int] = None
1237 |         self.t_b_act: Optional[int] = None
1238 |         self.t_first_emigration: Optional[int] = None
1239 |
1240 |     def deposit_antigen(self, amount: float):
1241 |         if amount <= 0:
1242 |             return
1243 |         j = (self.qi + self.p.dc_delay_ticks) % self.dc_queue.size
1244 |         self.dc_queue[j] += amount
1245 |
1246 |     def update(self, tick: int):
1247 |         p = self.p
1248 |         arrive = self.dc_queue[self.qi]
1249 |         self.dc_queue[self.qi] = 0.0
1250 |         self.qi = (self.qi + 1) % self.dc_queue.size
1251 |         if arrive > 0 and self.t_antigen_arrival is None:
1252 |             self.t_antigen_arrival = tick
1253 |         self.antigen += arrive
1254 |         self.antigen *= (1.0 - p.antigen_decay)
1255 |
1256 |         if not self.primed and self.antigen >= p.antigen_threshold:
1257 |             self.primed = True
1258 |             self.prime_clock = 0
1259 |         if self.primed and not self.activated:

```



```

1260 |         self.prime_clock += 1
1261 |         if self.prime_clock >= p.priming_ticks:
1262 |             self.activated = True
1263 |             self.t_priming_done = tick
1264 |             self.t_cd4_act = tick
1265 |             self.t_cd8_act = tick
1266 |             self.t_b_act = tick
1267 |         if not self.activated:
1268 |             return
1269 |
1270 |         elapsed = tick - (self.t_priming_done or tick)
1271 |         expansion_on = elapsed < p.program_ticks
1272 |         gr = p.net_expansion
1273 |         help_f = self.n_cd4 / (self.n_cd4 + 5.0 * p.precursor_cd4)
1274 |
1275 |         if expansion_on:
1276 |             if self.n_cd4 < p.precursor_cd4 * p.cd4_max_expansion:
1277 |                 self.n_cd4 *= math.exp(p.cd4_growth * gr)
1278 |             if self.n_cd8 < p.precursor_cd8 * p.cd8_max_expansion:
1279 |                 self.n_cd8 *= math.exp(p.cd8_growth * gr * (0.55 + 0.9 * help_f))
1280 |             if self.n_b < p.precursor_b * p.b_max_expansion:
1281 |                 self.n_b *= math.exp(p.b_growth * gr * (0.45 + 1.0 * help_f))
1282 |         else:
1283 |             self.contracting = True
1284 |             self.n_cd8 = max(self.n_cd8 * (1 - p.contraction_p),
1285 |                             self.peak_cd8 * p.memory_fraction)
1286 |             self.n_cd4 = max(self.n_cd4 * (1 - p.contraction_p),
1287 |                             self.peak_cd4 * p.memory_fraction)
1288 |             self.n_b = max(self.n_b * (1 - p.b_contraction_p),
1289 |                             self.peak_b * p.memory_b_fraction)
1290 |             self.memory_t = min(self.n_cd8 + self.n_cd4,
1291 |                                 (self.peak_cd8 + self.peak_cd4) * p.memory_fraction)
1292 |             self.memory_b = min(self.n_b, self.peak_b * p.memory_b_fraction)
1293 |
1294 |         if expansion_on and elapsed >= p.ef_delay_ticks:
1295 |             self.n_pc_short += p.ef_diff_rate * self.n_b
1296 |             self.n_pc_short *= (1.0 - p.pc_short_decay)
1297 |         if p.gc_delay_ticks <= elapsed < p.gc_end_ticks:
1298 |             self.n_pc_gc += p.gc_diff_rate * self.n_b * min(1.0, help_f * 2.0)
1299 |             self.n_pc_gc *= (1.0 - p.pc_gc_decay)
1300 |
1301 |         self.peak_cd8 = max(self.peak_cd8, self.n_cd8)
1302 |         self.peak_cd4 = max(self.peak_cd4, self.n_cd4)
1303 |         self.peak_b = max(self.peak_b, self.n_b)
1304 |         self.peak_pc = max(self.peak_pc, self.n_pc_short + self.n_pc_gc)
1305 |
1306 |     def emigrate(self, tick: int) -> Tuple[int, int]:
1307 |         p = self.p
1308 |         if not self.activated or tick * Scale.DT_HOUR < p.emigration_start_h:
1309 |             return 0, 0
1310 |         n_ctl = int(self.rng.poisson(p.emigration_p * self.n_cd8))
1311 |         n_th = int(self.rng.poisson(p.emigration_p * 0.30 * self.n_cd4))
1312 |         if (n_ctl or n_th) and self.t_first_emigration is None:

```

```

1313         self.t_first_emigration = tick
1314     return n_ctl, n_th
1315
1316
1317 class Antibody:
1318
1319     def __init__(self, p: Params):
1320         self.p = p
1321         self.igm = 0.0
1322         self.igg = 0.0
1323         self.t_igm: Optional[int] = None
1324         self.t_igg: Optional[int] = None
1325         self.peak_total = 0.0
1326         self.neutralized = 0
1327
1328     @property
1329     def total(self) -> float:
1330         return self.igm + self.igg
1331
1332     def update(self, ln: LymphNode, tick: int):
1333         p = self.p
1334         self.igm += p.ab_per_pc_igm * ln.n_pc_short
1335         self.igg += p.ab_per_pc_igg * ln.n_pc_gc
1336         self.igm *= (1.0 - p.igm_decay_p)
1337         self.igg *= (1.0 - p.igg_decay_p)
1338         if self.t_igm is None and self.igm >= p.ab_detect_threshold:
1339             self.t_igm = tick
1340         if self.t_igg is None and self.igg >= p.ab_detect_threshold:
1341             self.t_igg = tick
1342         self.peak_total = max(self.peak_total, self.total)
1343
1344     def neutralization_rate(self) -> float:
1345         c = self.total
1346         return self.p.ab_neutralize_max * c / (c + self.p.ab_kd)
1347
1348     def opsonin_bonus(self) -> float:
1349         c = self.total
1350         return self.p.ab_opsonin * c / (c + self.p.ab_kd)
1351
1352     def infection_block(self) -> float:
1353         c = self.total
1354         return self.p.ab_block_max * c / (c + self.p.ab_block_kd)
1355
1356
1357 # =====
1358 # SECTION 10. SIMULATION CONTROLLER
1359 # =====
1360
1361
1362 class SystematicImmuneSimulation:
1363
1364     def __init__(self, p: Optional[Params] = None, seed: Optional[int] = None,
1365                 verbose: bool = True):

```

```

1366 |     self.p = p or Params()
1367 |     if seed is not None:
1368 |         self.p.seed = seed
1369 |     self.rng = np.random.default_rng(self.p.seed)
1370 |     self.movement_mode = "systematic"
1371 |     self.mover = SystematicMovementEngine()
1372 |     self.verbose = verbose
1373 |
1374 |     self.env = Environment(self.p, self.rng)
1375 |     self.virus = VirusPool(self.p, self.rng)
1376 |     self.cells = ImmuneCellPool()
1377 |     self.ln = LymphNode(self.p, self.rng)
1378 |     self.ab = Antibody(self.p)
1379 |
1380 |     self.tick = 0
1381 |     self.history: List[dict] = []
1382 |     self.daily: List[dict] = []
1383 |
1384 |     self.cleared_virus = 0
1385 |     self.cleared_by_phago = 0
1386 |     self.cleared_by_decay = 0
1387 |     self.cleared_by_complement = 0
1388 |     self.cleared_by_antibody = 0
1389 |     self.killed_infected = 0
1390 |     self.killed_by_nk = 0
1391 |     self.killed_by_ctl = 0
1392 |     self.apoptosis_infected = 0
1393 |     self.dead_immune = 0
1394 |     self.total_infections = 0
1395 |     self.ctl_contact = 0
1396 |     self.ctl_agent_ticks = 0
1397 |     self.n_extravasated = 0
1398 |
1399 |     self.t_first_arrival: Optional[int] = None # 사양서 40-㉔
1400 |     self.t_first_extravasation: Optional[int] = None
1401 |
1402 |     self._init_population()
1403 |
1404 | # -----
1405 | def _vessel_positions(self, k: int) -> Tuple[np.ndarray, np.ndarray]:
1406 |     """혈관 칸 위에 균등 배치 (초기 배치는 이동이 아니라 초기조건)"""
1407 |     idx = self.rng.choice(self.env.vessel_sites, size=k, replace=True)
1408 |     row = idx // Scale.GRID
1409 |     col = idx - row * Scale.GRID
1410 |     return (col + 0.5).astype(np.float32), (row + 0.5).astype(np.float32)
1411 |
1412 | def _ln_exit_positions(self, k: int) -> Tuple[np.ndarray, np.ndarray]:
1413 |     idx = self.rng.choice(self.env.ln_exit_sites, size=k, replace=True)
1414 |     row = idx // Scale.GRID
1415 |     col = idx - row * Scale.GRID
1416 |     return (col + 0.5).astype(np.float32), (row + 0.5).astype(np.float32)
1417 |
1418 | def _init_population(self):

```

```

1419 |         p = self.p
1420 |         self.virus.seed_uniform(p.n_virus0)
1421 |
1422 |     def put(k, ctype, speed, life, cap=0):
1423 |         x, y = self._vessel_positions(k)
1424 |         age = self.rng.integers(0, max(1, life), k)
1425 |         self.cells.add(k, ctype, speed, life, cap, x=x, y=y,
1426 |                        in_vessel=True, age=age)
1427 |
1428 |     # 사양서 ㉔: 면역세포는 혈관에서 출발한다
1429 |     put(p.n_neutrophil, NEUT, p.speed_neut, p.life_neut, p.neut_capacity)
1430 |     put(p.n_monocyte, MONO, p.speed_mono, p.life_mono, p.mono_capacity)
1431 |     put(p.n_nk, NK, p.speed_nk, p.life_nk)
1432 |     put(p.n_helper_t, TH, p.speed_naive_t, p.life_naive)
1433 |     put(p.n_killer_t, TC, p.speed_naive_t, p.life_naive)
1434 |     put(p.n_bcell, BC, p.speed_b, p.life_naive)
1435 |
1436 |     # -----
1437 |     def _match_same_site(self, p_site: np.ndarray, v_site: np.ndarray):
1438 |         if p_site.size == 0 or v_site.size == 0:
1439 |             return np.empty(0, np.int64), np.empty(0, np.int64)
1440 |         pu = np.unique(p_site)
1441 |         cand = np.flatnonzero(np.isin(v_site, pu, kind="table"))
1442 |         if cand.size == 0:
1443 |             return np.empty(0, np.int64), np.empty(0, np.int64)
1444 |         cs = v_site[cand]
1445 |         order = np.argsort(cs, kind="stable")
1446 |         cs_s, cand_s = cs[order], cand[order]
1447 |         uniq, start, counts = np.unique(cs_s, return_index=True, return_counts=True)
1448 |         pos = np.minimum(np.searchsorted(uniq, p_site), uniq.size - 1)
1449 |         has = uniq[pos] == p_site
1450 |         idx_p = np.flatnonzero(has)
1451 |         if idx_p.size == 0:
1452 |             return np.empty(0, np.int64), np.empty(0, np.int64)
1453 |         g = pos[idx_p]
1454 |         off = self.rng.integers(0, counts[g])
1455 |         v_idx = cand_s[start[g] + off]
1456 |         _, keep = np.unique(v_idx, return_index=True)
1457 |         return idx_p[keep], v_idx[keep]
1458 |
1459 |     # -----
1460 |     def step(self):
1461 |         p, env, c = self.p, self.env, self.cells
1462 |         self.tick += 1
1463 |         t = self.tick
1464 |
1465 |         # ----- A. 감염세포 진행 -----
1466 |         prod_sites, n_apop = env.advance_infected()
1467 |         self.apoptosis_infected += n_apop
1468 |
1469 |         # ----- B. 바이러스 생산 -----
1470 |         if prod_sites.size:
1471 |             fr, fc = Environment.field_of_site(prod_sites)

```

```

1472 |         av = env.antiviral[fr, fc]
1473 |         rate = (p.burst_agents_per_site / p.productive_ticks) * \
1474 |             (1.0 - p.ifn_block_release * av) / max(1.0, self.virus.thin_weight)
1475 |         k = self.rng.poisson(np.maximum(rate, 0.0))
1476 |         tot = int(k.sum())
1477 |         if tot:
1478 |             src = np.repeat(prod_sites, k)
1479 |             row = src // Scale.GRID
1480 |             col = src - row * Scale.GRID
1481 |             self.virus.add((col + self.rng.random(tot)).astype(np.float32),
1482 |                           (row + self.rng.random(tot)).astype(np.float32))
1483 |
1484 | # ----- C. 바이러스 이동 (확산/수송) -----
1485 | self.virus.diffuse()
1486 |
1487 | # ----- D. 감염 -----
1488 | if self.virus.n:
1489 |     v_site = Environment.site_of(self.virus.x, self.virus.y)
1490 |     sus = np.flatnonzero(env.state[v_site] == HEALTHY)
1491 |     if sus.size:
1492 |         s_sites = v_site[sus]
1493 |         fr, fc = Environment.field_of_site(s_sites)
1494 |         pe = p.p_infect * (1.0 - env.antiviral[fr, fc]) * \
1495 |             (1.0 - self.ab.infection_block())
1496 |         if self.virus.thin_weight > 1.0:
1497 |             pe = 1.0 - (1.0 - pe) ** self.virus.thin_weight
1498 |         ok = self.rng.random(sus.size) < pe
1499 |         if ok.any():
1500 |             n_new = env.infect_sites(np.unique(s_sites[ok]))
1501 |             self.total_infections += n_new
1502 |             self.virus.remove_idx(sus[ok])
1503 |
1504 | # ----- E. 면역세포 노화/사멸 -----
1505 | n = c.n
1506 | if n:
1507 |     alive = c.alive[:n]
1508 |     c.age[:n][alive] += 1
1509 |     died = alive & (c.age[:n] >= c.life[:n])
1510 |     eff = alive & c.specific[:n] & ((c.ctype[:n] == TC) | (c.ctype[:n] == TH))
1511 |     if eff.any():
1512 |         pd_eff = (p.eff_t_death_contract if self.ln.contracting
1513 |                   else p.eff_t_death_expand)
1514 |         died |= eff & (self.rng.random(n) < pd_eff)
1515 |     k = int(np.count_nonzero(died))
1516 |     if k:
1517 |         c.alive[:n][died] = False
1518 |         self.dead_immune += k
1519 |     if n > 5000 and c.alive[:n].mean() < 0.85:
1520 |         c.compact()
1521 |
1522 | # ----- F. 체계적 이동 (★ 핵심) -----
1523 | chemo_stack = env.build_chemo_stack()
1524 | n = c.n

```

```

1525 | idx = np.flatnonzero(c.alive[:n])
1526 | if idx.size:
1527 |     fr, fc = Environment.field_of_xy(c.x[idx], c.y[idx])
1528 |     here = env.chemokine[fr, fc]
1529 |
1530 |     in_v = c.in_vessel[idx]
1531 |     # -- 혈관의 유출 판정 (결정론적 역치, 사양서 ㉔) --
1532 |     ex = in_v & (here > p.extravasation_threshold)
1533 |     if ex.any():
1534 |         gi = idx[ex]
1535 |         c.in_vessel[gi] = False
1536 |         self.n_extravasated += gi.size
1537 |         if self.t_first_extravasation is None:
1538 |             self.t_first_extravasation = t
1539 |     # -- 조직 내 세포가 신호를 완전히 잃으면 혈관으로 복귀 --
1540 |     back = (~in_v) & (here < p.reentry_threshold) & (c.ctype[idx] == NEUT)
1541 |     in_v = c.in_vessel[idx]
1542 |
1543 |     vsel = np.flatnonzero(in_v)
1544 |     tsel = np.flatnonzero(~in_v)
1545 |
1546 |     if vsel.size:
1547 |         gi = idx[vsel]
1548 |         xs, ys = c.x[gi].copy(), c.y[gi].copy()
1549 |         hd = np.stack([c.hx[gi], c.hy[gi]], axis=1)
1550 |         site = Environment.site_of(xs, ys)
1551 |         fdx = env.flow_dx[site].copy()
1552 |         fdy = env.flow_dy[site].copy()
1553 |         # 혈관 밖으로 벗어난 경우 가장 가까운 혈관 행으로 스넵
1554 |         nov = (np.abs(fdx) + np.abs(fdy)) < 1e-6
1555 |         if nov.any():
1556 |             rr = ys[nov].astype(np.int32)
1557 |             base = (rr // Scale.EPI_PERIOD) * Scale.EPI_PERIOD + 13
1558 |             ys[nov] = np.clip(base, 0, Scale.GRID - 1) + 0.5
1559 |             site2 = Environment.site_of(xs[nov], ys[nov])
1560 |             fdx[nov] = env.flow_dx[site2]
1561 |             fdy[nov] = env.flow_dy[site2]
1562 |             fdx[nov] = np.where(np.abs(fdx[nov]) + np.abs(fdy[nov]) < 1e-6,
1563 |                               1.0, fdx[nov])
1564 |             vfr, vfc = Environment.field_of_xy(xs, ys)
1565 |             self.mover.move_vessel(xs, ys, hd, fdx, fdy, p.vessel_speed,
1566 |                                   chemo_stack, vfr, vfc)
1567 |             c.x[gi], c.y[gi] = xs, ys
1568 |             c.hx[gi], c.hy[gi] = hd[:, 0], hd[:, 1]
1569 |
1570 |     if tsel.size:
1571 |         gi = idx[tsel]
1572 |         xs, ys = c.x[gi].copy(), c.y[gi].copy()
1573 |         hd = np.stack([c.hx[gi], c.hy[gi]], axis=1)
1574 |         tfr, tfc = Environment.field_of_xy(xs, ys)
1575 |         self.mover.move_tissue(xs, ys, hd, c.speed[gi],
1576 |                               chemo_stack, tfr, tfc)
1577 |         c.x[gi], c.y[gi] = xs, ys

```

```

1578         c.hx[gi], c.hy[gi] = hd[:, 0], hd[:, 1]
1579
1580     # ----- G. 활성화 상태 (사이토카인, 사양서 ㉓) -----
1581     n = c.n
1582     idx = np.flatnonzero(c.alive[:n])
1583     if idx.size:
1584         fr, fc = Environment.field_of_xy(c.x[idx], c.y[idx])
1585         cyto = env.cytokine[fr, fc]
1586         c.activated[idx] = cyto > p.cyto_activation_threshold
1587         cell_site = Environment.site_of(c.x[idx], c.y[idx])
1588     else:
1589         cell_site = np.empty(0, np.int32)
1590     ct = c.ctype[idx] if idx.size else np.empty(0, np.uint8)
1591
1592     # ----- H. 식균작용 -----
1593     antigen_in = 0.0
1594     phago_field = None
1595     if idx.size and self.virus.n:
1596         v_site = Environment.site_of(self.virus.x, self.virus.y)
1597         opson = min(0.95, self.ab.opsonin_bonus()
1598                     + p.complement_opsonin * env.complement)
1599         act_fr, act_fc = [], []
1600         for ctype, base_p in ((NEUT, p.neut_phago_p), (MONO, p.mono_phago_p)):
1601             sel = np.flatnonzero((ct == ctype) & (c.capacity[idx] > 0)
1602                                   & (~c.in_vessel[idx]))
1603             if sel.size == 0:
1604                 continue
1605             loc, vidx = self._match_same_site(cell_site[sel], v_site)
1606             if loc.size == 0:
1607                 continue
1608             succ = self.rng.random(loc.size) < min(0.98, base_p + opson)
1609             if not succ.any():
1610                 continue
1611             gi = idx[sel[loc[succ]]]
1612             c.capacity[gi] -= 1
1613             spent = gi[c.capacity[gi] <= 0]
1614             if spent.size:
1615                 c.alive[spent] = False
1616                 self.dead_immune += spent.size
1617             k = self.virus.remove_idx(vidx[succ])
1618             self.cleared_by_phago += k
1619             self.cleared_virus += k
1620             # 활성화 식세포는 2 차 케모카인을 방출한다 (사양서 ㉔)
1621             a, b = Environment.field_of_xy(c.x[gi], c.y[gi])
1622             act_fr.append(a); act_fc.append(b)
1623             if ctype == MONO:
1624                 antigen_in += k * 1.0
1625             v_site = Environment.site_of(self.virus.x, self.virus.y)
1626         if act_fr:
1627             phago_field = (np.concatenate(act_fr), np.concatenate(act_fc))
1628
1629     # ----- I. NK 세포 -----
1630     if idx.size:

```

```

1631 |         sel = np.flatnonzero((ct == NK) & (~c.in_vessel[idx]))
1632 |         if sel.size:
1633 |             s = cell_site[sel]
1634 |             stt = env.state[s]
1635 |             tgt = (stt == PRODUCTIVE) | (stt == ECLIPSE)
1636 |             if tgt.any():
1637 |                 if self.t_first_arrival is None:
1638 |                     self.t_first_arrival = t
1639 |                     boost = 1.0 + 0.5 * float(env.inflammation)
1640 |                     ok = self.rng.random(int(tgt.sum())) < min(0.95, p.nk_kill_p * boost)
1641 |                     k = env.kill_sites(s[tgt][ok])
1642 |                     self.killed_by_nk += k
1643 |                     self.killed_infected += k
1644 |
1645 |         # ----- J. 살해 T 세포 -----
1646 |         if idx.size:
1647 |             sel = np.flatnonzero((ct == TC) & c.specific[idx] & (~c.in_vessel[idx]))
1648 |             self.ctl_agent_ticks += int(np.count_nonzero(
1649 |                 (ct == TC) & c.specific[idx]))
1650 |             if sel.size:
1651 |                 s = cell_site[sel]
1652 |                 stt = env.state[s]
1653 |                 tgt = (stt == PRODUCTIVE) | (stt == ECLIPSE)
1654 |                 nt = int(tgt.sum())
1655 |                 self.ctl_contact += nt
1656 |                 if nt:
1657 |                     ok = self.rng.random(nt) < p.cctl_kill_p
1658 |                     k = env.kill_sites(s[tgt][ok])
1659 |                     self.killed_by_ctl += k
1660 |                     self.killed_infected += k
1661 |
1662 |         # 첫 도착 시각: 식세포가 감염칸에 도달한 경우도 포함
1663 |         if self.t_first_arrival is None and idx.size:
1664 |             tis = np.flatnonzero(~c.in_vessel[idx])
1665 |             if tis.size:
1666 |                 stt = env.state[cell_site[tis]]
1667 |                 if np.any((stt == PRODUCTIVE) | (stt == ECLIPSE)):
1668 |                     self.t_first_arrival = t
1669 |
1670 |         # ----- K. 항원 전달 -----
1671 |         n_infected_now = int(env.inf_alive.sum()) if env.inf_alive.size else 0
1672 |         antigen_in += 0.005 * n_infected_now
1673 |         self.ln.deposit_antigen(antigen_in)
1674 |
1675 |         # ----- L. 바이러스 제거 (비특이/보체/항체) -----
1676 |         if self.virus.n:
1677 |             pd_ = p.virus_decay_p
1678 |             pc = p.complement_lysis * env.complement
1679 |             pa = self.ab.neutralization_rate()
1680 |             ptot = 1.0 - (1 - pd_) * (1 - pc) * (1 - pa)
1681 |             rem = self.rng.random(self.virus.n) < ptot
1682 |             k = self.virus.remove(rem)
1683 |             if k:

```



```

1684         tot = pd_ + pc + pa
1685         self.cleared_by_decay += int(k * pd_ / tot)
1686         self.cleared_by_complement += int(k * pc / tot)
1687         self.cleared_by_antibody += int(k * pa / tot)
1688         self.cleared_virus += k
1689
1690     # ----- M. 림프절 / 항체 -----
1691     self.ln.update(t)
1692     n_ctl, n_th = self.ln.emigrate(t)
1693     if n_ctl:
1694         x, y = self._ln_exit_positions(n_ctl)
1695         c.add(n_ctl, TC, p.speed_eff_t, p.life_naive, specific=True,
1696              x=x, y=y, in_vessel=True)
1697     if n_th:
1698         x, y = self._ln_exit_positions(n_th)
1699         c.add(n_th, TH, p.speed_eff_t, p.life_naive, specific=True,
1700              x=x, y=y, in_vessel=True)
1701     self.ab.update(self.ln, t)
1702
1703     # ----- N. 신호장 / 체액 / 동원 / 재생 -----
1704     env.update_signals(prod_sites, phago_field)
1705     env.update_humoral(self.virus.n, n_infected_now)
1706     self._recruit()
1707     env.regenerate()
1708
1709     self._record()
1710
1711     # -----
1712     def _recruit(self):
1713         p = self.p
1714         infl = self.env.inflammation
1715         spec = {NEUT: (p.n_neutrophil, p.recruit_neut_max, p.speed_neut,
1716                       p.life_neut, p.neut_capacity),
1717                MONO: (p.n_monocyte, p.recruit_mono_max, p.speed_mono,
1718                      p.life_mono, p.mono_capacity),
1719                NK: (p.n_nk, p.recruit_nk_max, p.speed_nk, p.life_nk, 0)}
1720         for ctype, (base, mx, sp, lf, cap) in spec.items():
1721             gap = base * (1.0 + mx * infl) - self.cells.count(ctype)
1722             if gap <= 0:
1723                 continue
1724             k = int(self.rng.poisson(gap * p.recruit_rate))
1725             if k:
1726                 x, y = self._vessel_positions(k)
1727                 self.cells.add(k, ctype, sp, lf, cap, x=x, y=y, in_vessel=True)
1728
1729     # -----
1730     RECORD_EVERY = 5
1731
1732     def _snapshot(self) -> dict:
1733         env, c, ln, ab = self.env, self.cells, self.ln, self.ab
1734         cn = env.counts()
1735         infected = cn["eclipse"] + cn["productive"]
1736         n = c.n

```

```

1737 | alive = c.alive[:n]
1738 | in_tissue = int(np.count_nonzero(alive & ~c.in_vessel[:n]))
1739 | return {
1740 |     "tick": self.tick, "hours": self.tick * Scale.DT_HOUR,
1741 |     "day": self.tick / Scale.TICKS_PER_DAY,
1742 |     "virus": self.virus.n, "infected": infected,
1743 |     "eclipse": cn["eclipse"], "productive": cn["productive"],
1744 |     "healthy": cn["healthy"], "dead_epi": cn["dead"],
1745 |     "neutrophil": c.count(NEUT), "monocyte": c.count(MONO),
1746 |     "nk": c.count(NK), "helper_t": c.count(TH),
1747 |     "helper_t_spec": c.count_specific(TH),
1748 |     "killer_t": c.count(TC), "killer_t_spec": c.count_specific(TC),
1749 |     "bcell": c.count(BC),
1750 |     "memory": ln.memory_t + ln.memory_b,
1751 |     "antibody": ab.total, "igm": ab.igm, "igg": ab.igg,
1752 |     "chemokine": float(env.chemokine.mean()),
1753 |     "chemokine_max": float(env.chemokine.max()),
1754 |     "cytokine": float(env.cytokine.mean()),
1755 |     "interferon": float(env.ifn.mean()),
1756 |     "antiviral": float(env.antiviral.mean()),
1757 |     "complement": env.complement,
1758 |     "inflammation": env.inflammation,
1759 |     "damage": env.tissue_damage,
1760 |     "cleared_virus": self.cleared_virus,
1761 |     "killed_infected": self.killed_infected,
1762 |     "apoptosis": self.apoptosis_infected,
1763 |     "dead_immune": self.dead_immune,
1764 |     "in_tissue": in_tissue,
1765 |     "ln_cd8": ln.n_cd8, "ln_pc": ln.n_pc_short + ln.n_pc_gc,
1766 | }
1767 |
1768 | def _record(self):
1769 |     if self.tick % self.RECORD EVERY == 0 or self.tick % Scale.TICKS_PER_DAY == 0:
1770 |         self.history.append(self._snapshot())
1771 |
1772 | # -----
1773 | def run(self):
1774 |     p = self.p
1775 |     t0 = time.time()
1776 |     self.daily.append(self._snapshot())
1777 |     clear_streak = 0
1778 |     max_ticks = p.max_days * Scale.TICKS_PER_DAY
1779 |     while self.tick < max_ticks:
1780 |         self.step()
1781 |         if self.tick % Scale.TICKS_PER_DAY == 0:
1782 |             self.daily.append(self._snapshot())
1783 |             if self.verbose:
1784 |                 d = self.tick // Scale.TICKS_PER_DAY
1785 |                 s = self.daily[-1]
1786 |                 print(f" Day {d:2d}: V={s['virus']:>8}, I={s['infected']:>7}, "
1787 |                     f"H={s['healthy']:>7}, chemo={s['chemokine']:.4f} "
1788 |                     f"tissue-cells={s['in_tissue']:>7},", flush=True)
1789 |                 n_inf = int(self.env.inf_alive.sum()) if self.env.inf_alive.size else 0

```

```

1790 |         if self.virus.n == 0 and n_inf == 0:
1791 |             clear_streak += 1
1792 |         else:
1793 |             clear_streak = 0
1794 |         if clear_streak >= p.stop_after_clear_days * Scale.TICKS_PER_DAY:
1795 |             break
1796 |         if self.daily[-1]["tick"] != self.tick:
1797 |             self.daily.append(self._snapshot())
1798 |         self.wall_time = time.time() - t0
1799 |
1800 |
1801 | # =====
1802 | # SECTION 11.  DAILY TABLE  (사양서 ㉔ - 하루 경과마다 필수 출력)
1803 | # =====
1804 |
1805 | DAILY_ROWS = [
1806 |     ("병원체 수",          lambda s: f"{s['virus']:,}"),
1807 |     ("감염세포 수",        lambda s: f"{s['infected']:,}"),
1808 |     ("정상세포 수",        lambda s: f"{s['healthy']:,}"),
1809 |     ("중성구 수",          lambda s: f"{s['neutrophil']:,}"),
1810 |     ("단핵구/대식세포 수", lambda s: f"{s['monocyte']:,}"),
1811 |     ("NK 세포 수",          lambda s: f"{s['nk']:,}"),
1812 |     ("조력 T 세포 수",      lambda s: f"{s['helper_t']:,}"),
1813 |     ("살해 T 세포 수",      lambda s: f"{s['killer_t']:,}"),
1814 |     ("B 세포 수",           lambda s: f"{s['bcell']:,}"),
1815 |     ("기억세포 수",         lambda s: f"{s['memory']:,0f}"),
1816 |     ("항체 농도(ug/mL)",    lambda s: f"{s['antibody']:.3f}"),
1817 |     ("케모카인 농도",       lambda s: f"{s['chemokine']:.4f}"),
1818 |     ("사이토카인 농도",     lambda s: f"{s['cytokine']:.4f}"),
1819 |     ("인터페론 농도",       lambda s: f"{s['interferon']:.4f}"),
1820 |     ("보체 활성도(%)",      lambda s: f"{s['complement']*100:.2f}"),
1821 |     ("염증 정도(%)",        lambda s: f"{s['inflammation']*100:.2f}"),
1822 |     ("제거된 병원체 수",    lambda s: f"{s['cleared_virus']:,}"),
1823 |     ("제거된 감염세포 수",   lambda s: f"{s['killed_infected']:,}"),
1824 |     ("면역세포 사망/소모량", lambda s: f"{s['dead_immune']:,}"),
1825 | ]
1826 |
1827 |
1828 | def print_daily_table(daily: List[dict], title: str, width: int = 12):
1829 |     print()
1830 |     print("#" * 110)
1831 |     print(f" {title}")
1832 |     print("#" * 110)
1833 |     labels = [f"Day {int(round(s['day']))}" if abs(s['day'] - round(s['day'])) < 0.01
1834 |               else f"D{s['day']:.2f}" for s in daily]
1835 |     labels[-1] = labels[-1] + "*" if abs(daily[-1]['day']
1836 |                                         - round(daily[-1]['day'])) > 0.01 else labels[-1]
1837 |     hdr = f"{'항목':<22}" + "".join(f"{l:>{width}}" for l in labels)
1838 |     print(hdr)
1839 |     print("-" * len(hdr))
1840 |     for name, fn in DAILY_ROWS:
1841 |         print(f"{name:<22}" + "".join(f"{fn(s):>{width}}" for s in daily))
1842 |     print("-" * len(hdr))

```

```

1843 |     print(" * 마지막 열 = 병원체 완전 소멸 후 최종 시점 (일 단위가 아닐 수 있음)")
1844 |     print()
1845 |
1846 |
1847 | # =====
1848 | # SECTION 12. ANALYSIS & VALIDATION
1849 | # =====
1850 |
1851 |
1852 | def _ser(h, k):
1853 |     return np.array([x[k] for x in h], float)
1854 |
1855 |
1856 |
1857 |
1858 | def _first_reach(h, key, frac=0.95):
1859 |     """정점의 frac 배에 처음 도달한 시각. 포화 평탄구간에서 argmax 가
1860 |     임의의 낮은 시점을 고르는 문제를 피하기 위한 유입 시점 지표."""
1861 |     v = _ser(h, key)
1862 |     if v.size == 0 or v.max() <= 0:
1863 |         return float("nan")
1864 |     thr = v.max() * frac
1865 |     for j in range(v.size):
1866 |         if v[j] >= thr:
1867 |             return float(h[j]["day"])
1868 |     return float("nan")
1869 |
1870 |
1871 | def _peak(h, k):
1872 |     v = _ser(h, k)
1873 |     if v.size == 0:
1874 |         return 0.0, float("nan")
1875 |     i = int(np.argmax(v))
1876 |     return float(v[i]), float(h[i]["day"])
1877 |
1878 |
1879 | def _below(h, k, frac):
1880 |     v = _ser(h, k)
1881 |     if v.size == 0 or v.max() <= 0:
1882 |         return None
1883 |     i = int(np.argmax(v))
1884 |     thr = v[i] * frac
1885 |     for j in range(i, v.size):
1886 |         if v[j] <= thr:
1887 |             return float(h[j]["day"])
1888 |     return None
1889 |
1890 |
1891 | def _zero(h, k):
1892 |     v = _ser(h, k)
1893 |     if v.size == 0:
1894 |         return None
1895 |     i = int(np.argmax(v))

```

```

1896 |     for j in range(i, v.size):
1897 |         if v[j] <= 0:
1898 |             return float(h[j]["day"])
1899 |     return None
1900 |
1901 |
1902 | def analyze(sim: SystematicImmuneSimulation) -> dict:
1903 |     h = sim.history
1904 |     tpd = Scale.TICKS_PER_DAY
1905 |     R = {}
1906 |     R["virus_peak"], R["virus_peak_day"] = _peak(h, "virus")
1907 |     R["virus_final"] = h[-1]["virus"]
1908 |     R["virus_t50"] = _below(h, "virus", 0.5)
1909 |     R["virus_t90"] = _below(h, "virus", 0.1)
1910 |     R["virus_shed_end_lpcr"] = _below(h, "virus", 0.01)
1911 |     R["virus_clear_day"] = _zero(h, "virus")
1912 |     R["inf_peak"], R["inf_peak_day"] = _peak(h, "infected")
1913 |     R["inf_t50"] = _below(h, "infected", 0.5)
1914 |     R["inf_clear_day"] = _zero(h, "infected")
1915 |     R["neut_peak"], _ = _peak(h, "neutrophil")
1916 |     R["neut_peak_day"] = _first_reach(h, "neutrophil")
1917 |     R["mono_peak"], _ = _peak(h, "monocyte")
1918 |     R["mono_peak_day"] = _first_reach(h, "monocyte")
1919 |     R["nk_peak"], _ = _peak(h, "nk")
1920 |     R["nk_peak_day"] = _first_reach(h, "nk")
1921 |     R["chemo_peak"], R["chemo_peak_day"] = _peak(h, "chemokine")
1922 |     R["cyto_peak"], R["cyto_peak_day"] = _peak(h, "cytokine")
1923 |     R["ifn_peak"], R["ifn_peak_day"] = _peak(h, "interferon")
1924 |     R["comp_peak"], R["comp_peak_day"] = _peak(h, "complement")
1925 |     R["inflam_peak"], R["inflam_peak_day"] = _peak(h, "inflammation")
1926 |     R["damage_final"] = h[-1]["damage"]
1927 |     R["ctl_peak"], R["ctl_peak_day"] = _peak(h, "killer_t_spec")
1928 |     R["th_peak"], R["th_peak_day"] = _peak(h, "helper_t_spec")
1929 |     R["pc_peak"], R["pc_peak_day"] = _peak(h, "ln_pc")
1930 |     R["ab_peak"], R["ab_peak_day"] = _peak(h, "antibody")
1931 |     ln = sim.ln
1932 |     R["t_antigen"] = ln.t_antigen_arrival / tpd if ln.t_antigen_arrival else None
1933 |     R["t_prime"] = ln.t_priming_done / tpd if ln.t_priming_done else None
1934 |     R["t_cd4"] = ln.t_cd4_act / tpd if ln.t_cd4_act else None
1935 |     R["t_cd8"] = ln.t_cd8_act / tpd if ln.t_cd8_act else None
1936 |     R["t_b"] = ln.t_b_act / tpd if ln.t_b_act else None
1937 |     R["t_emig"] = ln.t_first_emigration / tpd if ln.t_first_emigration else None
1938 |     R["t_igm"] = sim.ab.t_igm / tpd if sim.ab.t_igm else None
1939 |     R["t_igg"] = sim.ab.t_igg / tpd if sim.ab.t_igg else None
1940 |     R["t_first_extravasation"] = (sim.t_first_extravasation / tpd
1941 |                                  if sim.t_first_extravasation else None)
1942 |     R["t_first_arrival"] = (sim.t_first_arrival / tpd
1943 |                             if sim.t_first_arrival else None)
1944 |     R["memory_final"] = ln.memory_t + ln.memory_b
1945 |     R["memory_ratio"] = ln.memory_t / ln.peak_cd8 if ln.peak_cd8 > 0 else 0.0
1946 |
1947 |     days = _ser(h, "day"); inf = _ser(h, "infected")
1948 |     m = (days >= 0.4) & (days <= 1.3) & (inf > 5)

```

```

1949 |     if m.sum() > 10:
1950 |         r = float(np.polyfit(days[m], np.log(inf[m]), 1)[0])
1951 |         Tg = (sim.p.eclipse_ticks + 0.5 * sim.p.productive_ticks) / Scale.TICKS_PER_HOUR
1952 |         R["growth_rate_per_day"] = r
1953 |         R["R0_est"] = float(math.exp(r * Tg / 24.0))
1954 |     else:
1955 |         R["growth_rate_per_day"] = float("nan"); R["R0_est"] = float("nan")
1956 |
1957 |     R["ctl_kill_per_day"] = (sim.killed_by_ctl / sim.ctl_agent_ticks * tpd
1958 |                             if sim.ctl_agent_ticks else float("nan"))
1959 |     R["ctl_contact_per_day"] = (sim.ctl_contact / sim.ctl_agent_ticks * tpd
1960 |                                 if sim.ctl_agent_ticks else float("nan"))
1961 |     R["max_target_depletion"] = 1.0 - float(_ser(h, "healthy").min()) / sim.env.n_target
1962 |     R["target_depletion"] = 1.0 - h[-1]["healthy"] / sim.env.n_target
1963 |     R["killed_by_nk"] = sim.killed_by_nk
1964 |     R["killed_by_ctl"] = sim.killed_by_ctl
1965 |     R["apoptosis"] = sim.apoptosis_infected
1966 |     R["cleared_by_phago"] = sim.cleared_by_phago
1967 |     R["cleared_by_decay"] = sim.cleared_by_decay
1968 |     R["cleared_by_complement"] = sim.cleared_by_complement
1969 |     R["cleared_by_antibody"] = sim.cleared_by_antibody
1970 |     R["total_infections"] = sim.total_infections
1971 |     R["n_extravasated"] = sim.n_extravasated
1972 |     R["grad_frac"] = (sim.mover.n_gradient_guided /
1973 |                       max(1, sim.mover.n_gradient_guided + sim.mover.n_persistence))
1974 |     return R
1975 |
1976 |
1977 | @dataclass
1978 | class Check:
1979 |     item: str
1980 |     lit: str
1981 |     ref: str
1982 |     sim: str
1983 |     verdict: str
1984 |
1985 |
1986 | def _judge(v, lo, hi, tol=0.35):
1987 |     if v is None or (isinstance(v, float) and math.isnan(v)):
1988 |         return "판정불가"
1989 |     if lo <= v <= hi:
1990 |         return "일치"
1991 |     span = max(hi - lo, 1e-9)
1992 |     if lo - span * tol <= v <= hi + span * tol:
1993 |         return "근접"
1994 |     return "불일치"
1995 |
1996 |
1997 | def validate(R: dict) -> List[Check]:
1998 |     def f(v):
1999 |         return "미발생" if v is None or (isinstance(v, float) and math.isnan(v)) \
2000 |             else f"Day {v:.2f}"
2001 |     C = []

```

```

2002 | C.append(Check("바이러스 정점 시각", "Day 2 (0.5~1 일 급증 후 2 일 정점)",
2003 |         "carrat2008", f(R["virus_peak_day"]),
2004 |         _judge(R["virus_peak_day"], 1.5, 2.5)))
2005 | C.append(Check("감염세포 정점 시각", "Day 1~3 (표적세포제한 모델)", "baccam2006",
2006 |         f(R["inf_peak_day"]), _judge(R["inf_peak_day"], 1.0, 3.0)))
2007 | C.append(Check("바이러스 배출 종료(정점 1%)", "Day 4.3~5.3 (평균 4.8 일)",
2008 |         "carrat2008", f(R["virus_shed_end_1pct"]),
2009 |         _judge(R["virus_shed_end_1pct"], 4.3, 6.5)))
2010 | C.append(Check("바이러스 완전 제거", "배출 종료 후 수일, 최대 Day 10 내외",
2011 |         "carrat2008", f(R["virus_clear_day"]),
2012 |         _judge(R["virus_clear_day"], 5.0, 12.0)))
2013 | C.append(Check("바이러스 50% 감소", "정점 후 1~2 일 내", "carrat2008",
2014 |         f(R["virus_t50"]),
2015 |         _judge((R["virus_t50"] - R["virus_peak_day"])
2016 |                 if R["virus_t50"] else None, 0.2, 2.0)))
2017 | C.append(Check("체내 R0", "약 22 (감염세포 1 개당 신규 생산감염)", "baccam2006",
2018 |         f("{R['R0_est']:.1f}", _judge(R["R0_est"], 8.0, 40.0)))
2019 | C.append(Check("인터페론 정점", "Day 2 (비강세척액)", "hayden1998",
2020 |         f(R["ifn_peak_day"]), _judge(R["ifn_peak_day"], 1.5, 3.0)))
2021 | C.append(Check("사이토카인 정점", "Day 2 (IL-6/TNF 비강세척액)", "hayden1998",
2022 |         f(R["cyto_peak_day"]), _judge(R["cyto_peak_day"], 1.5, 3.0)))
2023 | C.append(Check("염증/중상 정점", "Day 2~3", "carrat2008",
2024 |         f(R["inflam_peak_day"]), _judge(R["inflam_peak_day"], 1.5, 3.5)))
2025 | C.append(Check("중성구 조직유입 정점", "Day 2~3", "hayden1998",
2026 |         f(R["neut_peak_day"]), _judge(R["neut_peak_day"], 1.5, 3.5)))
2027 | C.append(Check("NK 세포 유입 정점", "Day 2~3", "zhang2007",
2028 |         f(R["nk_peak_day"]), _judge(R["nk_peak_day"], 1.5, 3.5)))
2029 | C.append(Check("T 세포 프라이밍 완료", "Day 1.5~2.5", "mempel2004",
2030 |         f(R["t_prime"]), _judge(R["t_prime"], 1.2, 3.0)))
2031 | C.append(Check("항원특이 CD8 조직 정점", "Day 8~10", "lawrence2005",
2032 |         f(R["ctl_peak_day"]), _judge(R["ctl_peak_day"], 7.0, 11.0)))
2033 | C.append(Check("형질모세포 정점", "Day 7", "wrammert2008",
2034 |         f(R["pc_peak_day"]), _judge(R["pc_peak_day"], 6.0, 9.0)))
2035 | C.append(Check("IgM 검출", "Day 5~7", "clements1986", f(R["t_igm"]),
2036 |         _judge(R["t_igm"], 4.5, 8.0)))
2037 | C.append(Check("IgG 검출", "Day 10~14", "clements1986", f(R["t_igg"]),
2038 |         _judge(R["t_igg"], 9.0, 15.0)))
2039 | C.append(Check("기억세포 형성비율", "정점 대비 5~10%", "kaech2002",
2040 |         f("{R['memory_ratio']*100:.1f}% ",
2041 |         _judge(R["memory_ratio"] * 100, 5.0, 10.0)))
2042 | C.append(Check("CTL 세포당 살상률", "2~16 표적/CTL/일 (모델의존적)",
2043 |         "regoes2007/ganusov2008",
2044 |         f("{R['ctl_kill_per_day']:.2f} 칸/CTL/일",
2045 |         _judge(R["ctl_kill_per_day"], 2.0, 16.0)))
2046 | C.append(Check("표적 상피 소모율", "표적세포제한 모델에서 상당 비율 소모",
2047 |         "baccam2006", f("{R['max_target_depletion']*100:.1f}% ",
2048 |         _judge(R["max_target_depletion"] * 100, 20.0, 95.0)))
2049 | C.append(Check("첫 면역세포 감염부위 도착", "혈관외유출 개시 수시간 내 (급성 염증)",
2050 |         "ley2007/nourshargh2010", f(R["t_first_arrival"]),
2051 |         _judge(R["t_first_arrival"], 0.2, 2.0)))
2052 | return C
2053 |
2054 |

```

```

2055 | # =====
2056 | # SECTION 13. REPORT
2057 | # =====
2058 |
2059 |
2060 | def print_param_table():
2061 |     print()
2062 |     print("#" * 130)
2063 |     print("    [논문 근거 표]   실제값 -> 논문출처 -> 축소비율 -> 시뮬레이션값 (사양서 ㉔㉕)")
2064 |     print("#" * 130)
2065 |     hdr = (f"{'항목':<24}{'실제값':>22} {'단위':<20}{'출처':<24}{' '}"
2066 |           f"{'축소':<14}{'시뮬레이션값'}")
2067 |     print(hdr); print("-" * 150)
2068 |     for r in PARAM_TABLE:
2069 |         src = REFS[r.source].short() if r.source in REFS else r.source
2070 |         print(f"{'r.item':<24}{'r.real':>22} {'r.unit':<20}{'src':<24}{'r.scaling':<14}{'r.sim'}")
2071 |         if r.note:
2072 |             print(f"{' ':<24}    {r.note}")
2073 |     print("-" * 150)
2074 |     print("    * '모델 가정값' 항목은 논문에서 직접 대응 수치를 찾지 못한 값이며, "
2075 |           "실제값처럼 제시하지 않았다 (사양서 ㉔).")
2076 |     print()
2077 |     print("    [인용 문헌]")
2078 |     for k in sorted(REFS):
2079 |         print(f"        - {REFS[k].full()}")
2080 |     print("    * DOI/PMID 는 작성 시점 기준. 인용 전 PubMed 재확인 권장.")
2081 |     print()
2082 |
2083 |
2084 | def print_final_report(sim, R, checks):
2085 |     p = sim.p
2086 |     def d(v):
2087 |         return "미발생" if v is None or (isinstance(v, float) and math.isnan(v)) \
2088 |             else f"Day {v:.2f}"
2089 |
2090 |     print()
2091 |     print("#" * 110)
2092 |     print("    최종 결과 (체계적 이동 조건, 1 회 실행)")
2093 |     print("#" * 110)
2094 |     print("    [병원체]")
2095 |     print(f"        · 최대 병원체 수          : {R['virus_peak']:,.0f} agent "
2096 |           f"        (= {R['virus_peak']*Scale.INV_KAPPA:.2e} 감염단위) @ {d(R['virus_peak_day'])}")
2097 |     print(f"        · 감소 시작 시각          : {d(R['virus_peak_day'])}")
2098 |     print(f"        · 50% / 90% 감소          : {d(R['virus_t50'])} / {d(R['virus_t90'])}")
2099 |     print(f"        · 배출 종료(정점 1%)      : {d(R['virus_shed_end_1pct'])}")
2100 |     print(f"        · 완전 제거              : {d(R['virus_clear_day'])}")
2101 |     print(f"        · 최종 병원체 수          : {R['virus_final']:,.0f}")
2102 |     print(f"        · 초기 증식률 / 추정 R0    : {R['growth_rate_per_day']:~.2f}/일 "
2103 |           f"        f"/ {R['R0_est']:~.1f}")
2104 |     print()
2105 |     print("    [감염세포]")
2106 |     print(f"        · 최대 감염세포 수        : {R['inf_peak']:,.0f} 칸@ {d(R['inf_peak_day'])}")
2107 |     print(f"        · 50% 감소 / 제거        : {d(R['inf_t50'])} / {d(R['inf_clear_day'])}")

```



```

2108 | print(f"    · 누적 감염 발생칸    : {R['total_infections':,]}")
2109 | print(f"    · 표적세포 최대 소모율    : {R['max_target_depletion']*100:.1f}%")
2110 | print()
2111 | print("    [면역세포 도달]")
2112 | print(f"    · 첫 혈관외유출        : {d(R['t_first_extravasation'])}")
2113 | print(f"    · 첫 감염부위 도착        : {d(R['t_first_arrival'])} ★사양서 40-㉔")
2114 | print(f"    · 누적 혈관외유출 세포    : {R['n_extravasated':,]} agent")
2115 | print(f"    · 구배로 방향 결정된 비율 : {R['grad_frac']*100:.1f}% "
2116 |       f"(나머지는 극성 유지, 무작위 0%)")
2117 | print()
2118 | print("    [선천면역]")
2119 | print(f"    · 중성구 정점 / 소모    : {R['neut_peak':,].0f} @ {d(R['neut_peak_day'])}"
2120 |       f" / 식균제거 {R['cleared_by_phago':,]}")
2121 | print(f"    · 단핵구/대식세포 정점  : {R['mono_peak':,].0f} @ {d(R['mono_peak_day'])}")
2122 | print(f"    · NK 세포 정점 / 살상    : {R['nk_peak':,].0f} @ {d(R['nk_peak_day'])}"
2123 |       f" / 감염세포 {R['killed_by_nk':,]} 칸")
2124 | print(f"    · 면역세포 총 사망/소모  : {sim.dead_immune:,} agent")
2125 | print(f"    · 케모카인 정점        : {R['chemo_peak':.4f} @ {d(R['chemo_peak_day'])}")
2126 | print(f"    · 사이토카인 정점      : {R['cyto_peak':.4f} @ {d(R['cyto_peak_day'])}")
2127 | print(f"    · 인터페론 정점        : {R['ifn_peak':.4f} @ {d(R['ifn_peak_day'])}")
2128 | print(f"    · 보체 정점            : {R['comp_peak']*100:.1f}% @ {d(R['comp_peak_day'])}")
2129 | print(f"    · 염증 정점 / 조직손상   : {R['inflam_peak']*100:.1f}% "
2130 |       f"@ {d(R['inflam_peak_day'])} / 누적 {R['damage_final':.3f]}")
2131 | print()
2132 | print("    [후천면역]")
2133 | print(f"    · 항원 림프절 도착      : {d(R['t_antigen'])}")
2134 | print(f"    · 조력 T/살해 T/B 활성화 : {d(R['t_cd4'])} / {d(R['t_cd8'])} / {d(R['t_b'])}")
2135 | print(f"    · 항원특이 CTL 조직 정점 : {R['ctl_peak':,].0f} @ {d(R['ctl_peak_day'])}")
2136 | print(f"    · 형질세포 정점          : {R['pc_peak':,].0f} @ {d(R['pc_peak_day'])}")
2137 | print(f"    · 항체 생성 시작(IgM/IgG) : {d(R['t_igm'])} / {d(R['t_igg'])}")
2138 | print(f"    · 최대 항체 농도          : {R['ab_peak':.2f} ug/mL @ {d(R['ab_peak_day'])}")
2139 | print(f"    · CTL 감염세포 제거      : {R['killed_by_ctl':,]} 칸")
2140 | print(f"    · 기억세포 형성량        : {R['memory_final':,].0f} "
2141 |       f"(정점의 {R['memory_ratio']*100:.1f}%)")
2142 | print()
2143 | tot = max(1, sim.cleared_virus)
2144 | print("    [병원체 제거 기여도]")
2145 | for name, v in (("비특이 제거", R['cleared_by_decay']),
2146 |               ("식세포 포식", R['cleared_by_phago']),
2147 |               ("보체", R['cleared_by_complement']),
2148 |               ("항체 중화", R['cleared_by_antibody'])):
2149 |     print(f"    · {name:<22}: {v:>10,} ({v/tot*100:5.1f}%)")
2150 | toti = max(1, sim.killed_infected + sim.apoptosis_infected)
2151 | print("    [감염세포 제거 기여도]")
2152 | for name, v in (("NK 세포 살상", R['killed_by_nk']),
2153 |               ("살해 T 세포 살상", R['killed_by_ctl']),
2154 |               ("바이러스 유도 세포자멸", R['apoptosis'])):
2155 |     print(f"    · {name:<22}: {v:>10,} ({v/toti*100:5.1f}%)")
2156 | print()
2157 |
2158 | print("#" * 110)
2159 | print("    [이론(문헌) 대비 검증표] - 입력값이 아니라 '창발 결과'만 대조")
2160 | print("#" * 110)

```

```

2161 |     print(f"{'검증 항목':<26}{'문헌 기준값':<42}{'시뮬레이션':<22}{'판정'}")
2162 |     print("-" * 110)
2163 |     ok = near = bad = 0
2164 |     for ck in checks:
2165 |         print(f"{ck.item:<26}{ck.lit:<42}{ck.sim:<22}{ck.verdict}")
2166 |         print(f"{'':<26}└─ 출처: "
2167 |               f"{REFS[ck.ref].short() if ck.ref in REFS else ck.ref}")
2168 |         ok += ck.verdict == "일치"
2169 |         near += ck.verdict == "근접"
2170 |         bad += ck.verdict == "불일치"
2171 |     tot_c = ok + near + bad
2172 |     print("-" * 110)
2173 |     print(f"  종합: 일치 {ok} / 근접 {near} / 불일치 {bad}   "
2174 |           f"(판정대상 {tot_c}개)")
2175 |     print(f"  ★ 이론 일치율 = {(ok+near)/max(1,tot_c)*100:.1f}% "
2176 |           f"(엄격기준 '일치'만 = {ok/max(1,tot_c)*100:.1f}%)")
2177 |     print()
2178 |     return {"ok": ok, "near": near, "bad": bad, "total": tot_c,
2179 |            "rate": (ok + near) / max(1, tot_c) * 100,
2180 |            "strict": ok / max(1, tot_c) * 100}
2181 |
2182 |
2183 | # =====
2184 | # SECTION 14.  COMPARISON WITH RANDOM-WALK RESULT
2185 | # =====
2186 |
2187 |
2188 | def print_comparison(R: dict, score: dict, rand_path: str):
2189 |     if not os.path.exists(rand_path):
2190 |         print("  (무작위 이동 결과 파일을 찾을 수 없어 비교를 생략한다)")
2191 |         return
2192 |     with open(rand_path, encoding="utf-8") as f:
2193 |         rnd = json.load(f)["summary"]
2194 |
2195 |     def g(k, default=None):
2196 |         v = rnd.get(k, default)
2197 |         return v
2198 |
2199 |     def fmt(v, kind="day"):
2200 |         if v is None:
2201 |             return "미발생"
2202 |         if kind == "day":
2203 |             return f"Day {v:.2f}"
2204 |         if kind == "pct":
2205 |             return f"{v*100:.1f}%"
2206 |         if kind == "num":
2207 |             return f"{v:,.0f}"
2208 |         return f"{v:.2f}"
2209 |
2210 |     print("#" * 110)
2211 |     print("  [무작위 이동 vs 체계적 이동 비교]")
2212 |     print("  ※ 두 실험은 서로 독립적으로 실행되었으며, 상태를 공유하지 않는다.")
2213 |     print("  생물학 파라미터 (세포수/비율/바이러스동역학/후천면역시간/항체/보체/IFN)는")

```

```

2214 | print("    동일 문헌에서 동일하게 산출하였고, '이동 규칙'만 다르다.")
2215 | print("#" * 110)
2216 | rows = [
2217 |     ( "최대 병원체 수",          fmt(g("virus_peak"), "num"),    fmt(R["virus_peak"], "num"), "낮을수록 우세"),
2218 |     ( "병원체 정점 시각",        fmt(g("virus_peak_day")),      fmt(R["virus_peak_day"], "-"),
2219 |     ( "병원체 50% 감소",          fmt(g("virus_t50")),          fmt(R["virus_t50"], "빠를수록 우세"),
2220 |     ( "배출 종료(정점 1%)",       fmt(g("virus_shed_end_1pct")), fmt(R["virus_shed_end_1pct"], "빠를수록 우세"),
2221 |     ( "병원체 완전 제거",        fmt(g("virus_clear_day")),    fmt(R["virus_clear_day"], "빠를수록 우세"),
2222 |     ( "최대 감염세포 수",         fmt(g("inf_peak"), "num"),    fmt(R["inf_peak"], "num"), "낮을수록 우세"),
2223 |     ( "감염세포 제거 시각",       fmt(g("inf_clear_day")),      fmt(R["inf_clear_day"], "빠를수록 우세"),
2224 |     ( "누적 감염 발생칸",         fmt(g("total_infections"), "num"), fmt(R["total_infections"], "num"), "낮을수록 우세"),
2225 |     ( "표적세포 최대 소모율",     fmt(g("max_target_depletion"), "pct"), fmt(R["max_target_depletion"], "pct"), "낮을수록 우세"),
2226 |     ( "누적 조직손상",           fmt(g("damage_final"), "raw"), fmt(R["damage_final"], "raw"), "낮을수록 우세"),
2227 |     ( "염증 정점",               fmt(g("inflam_peak"), "pct"),  fmt(R["inflam_peak"], "pct"), "낮을수록 우세"),
2228 |     ( "면역세포 총 사망",         fmt(g("cleared_by_phago") and None, "num") if False else "-", "-", "-"),
2229 |     ( "식세포 포식 제거량",       fmt(g("cleared_by_phago"), "num"), fmt(R["cleared_by_phago"], "num"), "높을수록 탐색기여"),
2230 |     ( "NK 감염세포 살상",        fmt(g("killed_by_nk"), "num"),  fmt(R["killed_by_nk"], "num"), "높을수록 탐색기여"),
2231 |     ( "CTL 감염세포 살상",       fmt(g("killed_by_ctl"), "num"),  fmt(R["killed_by_ctl"], "num"), "높을수록 탐색기여"),
2232 |     ( "CTL 접촉률(회/CTL/일)",    fmt(g("ctl_contact_per_day"), "raw"), fmt(R["ctl_contact_per_day"], "raw"), "높을수록 탐색효율"),
2233 |     ( "첫 감염부위 도착",        "해당없음(전역배치)",          fmt(R["t_first_arrival"], "-"),
2234 | ]
2235 | print(f"{'지표':<24}{'무작위 이동':>22}{'체계적 이동':>22}    {'해석'}")
2236 | print("-" * 110)
2237 | for name, a, b, note in rows:
2238 |     if a == "-" and b == "-":
2239 |         continue
2240 |     print(f"{name:<24}{a:>22}{b:>22}    {note}")
2241 | print("-" * 110)
2242 | print()
2243 | return rnd
2244 |
2245 |
2246 | # =====
2247 | # SECTION 15.  VISUALIZATION
2248 | # =====
2249 |
2250 |
2251 | def save_figures(sim, outdir):
2252 |     import matplotlib
2253 |     matplotlib.use("Agg")
2254 |     import matplotlib.pyplot as plt
2255 |     h = sim.history
2256 |     d = _ser(h, "day")
2257 |     fig, ax = plt.subplots(2, 3, figsize=(19, 9))
2258 |     fig.suptitle("Influenza A ABM - SYSTEMATIC (chemotaxis-only) immune search",
2259 |                 fontweight="bold")
2260 |
2261 |     a = ax[0, 0]
2262 |     a.plot(d, _ser(h, "virus"), "r-", lw=1.6, label="free virus")
2263 |     a.plot(d, _ser(h, "infected"), "m--", label="infected cells")
2264 |     a.plot(d, _ser(h, "healthy"), "g-", alpha=.6, label="healthy epithelium")
2265 |     a.set_yscale("log"); a.set_ylim(1, None); a.set_title("Infection dynamics")
2266 |     a.set_xlabel("day"); a.grid(alpha=.3); a.legend(fontsize=8)

```

```

2267 |
2268 | a = ax[0, 1]
2269 | a.plot(d, _ser(h, "neutrophil"), label="neutrophil")
2270 | a.plot(d, _ser(h, "monocyte"), label="monocyte/macrophage")
2271 | a.plot(d, _ser(h, "nk"), label="NK")
2272 | a.plot(d, _ser(h, "in_tissue"), "k--", alpha=.6, label="cells in tissue")
2273 | a.set_title("Innate cells"); a.set_xlabel("day"); a.grid(alpha=.3); a.legend(fontsize=8)
2274 |
2275 | a = ax[0, 2]
2276 | a.plot(d, _ser(h, "chemokine"), label="chemokine (mean)")
2277 | a.plot(d, _ser(h, "cytokine"), label="cytokine (mean)")
2278 | a.plot(d, _ser(h, "interferon"), label="interferon (mean)")
2279 | a.set_yscale("log"); a.set_title("Chemical signal fields")
2280 | a.set_xlabel("day"); a.grid(alpha=.3); a.legend(fontsize=8)
2281 |
2282 | a = ax[1, 0]
2283 | a.plot(d, _ser(h, "killer_t_spec"), "g-", label="specific CTL (tissue)")
2284 | a.plot(d, _ser(h, "helper_t_spec"), "b-", label="specific Th (tissue)")
2285 | a.plot(d, _ser(h, "ln_cd8"), "g--", alpha=.5, label="CD8 in lymph node")
2286 | a.set_yscale("symlog"); a.set_title("Adaptive cellular response")
2287 | a.set_xlabel("day"); a.grid(alpha=.3); a.legend(fontsize=8)
2288 |
2289 | a = ax[1, 1]
2290 | a.plot(d, _ser(h, "antibody"), "b-", label="antibody (ug/mL)")
2291 | a.plot(d, _ser(h, "complement") * 100, "purple", label="complement (%)")
2292 | a.plot(d, _ser(h, "inflammation") * 100, "r--", label="inflammation (%)")
2293 | a.set_title("Humoral factors"); a.set_xlabel("day"); a.grid(alpha=.3)
2294 | a.legend(fontsize=8)
2295 |
2296 | a = ax[1, 2]
2297 | im = a.imshow(np.log10(sim.env.chemokine + 1e-6), origin="lower", cmap="inferno")
2298 | a.set_title("Chemokine field at end (log10)")
2299 | fig.colorbar(im, ax=a, fraction=0.046)
2300 |
2301 | plt.tight_layout()
2302 | path = os.path.join(outdir, "immune_systematic_result.png")
2303 | fig.savefig(path, dpi=125)
2304 | return path
2305 |
2306 |
2307 | # =====
2308 | # SECTION 16.  MAIN
2309 | # =====
2310 |
2311 |
2312 | def main(argv=None):
2313 |     ap = argparse.ArgumentParser()
2314 |     ap.add_argument("--days", type=int, default=Params.max_days)
2315 |     ap.add_argument("--seed", type=int, default=Params.seed)
2316 |     ap.add_argument("--outdir", default=".")
2317 |     ap.add_argument("--random-json", default="")
2318 |     ap.add_argument("--no-params", action="store_true")
2319 |     args = ap.parse_args(argv)

```

```

2320 |
2321 | print("=" * 110)
2322 | print(" 면역 시뮬레이션 - 체계적 이동(SYSTEMATIC / CHEMOTAXIS-ONLY) 조건")
2323 | print(" Influenza A / 상기도 상피 + 혈관 네트워크 + 배액 림프절 / kappa = 1/1000")
2324 | print(f" 격자 {Scale.GRID}x{Scale.GRID} = {Scale.SITES:,}칸, "
2325 |       f"1 tick = {Scale.DT_MIN:.0f}분, 1일 = {Scale.TICKS_PER_DAY} tick")
2326 | print(f" 이동 규칙: 케모카인 8방향 argmax + 혈류 + 혈관구조. 난수 방향 0%.")
2327 | print(f" seed = {args.seed}, 시행 횟수 = 1")
2328 | print("=" * 110)
2329 |
2330 | if not args.no_params:
2331 |     print_param_table()
2332 |
2333 | p = Params(); p.max_days = args.days; p.seed = args.seed
2334 | sim = SystematicImmuneSimulation(p, seed=args.seed, verbose=True)
2335 | print(f" [환경] 표적 상피 {sim.env.n_target:,}칸, 혈관 {sim.env.n_vessel:,}칸, "
2336 |       f"림프절 {(LN_R1-LN_R0)*(LN_C1-LN_C0):,}칸")
2337 | sim.run()
2338 |
2339 | R = analyze(sim)
2340 | checks = validate(R)
2341 | print_daily_table(sim.daily,
2342 |                  "하루 경과별 결과표 (Day 0 = 감염 직후, 마지막 열 = 병원체 소멸 후 최종)")
2343 | score = print_final_report(sim, R, checks)
2344 | print_comparison(R, score, args.random_json)
2345 |
2346 | try:
2347 |     path = save_figures(sim, args.outdir)
2348 |     print(f" [그래프 저장] {path}")
2349 | except Exception as e:
2350 |     print(f" (그래프 저장 실패: {e})")
2351 |
2352 | with open(os.path.join(args.outdir, "immune_systematic_history.json"),
2353 |          "w", encoding="utf-8") as f:
2354 |     json.dump({"daily": sim.daily,
2355 |              "summary": {k: (None if (isinstance(v, float) and math.isnan(v))
2356 |                             else v) for k, v in R.items()},
2357 |              "score": score}, f, ensure_ascii=False, indent=1)
2358 | print(f" [원자료 저장] immune_systematic_history.json")
2359 | print(f" [실행시간] {sim.wall_time:.1f}초")
2360 | return 0
2361 |
2362 |
2363 | if __name__ == "__main__":
2364 |     sys.exit(main())
2365 |

```

부록 C. 폐렴사슬알균 감염 모형 — 정상적 환경 및 염증성 환경 통합

세포의 증식균인 *Streptococcus pneumoniae* 감염 모형의 전체 소스코드. 이동 방식을 인자로 받아 두 조건을 동일한 코드에서 실행한다. 6 개 화학주성 축(FPR1 · C5aR1 · CXCR2 · CCR2 · CXCR3 · CXCR5)을 독립적으로 구현하였으며, 말단표적 신호가 중간매개 신호를 압도하는 화학주성 계층 구조를 반영하였다.

파일명	immune_pneumococcus.py	총 줄 수	1,342 줄
실행 환경	Python 3.12 / NumPy · SciPy · Matplotlib	난수 시드	700000, 700037, 700074, 700111, 700148

```

1 | # -*- coding: utf-8 -*-
2 | """
3 | =====
4 | immune_bacteria_abm.py
5 | 세균(Streptococcus pneumoniae) 감염 - 면역반응 Agent-Based Model
6 | 면역세포 이동방식 비교: 무작위 이동(Random Walk) vs 체계적 이동(Chemotaxis)
7 | =====
8 |
9 | 【병원체】 Streptococcus pneumoniae (폐렴사슬알균), 혐막 보유 침습성 혈청형
10 |   - 그람양성, 통성혐기성, 사슬알균속
11 |   - ★비운동성(non-motile) : 편모가 없어 스스로 헤엄치지 못한다
12 |   - 세포외 증식(extracellular) : 숙주세포 안으로 들어가지 않는다
13 |   - 혐막 다당(capsular polysaccharide)이 보체 침착을 방해한다
14 |
15 | 【바이러스 모델과의 근본적 차이 — 본 실험을 세균으로 바꾼 이유】
16 |   1) 감염세포가 없다 -> 세포자멸(apoptosis)이 없다
17 |     인플루엔자 모델에서는 감염세포 제거의 98%가 세포자멸이었고, 이는
18 |     면역세포의 '탐색'과 완전히 무관한 경로였다. 따라서 이동방식의 효과가
19 |     구조적으로 희석되었다. 세균에서는 제거의 거의 전부가 식균작용이므로
20 |     '탐색 효율'이 결과에 직접 반영된다.
21 |   2) 병원체가 스스로 유인물질을 방출한다
22 |     세균은 N-포르밀 펩타이드(fMLF)를 배출한다. 이는 호중구의 FPR1에
23 |     작용하는 최강 화학주성 인자다. 즉 '표적이 곧 신호원'이다.
24 |     자유 바이러스 입자는 어떤 화학주성 신호도 내지 않는다.
25 |   3) 확산 속도가 압도적으로 느리다
26 |     바이러스는 점액 수송과 확산으로 조직 전역에 퍼진다(sigma=30 칸/tick).
27 |     폐렴사슬알균은 비운동성이라 분열한 자리에 머물며 미세군락을 형성한다.
28 |     -> 표적이 공간적으로 '뭉쳐' 있으므로 주화성이 유리할 조건이 된다.
29 |
30 | 【핵심 가설】
31 |   인플루엔자에서 체계적 이동이 우세하지 못했던 원인이 '표적의 공간 분산'
32 |   이었다면, 표적이 국소에 뭉쳐 있고 스스로 신호를 내는 세균 감염에서는
33 |   체계적 이동이 무작위 이동보다 유의하게 우세해야 한다.
34 |
35 | 【축소비용】 kappa = 1/1000 (바이러스 모델과 동일 유지)
36 | =====
37 | """
38 | from __future__ import annotations
39 |
40 | import json

```

```

41 | import math
42 | import sys
43 | import time
44 | from dataclasses import dataclass, field
45 | from typing import Dict, List, Optional, Tuple
46 |
47 | import numpy as np
48 |
49 | # =====
50 | # SECTION 1. 참고문헌
51 | # =====
52 |
53 |
54 | @dataclass(frozen=True)
55 | class Ref:
56 |     key: str
57 |     authors: str
58 |     title: str
59 |     journal: str
60 |     year: int
61 |     doi: str = ""
62 |     pmid: str = ""
63 |
64 |     def cite(self) -> str:
65 |         tail = f" doi:{self.doi}" if self.doi else ""
66 |         tail += f" PMID:{self.pmid}" if self.pmid else ""
67 |         return f"{self.authors} ({self.year}). {self.title}. {self.journal}.{tail}"
68 |
69 |
70 | REFS: Dict[str, Ref] = {
71 |     # ----- 병원체 동역학 -----
72 |     "kadioglu2015": Ref(
73 |         "kadioglu2015",
74 |         "Jose RJ, Williams AE, Chambers RC, Brown JS, et al.",
75 |         "Importance of bacterial replication and alveolar macrophage-independent "
76 |         "clearance mechanisms during early lung infection with "
77 |         "Streptococcus pneumoniae",
78 |         "Infection and Immunity 83(4):1181-1189", 2015,
79 |         "10.1128/IAI.02788-14", "25583525"),
80 |     "lin2020": Ref(
81 |         "lin2020", "Lin J, Zhu L, Lau GW",
82 |         "Streptococcus pneumoniae elaborates persistent and prolonged competent "
83 |         "state during pneumonia-derived sepsis",
84 |         "Infection and Immunity 88(4):e00919-19", 2020,
85 |         "10.1128/IAI.00919-19", "31988172"),
86 |     "reppe2019": Ref(
87 |         "reppe2019",
88 |         "Hamilton JA, Nguyen VT, Wilson C, Mulchandani N, et al.",
89 |         "Clinically relevant model of pneumococcal pneumonia, ARDS, and "
90 |         "nonpulmonary organ dysfunction in mice",
91 |         "American Journal of Physiology - Lung Cellular and Molecular "
92 |         "Physiology 317(5):L659-L675", 2019,
93 |         "10.1152/ajplung.00132.2019", "31461312"),

```

```

94 | "hava2003": Ref(
95 |     "hava2003", "Hava DL, Camilli A",
96 |     "Large-scale identification of serotype 4 Streptococcus pneumoniae "
97 |     "virulence factors",
98 |     "Molecular Microbiology 45(5):1389-1406", 2002,
99 |     "10.1046/j.1365-2958.2002.03106.x", "12207705"),
100 |
101 | # ----- 식균작용 / 보체 -----
102 | "rubio2023": Ref(
103 |     "rubio2023",
104 |     "Rubio AJ, Bakker MG, Bhatnagar S, Sunderhaus A, et al.",
105 |     "Model-based assessment of neutrophil-mediated phagocytosis and "
106 |     "digestion of bacteria across in vitro and in vivo studies",
107 |     "CPT: Pharmacometrics & Systems Pharmacology 12(12):1934-1946", 2023,
108 |     "10.1002/psp4.13045", "37881153"),
109 | "hyams2010": Ref(
110 |     "hyams2010", "Hyams C, Camberlein E, Cohen JM, Bax K, Brown JS",
111 |     "The Streptococcus pneumoniae capsule inhibits complement activity and "
112 |     "neutrophil phagocytosis by multiple mechanisms",
113 |     "Infection and Immunity 78(2):704-715", 2010,
114 |     "10.1128/IAI.00881-09", "19948837"),
115 | "gordon1980": Ref(
116 |     "gordon1980", "Gordon DL, Rice J, Finlay-Jones JJ, McDonald PJ, et al.",
117 |     "Phagocytosis by human alveolar macrophages and neutrophils: qualitative "
118 |     "differences in the opsonic requirements for uptake of Staphylococcus "
119 |     "aureus and Streptococcus pneumoniae in vitro",
120 |     "Journal of Infectious Diseases 141(6):718-724", 1980,
121 |     "10.1093/infdis/141.6.718", "7352714"),
122 | "dalia2010": Ref(
123 |     "dalia2010", "Dalia AB, Standish AJ, Weiser JN",
124 |     "Three surface exoglycosidases from Streptococcus pneumoniae, NanA, "
125 |     "BgaA, and StrH, promote resistance to opsonophagocytic killing by "
126 |     "human neutrophils",
127 |     "Infection and Immunity 78(5):2108-2116", 2010,
128 |     "10.1128/IAI.01125-09", "20160017"),
129 | "bohlson2023": Ref(
130 |     "bohlson2023",
131 |     "Palmer A, Klein K, Cabral-Fernandes L, Gomes-Neto JF, et al.",
132 |     "Purified complement C3b triggers phagocytosis and activation of human "
133 |     "neutrophils via complement receptor 1",
134 |     "Scientific Reports 13:1-15", 2023,
135 |     "10.1038/s41598-022-27279-4", "36609671"),
136 |
137 | # ----- ★화학주성 계층 (본 모델의 핵심 기전) -----
138 | "heit2002": Ref(
139 |     "heit2002", "Heit B, Tavenor S, Raharjo E, Kubes P",
140 |     "An intracellular signaling hierarchy determines direction of migration "
141 |     "in opposing chemotactic gradients",
142 |     "Journal of Cell Biology 159(1):91-102", 2002,
143 |     "10.1083/jcb.200202114", "12370241"),
144 | "wang2016": Ref(
145 |     "wang2016", "Wang X, Qin W, Zhang Y, Zhang H, Sun B",
146 |     "Endotoxin promotes neutrophil hierarchical chemotaxis via the "

```



```

147 |         "p38-membrane receptor pathway",
148 |         "Oncotarget 7(45):73431-73444", 2016,
149 |         "10.18632/oncotarget.12093", "27655688"),
150 |     "weiss2020": Ref(
151 |         "weiss2020",
152 |         "Weiss E, Hanzelmann D, Fehlhaber B, Klos A, von Loewenich FD, et al.",
153 |         "Formyl-peptide receptor activation enhances phagocytosis of "
154 |         "community-acquired methicillin-resistant Staphylococcus aureus",
155 |         "Journal of Infectious Diseases 221(4):668-678", 2020,
156 |         "10.1093/infdis/jiz498", "31573603"),
157 |     "schepetkin2025": Ref(
158 |         "schepetkin2025", "Cattaneo F, Ammendola R, et al.",
159 |         "The N-formyl peptide receptors: much more than chemoattractant "
160 |         "receptors. Relevance in health and disease",
161 |         "Frontiers in Immunology 16:1568629", 2025,
162 |         "10.3389/fimmu.2025.1568629", ""),
163 |     "vanderlinden2017": Ref(
164 |         "vanderlinden2017",
165 |         "van der Linden M, Kuijpers TW, et al.",
166 |         "Differential neutrophil chemotactic response towards IL-8 and "
167 |         "bacterial N-formyl peptides in term newborn infants",
168 |         "Pediatric Research 81(1):142-147", 2017,
169 |         "10.1038/pr.2016.196", "27673422"),
170 |
171 |     # ----- 수용체 발현 -----
172 |     "rudd2019": Ref(
173 |         "rudd2019", "Rudd JM, Pulavendran S, Ashar HK, Ritchey JW, et al.",
174 |         "Neutrophils induce a novel chemokine receptors repertoire during "
175 |         "influenza pneumonia",
176 |         "Frontiers in Cellular and Infection Microbiology 9:108", 2019,
177 |         "10.3389/fcimb.2019.00108", "31041196"),
178 |     "lin2008": Ref(
179 |         "lin2008", "Lin KL, Suzuki Y, Nakano H, Ramsburg E, Gunn MD",
180 |         "CCR2+ monocyte-derived dendritic cells and exudate macrophages produce "
181 |         "influenza-induced pulmonary immune pathology and mortality",
182 |         "Journal of Immunology 180(4):2562-2572", 2008,
183 |         "10.4049/jimmunol.180.4.2562", "18250467"),
184 |     "groom2011": Ref(
185 |         "groom2011", "Groom JR, Luster AD",
186 |         "CXCR3 ligands: redundant, collaborative and antagonistic functions",
187 |         "Immunology and Cell Biology 89(2):207-215", 2011,
188 |         "10.1038/icb.2010.158", "21221121"),
189 |     "guo2005": Ref(
190 |         "guo2005", "Guo RF, Ward PA",
191 |         "Role of C5a in inflammatory responses",
192 |         "Annual Review of Immunology 23:821-852", 2005,
193 |         "10.1146/annurev.immunol.23.021704.115835", "15771587"),
194 |     "okada2002": Ref(
195 |         "okada2002", "Okada T, Ngo VN, Ekland EH, Forster R, Lipp M, et al.",
196 |         "Chemokine requirements for B cell entry to lymph nodes and "
197 |         "Peyer's patches",
198 |         "Journal of Experimental Medicine 196(1):65-75", 2002,
199 |         "10.1084/jem.20020201", "12093871"),

```

```

200 |
201 | # ----- Th17 / 점막 세균면역 -----
202 | "zhang2018": Ref(
203 |     "zhang2018", "Zhang Z, Clarke TB, Weiser JN",
204 |     "Cellular effectors mediating Th17-dependent clearance of pneumococcal "
205 |     "colonization in mice",
206 |     "Journal of Clinical Investigation 119(7):1899-1909", 2009,
207 |     "10.1172/JCI36731", "19509469"),
208 | "lu2008": Ref(
209 |     "lu2008", "Lu YJ, Gross J, Bogaert D, Finn A, et al.",
210 |     "Interleukin-17A mediates acquired immunity to pneumococcal "
211 |     "colonization",
212 |     "PLoS Pathogens 4(9):e1000159", 2008,
213 |     "10.1371/journal.ppat.1000159", "18802458"),
214 |
215 | # ----- 세포 이동 물리량 (바이러스 모델과 공유) -----
216 | "friedl2008": Ref(
217 |     "friedl2008", "Friedl P, Weigelin B",
218 |     "Interstitial leukocyte migration and immune function",
219 |     "Nature Immunology 9(9):960-969", 2008,
220 |     "10.1038/ni.f.212", "18711433"),
221 | "miller2002": Ref(
222 |     "miller2002", "Miller MJ, Wei SH, Parker I, Cahalan MD",
223 |     "Two-photon imaging of lymphocyte motility and antigen response in "
224 |     "intact lymph node",
225 |     "Science 296(5574):1869-1873", 2002,
226 |     "10.1126/science.1070051", "12016203"),
227 | "germain2012": Ref(
228 |     "germain2012", "Germain RN, Robey EA, Cahalan MD",
229 |     "A decade of imaging cellular motility and interaction dynamics in the "
230 |     "immune system",
231 |     "Science 336(6089):1676-1681", 2012,
232 |     "10.1126/science.1221063", "22745423"),
233 | }
234 |
235 | MODEL_ASSUMPTION = "모델 가정값"
236 |
237 |
238 | # =====
239 | # SECTION 2. 실제 측정값 (논문값) – 모든 파라미터의 출처
240 | # =====
241 |
242 | REAL: Dict[str, float] = {
243 |     # ---- 공간 / 시간 축소 ----
244 |     "kappa": 1.0 / 1000.0,          # 개체수 축소비율 (바이러스 모델과 동일)
245 |     "grid_side_mm": 15.0,          # 격자 한 변의 실제 길이
246 |     "tick_min": 6.0,              # 1 tick = 6 분
247 |     "epi_cell_um": 15.0,          # 상피세포 1 개의 대표 크기
248 |
249 |     # ---- 병원체 (Streptococcus pneumoniae) ----
250 |     # kadioglu2015 : 저용량 접종 시 폐 내 배가시간 56 분
251 |     "spn_doubling_min": 56.0,
252 |     # kadioglu2015 / lin2020 / reppe2019 : 마우스 비강내 접종 1e6~1e8 CFU

```

```

253 | "spn_inoculum_cfu": 1.0e6,
254 | # lin2020 : 빈사 상태 마우스의 폐 내 균량 1e8 CFU
255 | "spn_max_burden_cfu": 1.0e8,
256 | # kadioglu2015 : 폐포대식세포 의존성 제거 반감기 42 분
257 | "spn_am_clearance_half-life_min": 42.0,
258 | "spn_cell_um": 1.0, # 구균 직경 약 0.8~1.0 um
259 |
260 | # ---- 식균작용 ----
261 | # rubio2023 : 호중구 1 개의 포식 용량 약 50 CFU
262 | "neut_capacity_cfu": 50.0,
263 | # gordon1980 : 폐렴사슬알균은 40% 이상 고농도 혈청 (보체) 없이는 포식 불가
264 | "spn_opsonin_requirement": 0.40,
265 | # hyams2010 : 협막이 보체 침착과 호중구 포식을 다중 기전으로 저해
266 | "capsule_phago_resistance": 0.85,
267 |
268 | # ---- 화학주성 계층 (heit2002, wang2016) ----
269 | # 말단표적 (fMLF, C5a) 이 중간매개 (IL-8, LTB4) 보다 우선한다
270 | "endtarget_priority": 1.0,
271 |
272 | # ---- 면역세포 이동 물리량 ----
273 | "neut_speed_um_min": 12.0, # friedl2008
274 | "mono_speed_um_min": 4.0,
275 | "nk_speed_um_min": 8.0,
276 | "t_speed_um_min": 11.0, # miller2002
277 | "b_speed_um_min": 6.0,
278 |
279 | # ---- 신호물질 ----
280 | "fmlf_D_um2_s": 300.0, # 저분자 펩타이드, 빠른 확산 (모델 추정)
281 | "fmlf_half-life_min": 20.0, # 조직 펩티다제에 의한 분해 (모델 추정)
282 | "c5a_half-life_min": 1.0, # guo2005 : 카르복시펩티다제 N 에 의해 초고속 분해
283 | "cxcl8_half-life_h": 2.0,
284 | "ccl2_half-life_h": 4.0,
285 | "cxcr3l_half-life_h": 6.0,
286 | "cxcl12_half-life_h": 8.0,
287 |
288 | # ---- 후천면역 (항협막 IgG) ----
289 | "igm_detect_day": 5.0,
290 | "igg_detect_day": 9.0,
291 | "igm_half-life_d": 5.0,
292 | "igg_half-life_d": 21.0,
293 | "th17_peak_day": 7.0, # lu2008 / zhang2018
294 | }
295 |
296 |
297 | # =====
298 | # SECTION 3. 축척 변환
299 | # =====
300 |
301 |
302 | class Scale:
303 |     GRID = 1000
304 |     FIELD_BIN = 5
305 |     FIELD_N = GRID // FIELD_BIN # 200 x 200 화확신호장

```

```

306 | EPI_PERIOD = 25                # 상피 밴드 주기(칸)
307 | TICKS_PER_HOUR = 60.0 / REAL["tick_min"]      # 10
308 | TICKS_PER_DAY = int(24 * TICKS_PER_HOUR)      # 240
309 | UM_PER_CELL = REAL["grid_side_mm"] * 1000.0 / GRID    # 15 um/칸
310 | KAPPA = REAL["kappa"]
311 |
312 | @staticmethod
313 | def per_tick_from_half-life_h(h: float) -> float:
314 |     return 1.0 - 0.5 ** (1.0 / (h * Scale.TICKS_PER_HOUR))
315 |
316 | @staticmethod
317 | def per_tick_from_half-life_min(m: float) -> float:
318 |     return 1.0 - 0.5 ** (1.0 / (m / REAL["tick_min"]))
319 |
320 | @staticmethod
321 | def speed_cells_per_tick(um_per_min: float) -> float:
322 |     return um_per_min * REAL["tick_min"] / Scale.UM_PER_CELL
323 |
324 | @staticmethod
325 | def agents_from_cfu(cfu: float) -> int:
326 |     return max(1, int(round(cfu * Scale.KAPPA)))
327 |
328 | @staticmethod
329 | def division_prob_per_tick(doubling_min: float) -> float:
330 |     """배가시간 -> tick 당 분열 확률 (지수증식 등가)"""
331 |     ticks = doubling_min / REAL["tick_min"]
332 |     return 2.0 ** (1.0 / ticks) - 1.0
333 |
334 |
335 | print(f"[축척] 1 tick = {REAL['tick_min']:.0f}분, 1일 = {Scale.TICKS_PER_DAY} tick, "
336 |       f"1칸 = {Scale.UM_PER_CELL:.1f} um, kappa = 1/{int(1/Scale.KAPPA)}")
337 |
338 |
339 | # =====
340 | # SECTION 4. 파라미터
341 | # =====
342 |
343 | DIR8 = np.array([(1, 0), (1, 1), (0, 1), (-1, 1), (-1, 0), (-1, -1),
344 |                 (0, -1), (1, -1)], dtype=np.float32)
345 | DIR8 /= np.linalg.norm(DIR8, axis=1, keepdims=True)
346 | DIR8_OFF = [(1, 0), (1, 1), (0, 1), (-1, 1), (-1, 0), (-1, -1), (0, -1), (1, -1)]
347 |
348 |
349 | @dataclass
350 | class Params:
351 |     max_days: int = 25
352 |     seed: int = 20260809
353 |
354 |     # ----- 병원체 -----
355 |     inoculum: int = Scale.agents_from_cfu(REAL["spn_inoculum_cfu"])    # 1000
356 |     carrying_capacity: int = Scale.agents_from_cfu(REAL["spn_max_burden_cfu"])
357 |     div_p: float = Scale.division_prob_per_tick(REAL["spn_doubling_min"])
358 |     # ★비운동성: 딸세포는 모세포 바로 옆에 놓인다 (미세균락 형성)

```

```

359 | daughter_sigma: float = 1.6          # 칸
360 | drift_sigma: float = 0.60           # 점액섬모 수송에 의한 미세 이동 (칸/tick)
361 | # ★상주 폐포대식세포 (AM)의 제거는 개체수가 유한하므로 고균량에서 포화한다.
362 | #   저균량에서는 논문값 (반감기 42 분)에 근접하고, 고균량에서는 AM이 압도되어
363 | #   AM 비의존 기전 (중성구 동원)이 필요해진다 (Jose 등, 2015).
364 | am_clearance_p: float = Scale.per_tick_from_half-life_min(
365 |     REAL["spn_am_clearance_half-life_min"])          # 0.0943
366 | am_saturation_n: float = 260.0      # 포화 상수 (agent)
367 | # 협막 저항 (hyams2010)
368 | capsule_resist: float = REAL["capsule_phago_resistance"]
369 | # ★영양 면역(nutritional immunity): 염증 시 호중구가 분비하는 칼프로텍틴
370 | #   (S100A8/A9)이 Mn·Zn을 격리하여 세균 증식을 억제한다. 56 분 배가시간은
371 | #   저용량 점종 초기(무염증 상태)의 측정값이므로, 염증이 진행되면
372 | #   실험 증식률이 감소하는 것을 반영한다.
373 | nutritional_immunity: float = 0.78
374 | ops_decay: float = 0.006            # 표면 재생/보체 조절인자에 의한 옴소닌 소실
375 | # 조직 부착/침습에 의한 상피 손상
376 | epi_damage_p: float = 0.0016       # 균 1개가 접한 상피칸을 손상시킬 확률
377 |
378 | # ----- 식균작용 -----
379 | neut_capacity: int = int(REAL["neut_capacity_cfu"])   # 50 (rubio2023)          # 50
380 | mono_capacity: int = 120
381 | neut_phago_p: float = 0.92          # 옴소닌화된 균에 대한 기본 포식확률
382 | mono_phago_p: float = 0.80
383 | opsonin_need: float = REAL["spn_opsonin_requirement"] # 0.40
384 |
385 | # ----- 보체 -----
386 | complement_on: float = 0.030
387 | complement_off: float = 0.011
388 | c3b_deposit_rate: float = 0.085    # tick 당 균 표면 C3b 침착 확률
389 |
390 | # ----- 신호물질 축 (수용체별로 완전히 분리) -----
391 | # [축 1] fMLF -> FPR1 (말단표적) : ★세균 자신이 방출
392 | fmlf_per_bacterium: float = 0.00028
393 | fmlf_decay: float = Scale.per_tick_from_half-life_min(
394 |     REAL["fmlf_half-life_min"])
395 | fmlf_sigma: float = 3.0
396 | # [축 2] C5a -> C5aR1 (말단표적) : 균 표면 보체 활성화 산물
397 | c5a_per_bacterium: float = 0.00020
398 | c5a_decay: float = 0.55             # 초고속 분해 (guo2005), tick 하한 적용
399 | c5a_sigma: float = 1.6
400 | # [축 3] CXCL8 -> CXCR2 (중간매개) : 상피/식세포 유래
401 | cxcl8_from_epi: float = 0.016
402 | cxcl8_from_neut: float = 0.005
403 | cxcl8_decay: float = Scale.per_tick_from_half-life_h(REAL["cxcl8_half-life_h"])
404 | cxcl8_sigma: float = 2.8
405 | # [축 4] CCL2 -> CCR2 (중간매개) : 단핵구 동원
406 | ccl2_from_epi: float = 0.011
407 | ccl2_from_mono: float = 0.006
408 | ccl2_decay: float = Scale.per_tick_from_half-life_h(REAL["ccl2_half-life_h"])
409 | ccl2_sigma: float = 2.4
410 | # [축 5] CXCL9/10 -> CXCR3 : NK/T. IFN-gamma 유도성
411 | cxcr3l_basal: float = 0.0022

```

```

412 | cxcr3l_ifng_gain: float = 0.010
413 | cxcr3l_decay: float = Scale.per_tick_from_half-life_h(REAL["cxcr3l_half-life_h"])
414 | cxcr3l_sigma: float = 2.0
415 | # [축6] CXCL13 -> CXCR5 : B 세포 림프절 귀소
416 | cxcl13_from_ln: float = 0.004
417 | cxcl13_decay: float = Scale.per_tick_from_half-life_h(REAL["cxcl12_half-life_h"])
418 | cxcl13_sigma: float = 3.2
419 |
420 | # ----- ★화학주성 계층 (heit2002, wang2016) -----
421 | # 말단표적(fMLF/C5a) 신호가 중간매개(CXCL8/CCL2)보다 우선한다.
422 | # 말단표적 신호가 이 역치를 넘으면 중간매개 신호를 무시한다.
423 | endtarget_override_thr: float = 0.0016
424 | endtarget_weight: float = 1.0
425 | intermediary_weight: float = 0.35
426 |
427 | # ----- 사이토카인 / 인터페론감마 -----
428 | illb_from_epi: float = 0.012 # 염증 구동
429 | illb_decay: float = Scale.per_tick_from_half-life_h(3.0)
430 | illb_sigma: float = 2.2
431 | ifng_from_nk_t: float = 0.0060 # NK/Th1 유래 IFN-gamma
432 | ifng_decay: float = Scale.per_tick_from_half-life_h(4.0)
433 | ifng_sigma: float = 2.2
434 |
435 | # ----- 혈관외유출 역치 (수용체별로 다름) -----
436 | extrav_FPR1: float = 0.00060 # 호중구: 가장 민감
437 | extrav_C5aR1: float = 0.00070
438 | extrav_CXCR2: float = 0.0100
439 | extrav_CCR2: float = 0.0160
440 | extrav_CXCR3: float = 0.0240
441 | extrav_CXCR5: float = 9.9e9 # B 세포는 조직으로 나가지 않음
442 | reentry_thr: float = 1e-5
443 |
444 | # ----- 면역세포 초기 개체수 (바이러스 모델과 동일) -----
445 | n_neut0: int = 3000
446 | n_mono0: int = 300
447 | n_nk0: int = 195
448 | n_th0: int = 675
449 | n_tc0: int = 375
450 | n_b0: int = 180
451 |
452 | # ----- 동원 -----
453 | neut_recruit_gain: float = 42000.0
454 | mono_recruit_gain: float = 4200.0
455 | nk_recruit_gain: float = 1100.0
456 | neut_life_ticks: int = 240 # 약 1일
457 | mono_life_ticks: int = 1800
458 | nk_life_ticks: int = 2600
459 |
460 | # ----- 이동 속도 -----
461 | neut_speed: float = Scale.speed_cells_per_tick(REAL["neut_speed_um_min"])
462 | mono_speed: float = Scale.speed_cells_per_tick(REAL["mono_speed_um_min"])
463 | nk_speed: float = Scale.speed_cells_per_tick(REAL["nk_speed_um_min"])
464 | t_speed: float = Scale.speed_cells_per_tick(REAL["t_speed_um_min"])

```

```

465 |     b_speed: float = Scale.speed_cells_per_tick(REAL["b_speed_um_min"])
466 |     vessel_speed: float = 26.0
467 |
468 |     # ----- 후천면역 -----
469 |     ln_priming_ticks: int = 480          # 2 일
470 |     th17_gain: float = 1.0
471 |     ab_igm_rate: float = 0.055
472 |     ab_igg_rate: float = 0.030
473 |     igm_decay: float = Scale.per_tick_from_half-life_h(REAL["igm_half-life_d"] * 24)
474 |     igg_decay: float = Scale.per_tick_from_half-life_h(REAL["igg_half-life_d"] * 24)
475 |     ab_opsonin_gain: float = 0.075      # 항혈액 IgG 의 흡수인 기여
476 |     memory_fraction: float = 0.10
477 |     teff_life_ticks: int = 1440        # 효과기 T 세포 수명 약 6 일
478 |
479 |
480 | # =====
481 | # SECTION 5. 이동 엔진 (무작위 vs 체계적)
482 | # =====
483 |
484 | STROMA, HEALTHY, DEAD, VESSEL, LYMPH = 0, 1, 2, 3, 4
485 | LN_R0, LN_R1, LN_C0, LN_C1 = 460, 540, 60, 140
486 |
487 |
488 | class MovementEngine:
489 |     """
490 |     두 이동방식의 유일한 차이점을 담는 클래스.
491 |     random      : 방향을 균일난수로 뽑는다 (Brownian). 신호를 전혀 보지 않는다.
492 |     systematic  : 자기 수용체 축의 8 방향 농도 중 최대 방향으로 간다.
493 |                  *화학주성 계층: 말단표적 (fMLF/C5a) 신호가 역치를 넘으면
494 |                  중간매개 (CXCL8/CCL2) 신호를 무시한다.
495 |     """
496 |
497 |     def __init__(self, mode: str, rng: np.random.Generator):
498 |         assert mode in ("random", "systematic")
499 |         self.mode = mode
500 |         self.rng = rng
501 |         self.n_gradient_guided = 0
502 |         self.n_move_calls = 0
503 |         self.n_endtarget_led = 0
504 |
505 |     @staticmethod
506 |     def sample8(stack: np.ndarray, fr: np.ndarray, fc: np.ndarray) -> np.ndarray:
507 |         return stack[:, fr, fc].T          # (N, 8)
508 |
509 |     def step(self, x, y, hx, hy, speed, end_stack, mid_stack, fr, fc, p):
510 |         n = x.size
511 |         if n == 0:
512 |             return
513 |         self.n_move_calls += n
514 |
515 |         if self.mode == "random":
516 |             # ---- 무작위 이동: 균일난수 방향 ----
517 |             th = self.rng.uniform(0.0, 2.0 * math.pi, n).astype(np.float32)

```

```

518 |         dx = np.cos(th)
519 |         dy = np.sin(th)
520 |     else:
521 |         # ---- 체계적 이동: 화학주성 ----
522 |         e = self.sample8(end_stack, fr, fc) if end_stack is not None else None
523 |         m = self.sample8(mid_stack, fr, fc) if mid_stack is not None else None
524 |
525 |         if e is None:
526 |             vals = m
527 |             lead_end = np.zeros(n, bool)
528 |         elif m is None:
529 |             vals = e
530 |             lead_end = np.ones(n, bool)
531 |         else:
532 |             # *계층 판정: 말단표적 신호가 역치를 넘는 세포는 그것만 따른다
533 |             emax = e.max(axis=1)
534 |             lead_end = emax > p.endtarget_override_thr
535 |             vals = np.where(lead_end[:, None],
536 |                             e * p.endtarget_weight,
537 |                             m * p.intermediary_weight)
538 |             self.n_endtarget_led += int(np.count_nonzero(lead_end))
539 |
540 |             best = np.argmax(vals, axis=1)
541 |             vmax = vals[np.arange(n), best]
542 |             vmin = vals.min(axis=1)
543 |             has_grad = (vmax - vmin) > 1e-12
544 |             self.n_gradient_guided += int(np.count_nonzero(has_grad))
545 |
546 |             d = DIR8[best]
547 |             # 구배가 없으면 기존 극성(heading)을 유지한다 (난수 사용 안 함)
548 |             dx = np.where(has_grad, d[:, 0], hx)
549 |             dy = np.where(has_grad, d[:, 1], hy)
550 |             nz = (np.abs(dx) + np.abs(dy)) < 1e-9
551 |             if nz.any():
552 |                 dx[nz], dy[nz] = 1.0, 0.0
553 |
554 |             hx[:] = dx
555 |             hy[:] = dy
556 |             x += dx * speed
557 |             y += dy * speed
558 |             self.bound(x, y)
559 |
560 |     @staticmethod
561 |     def bound(x, y):
562 |         lim = float(Scale.GRID) - 1e-3
563 |         np.abs(x, out=x)
564 |         np.abs(y, out=y)
565 |         for a in (x, y):
566 |             over = a > lim
567 |             if over.any():
568 |                 a[over] = 2.0 * lim - a[over]
569 |         np.clip(a, 0.0, lim, out=a)
570 |

```



```

571 |
572 | # =====
573 | # SECTION 6. 환경 (조직 + 혈관 + 림프절 + 6개 신호축)
574 | # =====
575 |
576 |
577 | class Environment:
578 |     def __init__(self, p: Params, rng):
579 |         self.p = p
580 |         self.rng = rng
581 |         g = Scale.GRID
582 |         st = np.full((g, g), STROMA, np.uint8)
583 |         # 상피밴드 (주기 25 칸 중 12 칸이 상피)
584 |         for r0 in range(0, g, Scale.EPI_PERIOD):
585 |             st[r0:r0 + 12, :] = HEALTHY
586 |         # 혈관 (상피밴드 사이)
587 |         for r0 in range(0, g, Scale.EPI_PERIOD):
588 |             rr = r0 + 13
589 |             if rr < g:
590 |                 st[rr, :] = VESSEL
591 |         for c0 in range(0, g, 120):
592 |             st[:, c0] = VESSEL
593 |         # 림프절
594 |         st[LN_R0:LN_R1, LN_C0:LN_C1] = LYMPH
595 |         self.state = st.reshape(-1)
596 |
597 |         self.n_target0 = int(np.count_nonzero(self.state == HEALTHY))
598 |         self.vessel_sites = np.flatnonzero(self.state == VESSEL).astype(np.int32)
599 |
600 |         f = Scale.FIELD_N
601 |         z = lambda: np.zeros((f, f), np.float32)
602 |         # ---- 6개 신호축 (수용체별 완전 분리) ----
603 |         self.fmlf = z() # FPR1 (말단표적) : 세균 자신
604 |         self.c5a = z() # C5aR1 (말단표적) : 보체
605 |         self.cxcl8 = z() # CXCR2 (중간매개) : 상피/호중구
606 |         self.ccl2 = z() # CCR2 (중간매개) : 상피/단핵구
607 |         self.cxcr3l = z() # CXCR3 : IFN-gamma 유도
608 |         self.cxcl13 = z() # CXCR5 : 림프절
609 |         self.il1b = z() # 염증 구동
610 |         self.ifng = z() # IFN-gamma
611 |
612 |         lr = slice(LN_R0 // Scale.FIELD_BIN, LN_R1 // Scale.FIELD_BIN + 1)
613 |         lc = slice(LN_C0 // Scale.FIELD_BIN, LN_C1 // Scale.FIELD_BIN + 1)
614 |         self.ln_slice = (lr, lc)
615 |
616 |         self.complement = 0.0
617 |         self.inflammation = 0.0
618 |         self.damage = 0.0
619 |         self.n_dead_epi = 0
620 |
621 | # -----
622 | @staticmethod
623 | def site_of(x, y):

```

```

624 |         c = x.astype(np.int32)
625 |         r = y.astype(np.int32)
626 |         np.clip(c, 0, Scale.GRID - 1, out=c)
627 |         np.clip(r, 0, Scale.GRID - 1, out=r)
628 |         return r * Scale.GRID + c
629 |
630 |     @staticmethod
631 |     def field_of_xy(x, y):
632 |         fc = (x / Scale.FIELD_BIN).astype(np.int32)
633 |         fr = (y / Scale.FIELD_BIN).astype(np.int32)
634 |         np.clip(fc, 0, Scale.FIELD_N - 1, out=fc)
635 |         np.clip(fr, 0, Scale.FIELD_N - 1, out=fr)
636 |         return fr, fc
637 |
638 |     @staticmethod
639 |     def _diffuse(a, sigma):
640 |         from scipy.ndimage import gaussian_filter
641 |         return gaussian_filter(a, sigma=sigma, mode="nearest")
642 |
643 |     @staticmethod
644 |     def stack_of(field):
645 |         st = np.empty((8, Scale.FIELD_N, Scale.FIELD_N), np.float32)
646 |         for i, (dc, dr) in enumerate(DIR8_OFF):
647 |             st[i] = np.roll(np.roll(field, -dr, axis=0), -dc, axis=1)
648 |         return st
649 |
650 |     # -----
651 |     def update_signals(self, bfr, bfc, neut_field, mono_field, ifng_field):
652 |         p = self.p
653 |         if bfr is not None and bfr.size:
654 |             # * [축 1] fMLF: 세균이 스스로 방출한다 (표적 = 신호원)
655 |             np.add.at(self.fmlf, (bfr, bfc), p.fmlf_per_bacterium)
656 |             # * [축 2] C5a: 균 표면 보체활성화에 비례
657 |             if self.complement > 1e-6:
658 |                 np.add.at(self.c5a, (bfr, bfc),
659 |                             p.c5a_per_bacterium * float(self.complement))
660 |             # [축 3][축 4] 상피/조직 반응성 케모카인
661 |             np.add.at(self.cxcl8, (bfr, bfc), p.cxcl8_from_epi)
662 |             np.add.at(self.ccl2, (bfr, bfc), p.ccl2_from_epi)
663 |             np.add.at(self.illb, (bfr, bfc), p.illb_from_epi)
664 |             np.add.at(self.cxcr3l, (bfr, bfc), p.cxcr3l_basal)
665 |             if neut_field is not None and neut_field[0].size:
666 |                 np.add.at(self.cxcl8, neut_field, p.cxcl8_from_neut)
667 |             if mono_field is not None and mono_field[0].size:
668 |                 np.add.at(self.ccl2, mono_field, p.ccl2_from_mono)
669 |             if ifng_field is not None and ifng_field[0].size:
670 |                 np.add.at(self.ifng, ifng_field, p.ifng_from_nk_t)
671 |
672 |             # [축 5] CXCL9/10 은 IFN-gamma 유도성
673 |             self.cxcr3l += p.cxcr3l_ifng_gain * self.ifng
674 |             # [축 6] 림프질의 항상적 CXCL13
675 |             self.cxcl13[self.ln_slice] += p.cxcl13_from_ln
676 |

```

```

677 |         self.fmlf = self._diffuse(self.fmlf, p.fmlf_sigma) * (1 - p.fmlf_decay)
678 |         self.c5a = self._diffuse(self.c5a, p.c5a_sigma) * (1 - p.c5a_decay)
679 |         self.cxcl8 = self._diffuse(self.cxcl8, p.cxcl8_sigma) * (1 - p.cxcl8_decay)
680 |         self.ccl2 = self._diffuse(self.ccl2, p.ccl2_sigma) * (1 - p.ccl2_decay)
681 |         self.cxcr3l = self._diffuse(self.cxcr3l, p.cxcr3l_sigma) * (1 - p.cxcr3l_decay)
682 |         self.cxcl13 = self._diffuse(self.cxcl13, p.cxcl13_sigma) * (1 - p.cxcl13_decay)
683 |         self.illb = self._diffuse(self.illb, p.illb_sigma) * (1 - p.illb_decay)
684 |         self.ifng = self._diffuse(self.ifng, p.ifng_sigma) * (1 - p.ifng_decay)
685 |
686 |     def update_humoral(self, n_bact):
687 |         p = self.p
688 |         drive = min(1.0, n_bact / 40000.0)
689 |         self.complement += p.complement_on * drive * (1 - self.complement)
690 |         self.complement -= p.complement_off * self.complement
691 |         self.complement = float(np.clip(self.complement, 0.0, 1.0))
692 |         self.inflammation += 0.030 * min(1.0, float(self.illb.mean()) * 220.0) \
693 |             * (1 - self.inflammation)
694 |         self.inflammation -= 0.014 * self.inflammation
695 |         self.inflammation = float(np.clip(self.inflammation, 0.0, 1.0))
696 |
697 |     def kill_epithelium(self, sites):
698 |         if sites.size == 0:
699 |             return 0
700 |         sites = np.unique(sites)
701 |         ok = self.state[sites] == HEALTHY
702 |         s = sites[ok]
703 |         if s.size:
704 |             self.state[s] = DEAD
705 |             self.n_dead_epi += s.size
706 |             self.damage += s.size / max(1, self.n_target0) * 1.6
707 |         return int(s.size)
708 |
709 |     def regenerate(self, rate):
710 |         dead = np.flatnonzero(self.state == DEAD)
711 |         if dead.size == 0:
712 |             return
713 |         k = int(dead.size * rate)
714 |         if k > 0:
715 |             pick = self.rng.choice(dead, size=min(k, dead.size), replace=False)
716 |             self.state[pick] = HEALTHY
717 |
718 |
719 | # =====
720 | # SECTION 7. 세균 개체군 (비운동성 · 세포외 증식)
721 | # =====
722 |
723 |
724 | class BacteriaPopulation:
725 |     """
726 |     Streptococcus pneumoniae 개체군.
727 |     ★바이러스와 결정적으로 다른 점:
728 |     - 숙주세포 안으로 들어가지 않는다 (감염세포 상태가 존재하지 않음)
729 |     - 비운동성이라 분열한 자리 근처에 머문다 -> 미세군락(microcolony)

```

```

730 |         - 스스로 fMLF 를 방출하여 자기 위치를 광고한다
731 |         - 협막이 보체 침착을 방해하므로, 옴소닌화되어야만 잡아먹힌다
732 |         """
733 |
734 |     def __init__(self, p: Params, rng):
735 |         self.p = p
736 |         self.rng = rng
737 |         self.x = np.zeros(0, np.float32)
738 |         self.y = np.zeros(0, np.float32)
739 |         self.ops = np.zeros(0, np.float32)      # 표면 C3b/IgG 옴소닌 침착도 0~1
740 |
741 |     @property
742 |     def n(self):
743 |         return self.x.size
744 |
745 |     def seed(self, n, rng):
746 |         """비강 접종: 상피 표면 한 지점 부근에 집중 접종"""
747 |         cx, cy = 620.0, 430.0
748 |         self.x = np.clip(rng.normal(cx, 26.0, n), 0, Scale.GRID - 1).astype(np.float32)
749 |         self.y = np.clip(rng.normal(cy, 26.0, n), 0, Scale.GRID - 1).astype(np.float32)
750 |         self.ops = np.zeros(n, np.float32)
751 |
752 |     def divide(self, capacity_scale):
753 |         """이분법 분열. 딸세포는 모세포 바로 옆(비운동성)."""
754 |         if self.n == 0:
755 |             return 0
756 |         p = self.p
757 |         eff = p.div_p * max(0.0, capacity_scale)
758 |         if eff <= 0:
759 |             return 0
760 |         m = self.rng.random(self.n) < eff
761 |         k = int(np.count_nonzero(m))
762 |         if k == 0:
763 |             return 0
764 |         nx = self.x[m] + self.rng.normal(0, p.daughter_sigma, k).astype(np.float32)
765 |         ny = self.y[m] + self.rng.normal(0, p.daughter_sigma, k).astype(np.float32)
766 |         np.clip(nx, 0, Scale.GRID - 1, out=nx)
767 |         np.clip(ny, 0, Scale.GRID - 1, out=ny)
768 |         self.x = np.concatenate([self.x, nx])
769 |         self.y = np.concatenate([self.y, ny])
770 |         # 딸세포는 옴소닌을 물려받지 않는다 (새 표면)
771 |         self.ops = np.concatenate([self.ops, np.zeros(k, np.float32)])
772 |         return k
773 |
774 |     def drift(self):
775 |         """점액섬모 수송에 의한 미세 이동 (능동 운동 아님)"""
776 |         if self.n == 0:
777 |             return
778 |         s = self.p.drift_sigma
779 |         self.x += self.rng.normal(0, s, self.n).astype(np.float32)
780 |         self.y += self.rng.normal(0, s, self.n).astype(np.float32)
781 |         MovementEngine.bound(self.x, self.y)
782 |

```

```

783 | def opsonize(self, complement, ab_titer):
784 |     """보체 C3b 침착 + 항원항 IgG 결합. 험막이 이를 저해한다."""
785 |     if self.n == 0:
786 |         return
787 |     p = self.p
788 |     gain = (p.c3b_deposit_rate * complement
789 |             + p.ab_opsonin_gain * ab_titer) * (1.0 - p.capsule_resist * 0.55)
790 |     self.ops += gain * (1.0 - self.ops) - p.ops_decay * self.ops
791 |     np.clip(self.ops, 0.0, 1.0, out=self.ops)
792 |
793 | def remove(self, idx):
794 |     if idx.size == 0:
795 |         return 0
796 |     keep = np.ones(self.n, bool)
797 |     keep[idx] = False
798 |     self.x = self.x[keep]
799 |     self.y = self.y[keep]
800 |     self.ops = self.ops[keep]
801 |     return int(idx.size)
802 |
803 |
804 | # =====
805 | # SECTION 8. 면역세포 개체군
806 | # =====
807 |
808 | NEUT, MONO, NK, TH, TC, BC = 0, 1, 2, 3, 4, 5
809 | CTYPE_NAME = {NEUT: "중성구", MONO: "단핵구/대식세포", NK: "NK 세포",
810 |               TH: "조력 T 세포", TC: "살해 T 세포", BC: "B 세포"}
811 | # ★수용체는 세포마다 반드시 다르다 (사용자 요구사항)
812 | CTYPE_RECEPTOR = {
813 |     NEUT: ("FPR1+C5aR1", "CXCR2"),      # (말단표적축, 중간매개축)
814 |     MONO: ("FPR1", "CCR2"),
815 |     NK:    (None, "CXCR3"),
816 |     TH:    (None, "CXCR3"),
817 |     TC:    (None, "CXCR3"),
818 |     BC:    (None, "CXCR5"),
819 | }
820 |
821 |
822 | class CellPool:
823 |     __slots__ = ("x", "y", "hx", "hy", "ctype", "alive", "in_vessel", "age",
824 |                 "life", "capacity", "speed", "specific", "n", "cap")
825 |
826 |     def __init__(self, cap=620000):
827 |         self.cap = cap
828 |         self.n = 0
829 |         f32 = lambda: np.zeros(cap, np.float32)
830 |         self.x, self.y = f32(), f32()
831 |         self.hx, self.hy = f32(), f32()
832 |         self.speed = f32()
833 |         self.ctype = np.zeros(cap, np.uint8)
834 |         self.alive = np.zeros(cap, bool)
835 |         self.in_vessel = np.zeros(cap, bool)

```

```

836 |         self.specific = np.zeros(cap, bool)
837 |         self.age = np.zeros(cap, np.int32)
838 |         self.life = np.zeros(cap, np.int32)
839 |         self.capacity = np.zeros(cap, np.int32)
840 |
841 |     def add(self, k, ctype, x, y, speed, life, capacity, in_vessel, rng,
842 |            specific=False):
843 |         if k <= 0:
844 |             return
845 |         if self.n + k > self.cap:
846 |             k = self.cap - self.n
847 |             if k <= 0:
848 |                 return
849 |         s = slice(self.n, self.n + k)
850 |         self.x[s] = x[:k]
851 |         self.y[s] = y[:k]
852 |         th = rng.uniform(0, 2 * math.pi, k)
853 |         self.hx[s] = np.cos(th)
854 |         self.hy[s] = np.sin(th)
855 |         self.ctype[s] = ctype
856 |         self.alive[s] = True
857 |         self.in_vessel[s] = in_vessel
858 |         self.specific[s] = specific
859 |         self.age[s] = 0
860 |         self.life[s] = life
861 |         self.capacity[s] = capacity
862 |         self.speed[s] = speed
863 |         self.n += k
864 |
865 |     def compact(self):
866 |         idx = np.flatnonzero(self.alive[:self.n])
867 |         m = idx.size
868 |         for a in (self.x, self.y, self.hx, self.hy, self.speed):
869 |             a[:m] = a[idx]
870 |         for a in (self.ctype, self.alive, self.in_vessel, self.specific,
871 |                 self.age, self.life, self.capacity):
872 |             a[:m] = a[idx]
873 |         self.alive[m:self.n] = False
874 |         self.n = m
875 |
876 |
877 | # =====
878 | # SECTION 9. 림프절 (후천면역 프라이밍)
879 | # =====
880 |
881 |
882 | class LymphNode:
883 |     def __init__(self, p: Params):
884 |         self.p = p
885 |         self.antigen = 0.0
886 |         self.primed = False
887 |         self.t_primed = None
888 |         self.cd4 = 100.0

```

```

889 |         self.cd8 = 50.0
890 |         self.pc = 0.0
891 |         self.th17 = 0.0
892 |         self.igm = 0.0
893 |         self.igg = 0.0
894 |         self.clock = 0
895 |
896 |     def step(self, t, antigen_in):
897 |         p = self.p
898 |         self.antigen = self.antigen * 0.985 + antigen_in
899 |         if not self.primed and self.antigen > 60.0:
900 |             self.clock += 1
901 |             if self.clock >= p.ln_priming_ticks:
902 |                 self.primed = True
903 |                 self.t_primed = t
904 |         if self.primed:
905 |             d = (t - self.t_primed) / Scale.TICKS_PER_DAY
906 |             growth = 1.0 if d < 6.0 else 0.0
907 |             if growth:
908 |                 # 클론확장은 무한하지 않다: 전구세포 풀과 항원량에 의해 포화한다
909 |                 self.cd4 = min(self.cd4 * (1.0 + 0.0125), 60000.0)
910 |                 self.cd8 = min(self.cd8 * (1.0 + 0.0105), 30000.0)
911 |                 # *세균 점막면역의 주역은 Th17 (lu2008, zhang2018)
912 |                 self.th17 = min(self.th17 + p.th17_gain
913 |                                * (0.020 * self.cd4 - 0.010 * self.th17), 45000.0)
914 |                 self.pc = min(self.pc + 0.020 * self.cd4 - 0.006 * self.pc, 55000.0)
915 |             else:
916 |                 self.cd4 *= 0.985
917 |                 self.cd8 *= 0.985
918 |                 self.th17 *= 0.990
919 |                 self.pc *= 0.988
920 |             self.igm += p.ab_igm_rate * self.pc / 1000.0
921 |             self.igm *= 1 - p.igm_decay
922 |             if d > 3.5:
923 |                 self.igg += p.ab_igg_rate * self.pc / 1000.0
924 |                 self.igg *= 1 - p.igg_decay
925 |
926 |     @property
927 |     def titer(self):
928 |         return min(self.igm + self.igg * 1.8, 120.0)
929 |
930 |
931 | # =====
932 | # SECTION 10. 메인 시뮬레이션
933 | # =====
934 |
935 |
936 | class BacterialInfectionSim:
937 |     def __init__(self, p: Params, movement_mode: str, seed: int, verbose=False):
938 |         self.p = p
939 |         self.mode = movement_mode
940 |         self.rng = np.random.default_rng(seed)
941 |         self.verbose = verbose

```

```

942 |         self.env = Environment(p, self.rng)
943 |         self.bact = BacteriaPopulation(p, self.rng)
944 |         self.bact.seed(p.inoculum, self.rng)
945 |         self.cells = CellPool()
946 |         self.mover = MovementEngine(movement_mode, self.rng)
947 |         self.ln = LymphNode(p)
948 |         self.history: List[dict] = []
949 |
950 |         # ---- 집계 ----
951 |         self.cleared_phago_neut = 0
952 |         self.cleared_phago_mono = 0
953 |         self.cleared_complement = 0
954 |         self.cleared_antibody = 0
955 |         self.cleared_am = 0
956 |         self.dead_immune = 0
957 |         self.memory = 0
958 |         self.n_extravasated = 0
959 |         self.t_first_extrav = None
960 |         self.t_first_arrival = None
961 |         self.ctype_first_extrav = {}
962 |         self.ctype_first_arrival = {}
963 |         self.contact_events = 0
964 |         self.neut_cell_ticks = 0
965 |
966 |         self._seed_cells()
967 |
968 |         # -----
969 |         def _rand_vessel_xy(self, k):
970 |             s = self.rng.choice(self.env.vessel_sites, size=k, replace=True)
971 |             r = (s // Scale.GRID).astype(np.float32) + 0.5
972 |             c = (s - (s // Scale.GRID) * Scale.GRID).astype(np.float32) + 0.5
973 |             return c, r
974 |
975 |         def _seed_cells(self):
976 |             p, rng = self.p, self.rng
977 |             for ct, k, spd, life, cap in (
978 |                 (NEUT, p.n_neut0, p.neut_speed, p.neut_life_ticks, p.neut_capacity),
979 |                 (MONO, p.n_mono0, p.mono_speed, p.mono_life_ticks, p.mono_capacity),
980 |                 (NK, p.n_nk0, p.nk_speed, p.nk_life_ticks, 0),
981 |                 (TH, p.n_th0, p.t_speed, 100000, 0),
982 |                 (TC, p.n_tc0, p.t_speed, 100000, 0),
983 |                 (BC, p.n_b0, p.b_speed, 100000, 0)):
984 |                 x, y = self._rand_vessel_xy(k)
985 |                 self.cells.add(k, ct, x, y, spd, life, cap, True, rng)
986 |
987 |         # -----
988 |         def _axis_stacks(self, env):
989 |             """수용체별 축 스택. 말단표적축과 중간매개축을 분리해 만든다. """
990 |             end_neut = self.env.fmlf + self.env.c5a          # FPR1 + C5aR1
991 |             end_mono = self.env.fmlf                          # FPR1
992 |             return {
993 |                 "end": {NEUT: Environment.stack_of(end_neut),
994 |                     MONO: Environment.stack_of(end_mono)},

```



```

995         "mid": {NEUT: Environment.stack_of(env.cxcl8),
996                 MONO: Environment.stack_of(env.ccl2),
997                 NK: Environment.stack_of(env.cxc3l),
998                 TH: Environment.stack_of(env.cxc3l),
999                 TC: Environment.stack_of(env.cxc3l),
1000                 BC: Environment.stack_of(env.cxcl13)},
1001         "end_field": {NEUT: end_neut, MONO: end_mono},
1002         "mid_field": {NEUT: env.cxcl8, MONO: env.ccl2, NK: env.cxc3l,
1003                      TH: env.cxc3l, TC: env.cxc3l, BC: env.cxcl13},
1004     }
1005
1006     EXTRAV = {NEUT: "extrav_FPR1", MONO: "extrav_CCR2", NK: "extrav_CXCR3",
1007              TH: "extrav_CXCR3", TC: "extrav_CXCR3", BC: "extrav_CXCR5"}
1008
1009     # -----
1010     def step(self, t):
1011         p, env, c, rng = self.p, self.env, self.cells, self.rng
1012
1013         # ----- A. 세균 증식 -----
1014         occ = self.bact.n / p.carrying_capacity
1015         # ★영양 면역: 염증 강도에 비례해 실효 증식률이 감소한다
1016         nutri = 1.0 - p.nutritional_immunity * float(env.inflammation)
1017         self.bact.divide(max(0.0, (1.0 - occ) * max(0.0, nutri)))
1018         self.bact.drift()
1019
1020         # 상주 폐포대식세포에 의한 기저 제거
1021         if self.bact.n:
1022             am_eff = p.am_clearance_p / (1.0 + self.bact.n / p.am_saturation_n)
1023             m = rng.random(self.bact.n) < am_eff
1024             k = int(np.count_nonzero(m))
1025             if k:
1026                 self.cleared_am += self.bact.remove(np.flatnonzero(m))
1027
1028         # ----- B. 옵소닌화 -----
1029         self.bact.opsonize(env.complement, self.ln.titer)
1030
1031         # ----- C. 신호장 갱신 -----
1032         if self.bact.n:
1033             bfr, bfc = Environment.field_of_xy(self.bact.x, self.bact.y)
1034         else:
1035             bfr = bfc = None
1036         nn = c.n
1037         alive_idx = np.flatnonzero(c.alive[:nn])
1038         neut_f = mono_f = ifng_f = None
1039         if alive_idx.size:
1040             ct_all = c.ctype[alive_idx]
1041             tis = ~c.in_vessel[alive_idx]
1042             sel = alive_idx[(ct_all == NEUT) & tis]
1043             if sel.size:
1044                 neut_f = Environment.field_of_xy(c.x[sel], c.y[sel])
1045             sel = alive_idx[(ct_all == MONO) & tis]
1046             if sel.size:
1047                 mono_f = Environment.field_of_xy(c.x[sel], c.y[sel])

```

```

1048         sel = alive_idx[((ct_all == NK) | (ct_all == TH)) & tis]
1049         if sel.size:
1050             ifng_f = Environment.field_of_xy(c.x[sel], c.y[sel])
1051         env.update_signals(bfr, bfc, neut_f, mono_f, ifng_f)
1052         env.update_humoral(self.bact.n)
1053
1054         # ----- D. 동원 -----
1055         drive = float(np.clip(0.55 * env.inflammation
1056                               + 0.45 * min(1.0, self.bact.n / 25000.0), 0, 1))
1057         for ct, gain, spd, life, cap in (
1058             (NEUT, p.neut_recruit_gain, p.neut_speed, p.neut_life_ticks,
1059              p.neut_capacity),
1060             (MONO, p.mono_recruit_gain, p.mono_speed, p.mono_life_ticks,
1061              p.mono_capacity),
1062             (NK, p.nk_recruit_gain, p.nk_speed, p.nk_life_ticks, 0)):
1063             g = gain
1064             if ct == NEUT and self.ln.th17 > 0:
1065                 g = gain * (1.0 + min(1.4, self.ln.th17 / 120.0))
1066             k = int(g * drive / Scale.TICKS_PER_DAY)
1067             if k > 0:
1068                 x, y = self._rand_vessel_xy(k)
1069                 c.add(k, ct, x, y, spd, life, cap, True, rng)
1070
1071         # 후천면역 조직 진입
1072         if self.ln.primed:
1073             # *세포외 세균에서는 Th17(CD4)이 주역이다. CTL은 죽일 감염세포가
1074             # 없으므로 조직 진입이 미미하다(lu2008, zhang2018).
1075             k_th = int(self.ln.th17 * 0.00120)
1076             k_tc = int(self.ln.cd8 * 0.00012)
1077             if k_th > 0:
1078                 x, y = self._rand_vessel_xy(k_th)
1079                 c.add(k_th, TH, x, y, p.t_speed, p.teff_life_ticks, 0, True,
1080                      rng, specific=True)
1081             if k_tc > 0:
1082                 x, y = self._rand_vessel_xy(k_tc)
1083                 c.add(k_tc, TC, x, y, p.t_speed, p.teff_life_ticks, 0, True,
1084                      rng, specific=True)
1085
1086         # ----- E. 수명 -----
1087         nn = c.n
1088         if nn:
1089             al = c.alive[:nn]
1090             c.age[:nn][al] += 1
1091             died = al & (c.age[:nn] >= c.life[:nn])
1092             k = int(np.count_nonzero(died))
1093             if k:
1094                 c.alive[:nn][died] = False
1095                 self.dead_immune += k
1096             if nn > 20000 and (c.alive[:nn].mean() < 0.93
1097                               or nn > int(0.55 * c.cap)):
1098                 c.compact()
1099
1100         # ----- F. *이동 -----

```

```

1101 | stacks = self._axis_stacks(env)
1102 | nn = c.n
1103 | idx = np.flatnonzero(c.alive[:nn])
1104 | if idx.size:
1105 |     fr, fc = Environment.field_of_xy(c.x[idx], c.y[idx])
1106 |     ct_all = c.ctype[idx]
1107 |     self.neut_cell_ticks += int(np.count_nonzero(ct_all == NEUT))
1108 |
1109 |     # 각 세포가 '자기 수용체 축'에서 느끼는 신호 세기
1110 |     sense = np.zeros(idx.size, np.float32)
1111 |     for ctv in (NEUT, MONO, NK, TH, TC, BC):
1112 |         m = ct_all == ctv
1113 |         if not m.any():
1114 |             continue
1115 |         v = stacks["mid_field"][ctv][fr[m], fc[m]]
1116 |         if ctv in stacks["end_field"]:
1117 |             v = v + stacks["end_field"][ctv][fr[m], fc[m]] * 40.0
1118 |         sense[m] = v
1119 |
1120 |     in_v = c.in_vessel[idx]
1121 |     thr = np.zeros(idx.size, np.float32)
1122 |     for ctv in (NEUT, MONO, NK, TH, TC, BC):
1123 |         m = ct_all == ctv
1124 |         if m.any():
1125 |             thr[m] = getattr(p, self.EXTRAV[ctv])
1126 |     ex = in_v & (sense > thr)
1127 |     if ex.any():
1128 |         gi = idx[ex]
1129 |         c.in_vessel[gi] = False
1130 |         self.n_extravasated += gi.size
1131 |         if self.t_first_extrav is None:
1132 |             self.t_first_extrav = t
1133 |         for ctv in np.unique(c.ctype[gi]):
1134 |             ctv = int(ctv)
1135 |             if ctv not in self.ctype_first_extrav:
1136 |                 self.ctype_first_extrav[ctv] = t
1137 |         in_v = c.in_vessel[idx]
1138 |
1139 |     for ctv in (NEUT, MONO, NK, TH, TC, BC):
1140 |         m = ct_all == ctv
1141 |         if not m.any():
1142 |             continue
1143 |         gi = idx[m]
1144 |         iv = in_v[m]
1145 |         es = stacks["end"].get(ctv)
1146 |         ms = stacks["mid"][ctv]
1147 |         for invessel in (True, False):
1148 |             sub = gi[iv == invessel] if invessel else gi[~iv]
1149 |             if sub.size == 0:
1150 |                 continue
1151 |             xs, ys = c.x[sub].copy(), c.y[sub].copy()
1152 |             hxs, hys = c.hx[sub].copy(), c.hy[sub].copy()
1153 |             sfr, sfc = Environment.field_of_xy(xs, ys)

```

```

1154         spd = (p.vessel_speed if invessel else c.speed[sub])
1155         self.mover.step(xs, ys, hxs, hys, spd, es, ms, sfr, sfc, p)
1156         c.x[sub], c.y[sub] = xs, ys
1157         c.hx[sub], c.hy[sub] = hxs, hys
1158
1159     # ----- G. 식균작용 -----
1160     # 식세포 1개는 한 번의 조우에서 여러 균을 연속으로 삼킬 수 있다.
1161     # (호중구 포식 용량 약 50 CFU/세포, rubio2023)
1162     # 이를 R 회 라운드 로 구현한다.
1163     PHAGO_ROUNDS = 5
1164     nn = c.n
1165     idx = np.flatnonzero(c.alive[:nn])
1166     if idx.size and self.bact.n:
1167         csite = Environment.site_of(c.x[idx], c.y[idx])
1168         _sel_cache = {}
1169         for ctv, _b in ((NEUT, p.neut_phago_p), (MONO, p.mono_phago_p)):
1170             _sel_cache[ctv] = np.flatnonzero((c.ctype[idx] == ctv)
1171                                               & (~c.in_vessel[idx]))
1172         for _round in range(PHAGO_ROUNDS):
1173             if self.bact.n == 0:
1174                 break
1175             bsite = Environment.site_of(self.bact.x, self.bact.y)
1176             eaten_all = []
1177             for ctv, base in ((NEUT, p.neut_phago_p), (MONO, p.mono_phago_p)):
1178                 sel0 = _sel_cache[ctv]
1179                 if sel0.size == 0:
1180                     continue
1181                 sel = sel0[c.capacity[idx[sel0]] > 0]
1182                 if sel.size == 0:
1183                     continue
1184                 loc, bi = self._match_site(csite[sel], bsite)
1185                 if loc.size == 0:
1186                     continue
1187                 if _round == 0:
1188                     self.contact_events += loc.size
1189                 # ★협막 저항: 옴소닌화 정도가 요구치를 넘어야 포식 가능
1190                 # (gordon1980: 폐렴사슬알균은 40% 이상 혈청 필요)
1191                 ops = self.bact.ops[bi]
1192                 eff = base * np.clip(
1193                     (ops - p.opsonin_need * 0.35)
1194                     / max(1e-6, 1 - p.opsonin_need * 0.35), 0, 1)
1195                 ok = rng.random(loc.size) < eff
1196                 if not ok.any():
1197                     continue
1198                 gi = idx[sel[loc[ok]]]
1199                 tb = bi[ok]
1200                 uq, first = np.unique(tb, return_index=True)
1201                 gi = gi[first]
1202                 np.subtract.at(c.capacity, gi, 1)
1203                 if ctv == NEUT:
1204                     self.cleared_phago_neut += uq.size
1205                 else:
1206                     self.cleared_phago_mono += uq.size

```

```

1207         eaten_all.append(uq)
1208     if eaten_all:
1209         self.bact.remove(np.unique(np.concatenate(eaten_all)))
1210     else:
1211         break
1212
1213 # ----- H. 항체/보체 직접 사멸 -----
1214 if self.bact.n:
1215     # 그람양성균은 두꺼운 펩티도글리칸과 험막 때문에 보체 MAC 에 저항한다.
1216     # 따라서 항체/보체의 기여는 '직접 살균' 이 아니라 '옵소닌화' 가 본질이며,
1217     # 여기서는 미미한 직접 효과만 남긴다. (hyams2010, gordon1980)
1218     # ★항협막 IgG: 폐렴구균 방어의 핵심 기전이자 폐렴구균 백신의 원리.
1219     #   옵소닌화에 더해 응집(agglutination)을 일으켜 점액섬모 수송으로
1220     #   제거되게 하므로, 식세포와의 조우 여부와 무관하게 작용한다.
1221     ti = self.ln.titer
1222     if ti > 0.02:
1223         m = rng.random(self.bact.n) < min(0.050, 0.00042 * ti)
1224         if m.any():
1225             self.cleared_antibody += self.bact.remove(np.flatnonzero(m))
1226     if env.complement > 0.35 and self.bact.n:
1227         m = rng.random(self.bact.n) < 0.00016 * env.complement
1228         if m.any():
1229             self.cleared_complement += self.bact.remove(np.flatnonzero(m))
1230
1231 # ----- I. 조직 손상 -----
1232 if self.bact.n:
1233     m = rng.random(self.bact.n) < p.epi_damage_p
1234     if m.any():
1235         env.kill_epithelium(Environment.site_of(self.bact.x[m],
1236                                             self.bact.y[m]))
1237
1238 # 호중구 매개 부수적 손상
1239 if idx.size:
1240     tis = idx[~c.in_vessel[idx]]
1241     if tis.size:
1242         nsel = tis[c.ctype[tis] == NEUT]
1243         if nsel.size:
1244             m = rng.random(nsel.size) < 4.0e-6 * (1 + 3 * env.inflammation)
1245             if m.any():
1246                 env.kill_epithelium(
1247                     Environment.site_of(c.x[nsel[m]], c.y[nsel[m]]))
1248 env.regenerate(0.0022 * (1 - env.inflammation))
1249
1250 # ----- J. 도착 기록 -----
1251 if idx.size and self.bact.n:
1252     tis = idx[~c.in_vessel[idx]]
1253     if tis.size:
1254         cs = Environment.site_of(c.x[tis], c.y[tis])
1255         bs = Environment.site_of(self.bact.x, self.bact.y)
1256         loc, _ = self._match_site(cs, bs)
1257         if loc.size:
1258             if self.t_first_arrival is None:
1259                 self.t_first_arrival = t
1260             for ctv in np.unique(c.ctype[tis[loc]]):

```

```

1260         ctv = int(ctv)
1261         if ctv not in self.ctype_first_arrival:
1262             self.ctype_first_arrival[ctv] = t
1263
1264         # ----- K. 림프절 -----
1265         self.ln.step(t, 0.0045 * self.bact.n)
1266         if self.ln.primed and self.memory == 0:
1267             d = (t - self.ln.t_primed) / Scale.TICKS_PER_DAY
1268             if d > 8.0:
1269                 self.memory = int((self.ln.cd8 + self.ln.cd4) * p.memory_fraction)
1270
1271         # -----
1272         @staticmethod
1273         def _match_site(a_sites, b_sites):
1274             """같은 격자칸에 있는 (a 인덱스, b 인덱스) 쌍을 찾는다."""
1275             if a_sites.size == 0 or b_sites.size == 0:
1276                 return np.empty(0, np.int64), np.empty(0, np.int64)
1277             order = np.argsort(b_sites, kind="stable")
1278             bs = b_sites[order]
1279             lo = np.searchsorted(bs, a_sites, "left")
1280             hi = np.searchsorted(bs, a_sites, "right")
1281             has = hi > lo
1282             ai = np.flatnonzero(has)
1283             if ai.size == 0:
1284                 return np.empty(0, np.int64), np.empty(0, np.int64)
1285             bi = order[lo[ai]]
1286             return ai, bi
1287
1288         # -----
1289         def snapshot(self, t):
1290             c, env = self.cells, self.env
1291             n = c.n
1292             al = c.alive[:n]
1293             ct = c.ctype[:n]
1294             cnt = lambda k: int(np.count_nonzero(al & (ct == k)))
1295             return {
1296                 "tick": t, "day": t / Scale.TICKS_PER_DAY,
1297                 "bacteria": self.bact.n,
1298                 "opsonized_frac": float(self.bact.ops.mean()) if self.bact.n else 0.0,
1299                 "healthy": int(np.count_nonzero(env.state == HEALTHY)),
1300                 "dead_epi": env.n_dead_epi,
1301                 "neutrophil": cnt(NEUT), "monocyte": cnt(MONO), "nk": cnt(NK),
1302                 "helper_t": cnt(TH), "killer_t": cnt(TC), "bcell": cnt(BC),
1303                 "in_tissue": int(np.count_nonzero(al & (~c.in_vessel[:n]))),
1304                 "memory": self.memory,
1305                 "igm": self.ln.igm, "igg": self.ln.igg, "antibody": self.ln.titer,
1306                 "th17": self.ln.th17, "ln_cd8": self.ln.cd8, "ln_pc": self.ln.pc,
1307                 "fmlf": float(env.fmlf.mean()), "fmlf_max": float(env.fmlf.max()),
1308                 "c5a": float(env.c5a.mean()),
1309                 "cxcl8": float(env.cxcl8.mean()), "ccl2": float(env.ccl2.mean()),
1310                 "cxcr3l": float(env.cxcr3l.mean()), "ifng": float(env.ifng.mean()),
1311                 "il1b": float(env.il1b.mean()),
1312                 "complement": env.complement, "inflammation": env.inflammation,

```

```

1313 |         "damage": env.damage,
1314 |         "cleared_phago": self.cleared_phago_neut + self.cleared_phago_mono,
1315 |         "cleared_phago_neut": self.cleared_phago_neut,
1316 |         "cleared_phago_mono": self.cleared_phago_mono,
1317 |         "cleared_am": self.cleared_am,
1318 |         "cleared_complement": self.cleared_complement,
1319 |         "cleared_antibody": self.cleared_antibody,
1320 |         "dead_immune": self.dead_immune,
1321 |         "n_extravasated": self.n_extravasated,
1322 |         "contact_events": self.contact_events,
1323 |     }
1324 |
1325 | def run(self):
1326 |     total = self.p.max_days * Scale.TICKS_PER_DAY
1327 |     self.history.append(self.snapshot(0))
1328 |     for t in range(1, total + 1):
1329 |         self.step(t)
1330 |         if t % Scale.TICKS_PER_DAY == 0:
1331 |             s = self.snapshot(t)
1332 |             self.history.append(s)
1333 |             if self.verbose:
1334 |                 print(f"   Day {s['day']:5.1f}  bact={s['bacteria']:>8,} "
1335 |                       f"neut={s['neutrophil']:>7,} H={s['healthy']:>7,} "
1336 |                       f"fmlf={s['fmlf']:.5f} comp={s['complement']:.3f}",
1337 |                       flush=True)
1338 |             if self.bact.n == 0 and t > Scale.TICKS_PER_DAY:
1339 |                 self.history.append(self.snapshot(t))
1340 |                 break
1341 |     return self.history
1342 |

```

부록 D. 페렴사슬알균 반복 시행 실행 스크립트

서로 다른 난수 시드로 각 조건을 5 회씩 독립 시행하고, 감염이 완전히 종식될 때까지 실행한 뒤 측정 지표를 산출·저장하는 스크립트.

파일명	run_replicates_pneumococcus.py	총 줄 수	50 줄
실행 환경	Python 3.12 / NumPy · SciPy · Matplotlib	난수 시드	700000, 700037, 700074, 700111, 700148

```
1 | import sys,os,json,time
2 | import numpy as np
3 | sys.path.insert(0, '/home/claude/bact')
4 | import immune_bacteria_abm as M
5 | OUT="/home/claude/bact/rep_full.json"; NREP=5; DAYS=20
6 | def one(mode,seed):
7 |     p=M.Params(); p.max_days=DAYS; p.seed=seed
8 |     s=M.BacterialInfectionSim(p,mode,seed=seed,verbose=False); s.run()
9 |     h=s.history; d=np.array([x["day"] for x in h]); b=np.array([x["bacteria"] for x in h],float)
10 |    cl=[x["day"] for x in h if x["bacteria"]==0]
11 |    c=s.cells; n=c.n; al=c.alive[:n]; tis=al&(~c.in_vessel[:n])
12 |    ph=tis&((c ctype[:n]==0)|(c ctype[:n]==1)); B=32
13 |    j=int(np.argmax(b))
14 |    # 겹침도: 정점 시점 대신 전체 기간 대표값 필요 -> 마지막 비영(非零) 시점 사용 불가하므로 정점일 재현 불가
15 |    hp=np.histogram2d(c.y[:n][ph],c.x[:n][ph],bins=B,range=[[0,1000],[0,1000]])[0]
16 |    return {"mode":mode,"seed":seed,"daily":h,
17 |          "clear_day": float(cl[0]) if cl else None,
18 |          "bact_peak":float(b.max()),"bact_peak_day":float(d[j]),
19 |          "bact_auc":float(np.trapezoid(b,d)),
20 |          "healthy_min":float(min(x["healthy"] for x in h)),
21 |          "dead_epi":float(h[-1][["dead_epi"]]),"damage":float(h[-1][["damage"]]),
22 |          "phago_neut":s.cleared_phago_neut,"phago_mono":s.cleared_phago_mono,
23 |          "phago_total":s.cleared_phago_neut+s.cleared_phago_mono,
24 |          "cleared_am":s.cleared_am,"cleared_ab":s.cleared_antibody,
25 |          "cleared_comp":s.cleared_complement,
26 |          "dead_immune":s.dead_immune,"n_extrav":s.n_extravasated,
27 |          "contacts":s.contact_events,
28 |          "neut_peak":float(max(x["neutrophil"] for x in h)),
29 |          "mono_peak":float(max(x["monocyte"] for x in h)),
30 |          "ctl_peak":float(max(x["killer_t"] for x in h)),
31 |          "inflam_peak":float(max(x["inflammation"] for x in h)),
32 |          "comp_peak":float(max(x["complement"] for x in h)),
33 |          "ab_peak":float(max(x["antibody"] for x in h)),
34 |          "grad_frac":float(s.mover.n_gradient_guided/max(1,s.mover.n_move_calls)),
35 |          "endtarget_frac":float(s.mover.n_endtarget_led/max(1,s.mover.n_move_calls))}
36 | def main():
37 |     bud=float(os.environ.get("BUDGET","150"))
38 |     res=json.load(open(OUT)) if os.path.exists(OUT) else {"random":[],"systematic":[]}
39 |     t0=time.time()
40 |     for i in range(NREP):
41 |         for m in ("systematic","random"):
42 |             if len(res[m])>i: continue
```



```

43 |         if time.time()-t0>bud:
44 |             print("PAUSE s=%d r=%d"%(len(res["systematic"]),len(res["random"]))); return
45 |             r=one(m,700000+i*37); res[m].append(r); json.dump(res,open(OUT,"w"))
46 |             print("[%5.0fs] %-11s rep%d clear=%s peak=%.0f"%(
47 |                 time.time()-t0,m,i+1,r["clear_day"],r["bact_peak"]),flush=True)
48 |         print("ALL DONE")
49 |     main()
50 |

```

부록 E. 인플루엔자 A 반복 시행 실행 스크립트

인플루엔자 A 모형의 두 조건을 각각 5 회씩 독립 시행하고 결과를 저장하는 스크립트.

파일명	run_replicates_influenza.py	총 줄 수	44 줄
실행 환경	Python 3.12 / NumPy · SciPy · Matplotlib	난수 시드	700000, 700037, 700074, 700111, 700148

```
1 | import sys,os,json,time,math
2 | import numpy as np
3 | sys.path.insert(0,"/home/claude/immune"); sys.path.insert(0,"/home/claude/sys")
4 | N=5; BASE=700000; OUT="/home/claude/rep/rep.json"
5 | KEYS=["virus_peak","virus_peak_day","virus_t50","virus_shed_end_lpct",
6 | "virus_clear_day","inf_peak","inf_clear_day","total_infections",
7 | "max_target_depletion","damage_final","neut_peak","mono_peak","nk_peak",
8 | "inflam_peak","ctl_peak","ab_peak","memory_final","R0_est",
9 | "ctl_contact_per_day","killed_by_nk","killed_by_ctl","cleared_by_phago",
10 | "apoptosis","n_extravasated","grad_frac","t_first_arrival"]
11 | def extra(sim,R):
12 |     h=sim.history; d=np.array([x["day"] for x in h])
13 |     o={k:(None if (isinstance(R.get(k),float) and R.get(k) is not None and math.isnan(R.get(k))) else R.get(k)) for k in KEYS}
14 |     o["virus_auc"]=float(np.trapezoid(np.array([x["virus"] for x in h],float),d))
15 |     o["min_healthy"]=float(min(x["healthy"] for x in h))
16 |     o["dead_immune"]=float(h[-1]["dead_immune"]); return o
17 | def rr(seed):
18 |     import immune_random_walk_abm as M
19 |     p=M.Params(); p.max_days=30; p.seed=seed
20 |     s=M.ImmuneSimulation(p,movement_mode="random",seed=seed,verbose=False); s.run()
21 |     R=M.analyze(s); ch=M.validate(s,R); o=extra(s,R)
22 |     o["match_rate"]=(sum(c.verdict=="일치" for c in ch)+sum(c.verdict=="근접" for c in ch))/len(ch)*100; return o
23 | def rs(seed):
24 |     import immune_systematic_abm as M
25 |     p=M.Params(); p.max_days=30; p.seed=seed
26 |     s=M.SystematicImmuneSimulation(p,seed=seed,verbose=False); s.run()
27 |     R=M.analyze(s); ch=M.validate(R); o=extra(s,R)
28 |     o["match_rate"]=(sum(c.verdict=="일치" for c in ch)+sum(c.verdict=="근접" for c in ch))/len(ch)*100; return o
29 | def main():
30 |     b=float(os.environ.get("BUDGET","210"))
31 |     res=json.load(open(OUT,encoding="utf-8")) if os.path.exists(OUT) else {"random":[],"systematic":[]}
32 |     t0=time.time()
33 |     for i in range(N):
34 |         sd=BASE+i*37
35 |         for key,fn in (("random",rr),("systematic",rs)):
36 |             if len(res[key])>i: continue
37 |             if time.time()-t0>b:
38 |                 print(f"STOP r={len(res['random'])} s={len(res['systematic'])}"); return
39 |             r=fn(sd); res[key].append(r)
40 |             json.dump(res,open(OUT,"w",encoding="utf-8"),ensure_ascii=False)
41 |             print(f"[{time.time()-t0:6.1f}s] {key[:6].upper()} {i+1}/{N} clear={r['virus_clear_day']:.2f}",flush=True)
42 |     print("DONE")
```

```
43 | main()
44 |
```